

Chapter 1

INTRODUCTION

1.1 BACKGROUND

Modern infrastructure systems—including bridges, tunnels, highways, dams, and high-rise buildings—serve as vital pillars of urban and rural development. These structures are subjected to continuous stress from environmental conditions such as temperature variations, humidity, and seismic activity, as well as mechanical loads from usage and aging materials. Over time, these stressors can result in surface and sub-surface cracks, which are among the earliest indicators of structural fatigue and potential failure. Left undetected, such cracks can propagate, leading to critical failures with catastrophic consequences.

Historically, structural health monitoring (SHM) has relied heavily on manual inspection techniques such as visual surveys, ultrasonic testing, radiography, and thermographic imaging. These methods, while useful, are constrained by their dependency on human judgment, high labor intensity, cost, and limitations in scalability. Additionally, many of these methods require physical access to challenging or dangerous locations, delaying crucial maintenance interventions.

In recent years, the advent of deep learning and computer vision has revolutionized the domain of automated damage detection. Object detection models, particularly the YOLO (You Only Look Once) family, have emerged as powerful tools for detecting cracks in infrastructure images due to their ability to deliver real-time predictions with high accuracy. Unlike two-stage detectors, YOLO models perform localization and classification in a single forward pass, making them suitable for real-world deployment where rapid decision-making is essential.

The YOLO architecture has undergone significant architectural enhancements across versions. From YOLOv8, which brought efficiency improvements and anchor-free detection, to YOLOv11, which integrates lightweight transformer modules and better generalization techniques, each version introduces novel capabilities that merit performance comparison. Despite these developments, limited research has benchmarked these newer YOLO variants on crack detection datasets or investigated the influence of different optimizers such as SGD (Stochastic Gradient Descent) and AdamW (Adam with decoupled weight decay) on their training outcomes.

This research addresses these gaps by conducting a comparative study of YOLOv8 through YOLOv11, each tested across multiple model scales (nano, small, medium) and trained using both AdamW and SGD optimizers. The objective is to determine the most efficient model configuration for deployment on UAVs (Unmanned Aerial Vehicles) and surveillance cameras, supporting real-time, scalable, and accurate infrastructure monitoring systems.

1.2 MOTIVATIONS

The motivation for this research stems from the growing global concern over deteriorating infrastructure and the preventable tragedies that arise from undetected structural failures. Numerous high-profile incidents underscore the urgent need for intelligent, real-time structural health monitoring systems:

Table 1.1 Notable Incidents

Incident	Location	Year
Morandi Bridge Collapse	Genoa, Italy	2018
Champlain Towers South Collapse	Surfside, Florida, USA	2021
Kolkata Flyover Collapse	Kolkata, India	2016
Mumbai Footbridge Collapse	Mumbai, India	2019
Silkyara Bend–Barkot Tunnel Collapse	Uttarakhand, India	2023
I-40 Hernando DeSoto Bridge Fracture	Mississippi River, USA	2021

These cases resulted in loss of life, economic setbacks, and public distrust, often attributed to delayed or inadequate structural assessments. The common thread across these incidents is the failure to detect or act upon early warning signs, such as the presence of cracks and other forms of material distress.

In light of these events, AI-driven systems provide a compelling alternative. However, significant research gaps persist:

- **Model Generalization:** Most crack detection studies rely on legacy YOLO versions (v3–v7), overlooking advancements introduced in YOLOv8–YOLOv11.
- **Optimizer Sensitivity:** Default optimizers are typically used without testing alternative gradient-based strategies that may yield better convergence or detection robustness.
- **Environmental Complexity:** Crack datasets often lack diversity in lighting, texture, and surface geometry, affecting model reliability in practical scenarios.
- **Limited Real-World Deployment Research:** Few implementations demonstrate integration with UAVs or edge devices like surveillance cameras under bandwidth and latency constraints.

Thus, this research is driven by the need to identify the most suitable YOLO model variant and training configuration for real-world deployment, ensuring the early and reliable detection of cracks while minimizing false positives. The use of UAVs and fixed surveillance infrastructure as deployment platforms further strengthens the practical value of the study.

1.3 SCOPE OF THE PROJECT

The primary scope of this project is to evaluate and benchmark the performance of four advanced versions of the YOLO (You Only Look Once) object detection architecture—YOLOv8, YOLOv9, YOLOv10, and YOLOv11—specifically for the task of surface crack detection in civil structures. Rather than focusing on deployment or integration with UAVs or embedded systems, the study remains confined to a comparative analysis of model performance on a curated and well-annotated dataset, with the objective of identifying the most suitable model architecture for future use.

To ensure fairness and consistency in evaluation, all four YOLO versions will be trained on the same dataset sourced from Roboflow, which includes 4029 images of annotated surface cracks. The dataset has been carefully split into training (92%), validation (5%), and testing (3%) subsets. Each image in the dataset has undergone preprocessing, including resizing to a fixed dimension of 416×416 pixels. Additionally, multiple augmentation techniques—such as horizontal flips, random rotations, zoom cropping, and adjustments to hue, brightness, and saturation—have been applied. These augmentations simulate real-world environmental conditions like poor lighting, varied surface textures, and angled camera views, which are common challenges in real crack detection scenarios.

The study also seeks to explore the effect of optimizer selection on model performance. Two popular optimization algorithms—Stochastic Gradient Descent (SGD) and AdamW—will be employed across different training runs for each YOLO variant. This will help evaluate how different optimizers influence convergence speed, generalization ability, and final detection accuracy. By doing so, the project not only identifies the best-performing YOLO version but also sheds light on which training configuration is most effective for this type of visual inspection task.

The evaluation metrics for comparison will include Mean Average Precision (mAP) at various IoU thresholds, precision, recall, inference time per image, false positive/negative rates, and overall model size and resource consumption. These metrics will help in determining the trade-offs between accuracy and efficiency for each version. For instance, a slightly less accurate model with faster inference speed and lower memory footprint might be more suitable in future edge-device implementations, even if deployment is not the current focus.

Lastly, the outcomes of this project are expected to provide clear, evidence-backed recommendations on which YOLO architecture is best suited for crack detection tasks. These insights can serve as a baseline for future implementations, such as integration with UAVs, robotic crawlers, or smart surveillance systems. By narrowing down the model choice and training configuration, this project lays the groundwork for robust, real-time structural health monitoring solutions that can scale effectively in the real world.

Chapter 2

PROJECT DESCRIPTION AND GOALS

2.1 LITERATURE REVIEW

The transition from traditional, manual infrastructure inspection to automated, deep learning-based crack detection systems marks a significant advancement in Structural Health Monitoring (SHM). Initially, visual inspections, though fundamental, suffered from inherent limitations such as subjectivity, time consumption, and inaccessibility of certain structural areas. These shortcomings spurred the exploration of computer vision techniques, leading to the adoption of Convolutional Neural Networks (CNNs). CNNs, with their ability to automatically extract and learn complex features from image data, revolutionized object detection tasks, including crack identification. Within this domain, the You Only Look Once (YOLO) family of models has emerged as a frontrunner, owing to its real-time processing capabilities and high accuracy.

The YOLO architecture, a single-stage detector, streamlines the object detection process by performing classification and localization simultaneously, thereby offering a significant speed advantage over two-stage detectors like Faster R-CNN. Over the years, YOLO models have undergone substantial evolution, with each iteration introducing architectural refinements aimed at enhancing detection accuracy, speed, and robustness. For instance, YOLOv4 incorporated CSPNet, Spatial Pyramid Pooling, and Path Aggregation Networks, resulting in notable improvements in crack detection accuracy. Subsequent versions, such as YOLOv5, demonstrated further advancements in feature extraction and processing speed, establishing YOLO as a highly effective tool for real-time infrastructure monitoring. Recent advancements, including YOLOv8, YOLOv9, YOLOv10, and YOLOv11, have focused on addressing challenges related to small object detection, computational efficiency, and performance in complex environments, making them particularly relevant for UAV-based crack detection.

The performance of YOLO models is intrinsically linked to the quality and diversity of training datasets. High-resolution images with varied crack patterns, orientations, and sizes, captured under diverse lighting conditions, are crucial for training robust models. To mitigate overfitting and enhance generalization, data augmentation techniques, such as rotation, flipping, scaling, and color adjustments, are employed. Furthermore, transfer learning, which involves fine-tuning pre-trained models on crack-specific datasets, accelerates the training process and improves model performance, especially when training data is limited. However, challenges persist, including the occurrence of false positives and false negatives, variations in environmental conditions, computational resource constraints, and the labor-intensive nature of data annotation.

The choice of optimization algorithms plays a pivotal role in the training of deep learning models. Stochastic Gradient Descent (SGD) and AdamW are two widely used optimizers, each with distinct characteristics. SGD, known for its simplicity and stability, is effective for large datasets but may converge slowly. AdamW, on the other hand, incorporates a decoupled weight decay mechanism, often leading to faster convergence and improved generalization. Comparative studies on these optimizers in

the context of YOLO-based crack detection are limited, highlighting a critical research gap. Future directions in this field include the exploration of multi-modal detection systems, the development of lightweight YOLO models for edge devices, and the integration of UAVs for large-scale infrastructure inspections.

2.2 RESEARCH GAP

While existing research on YOLO-based crack detection has shown promising results, several gaps remain in the current state of the field that must be addressed to optimize crack detection systems for real-world applications. These gaps are critical for improving the accuracy, scalability, and efficiency of automated crack detection in infrastructure, particularly in large-scale and complex environments. The following are the key gaps identified in the literature:

- **Inconsistent Performance Across Different Environments:** A common challenge in deploying YOLO models for crack detection is the variability in performance across different environments. Cracks in concrete structures may vary in size, orientation, and appearance depending on factors like lighting conditions, surface texture, and occlusions from debris or other objects. Current research tends to focus on datasets captured under controlled conditions, which may not reflect real-world challenges such as poor lighting, extreme weather conditions, or high noise levels. The generalizability of YOLO models across these diverse conditions remains a significant concern.
- **False Positives and False Negatives:** Despite the advancements in YOLO models, false positives (incorrectly detecting a crack where there is none) and false negatives (failing to detect an actual crack) continue to be a problem, especially when cracks are small, faint, or partially occluded. This issue is particularly problematic in safety-critical applications like infrastructure monitoring, where missed cracks or incorrectly identified damage could lead to catastrophic failures. Previous research has not sufficiently addressed this issue, and there is a need for more robust algorithms that can better detect minor cracks while reducing the occurrence of false detections.
- **Lack of Adequate and Diverse Datasets:** The success of deep learning models such as YOLO depends heavily on the quality and diversity of the datasets used for training. While there are several available datasets for crack detection, they are often limited in terms of image variety, environmental conditions, or crack types. Many existing datasets feature cracks that are uniform in size and shape, collected under ideal lighting conditions. However, real-world infrastructure exhibits cracks of varying sizes, orientations, shapes, and severity, often in complex and hard-to-capture settings. This lack of diversity in training datasets results in models that perform well in specific scenarios but struggle to generalize in others.

Moreover, many datasets are not annotated consistently, which further complicates model training and evaluation.

- **Computational and Hardware Constraints:** While YOLO is known for its real-time detection capabilities, more advanced versions of YOLO (such as YOLOv8 and YOLOv9) require significant computational resources, including high-performance GPUs. This poses a challenge for the deployment of YOLO-based crack detection systems in resource-constrained environments, such as UAVs (Unmanned Aerial Vehicles), mobile devices, or edge devices. Many studies have not sufficiently explored ways to optimize YOLO models to make them lightweight and capable of running on devices with limited computational power. Additionally, YOLO's high computational demand makes it less practical for large-scale infrastructure inspections where real-time analysis of vast datasets is required.
- **Integration with Other Technologies for Enhanced Detection:** While YOLO models have demonstrated strong performance in detecting visible cracks, there is limited research on the integration of YOLO with other sensing technologies to improve detection in environments where cracks may not be visible or detectable by traditional optical means. For example, thermal imaging, infrared sensing, and laser scanning can detect cracks that are not visible to the naked eye but are critical to structural health. Multi-modal detection systems that combine YOLO with these other technologies are largely unexplored in the context of infrastructure crack detection. Such integrations could greatly enhance the detection capabilities, particularly in challenging environments, like historical structures or complex, multi-layered surfaces.
- **Lack of Real-World Deployment and Long-Term Evaluation:** Although many studies have demonstrated the potential of YOLO models for crack detection in controlled environments or datasets, there is a notable lack of research on the real-world deployment of these models for long-term monitoring of infrastructure. The field validation of YOLO models in diverse, real-world conditions is essential to assess the models' reliability, accuracy, and resilience over time. Existing research is often limited to short-term studies or lab-based simulations, which may not reflect the complexities of deploying these models at scale in large, dynamic environments.
- **Insufficient Focus on Cross-Version Comparisons and Optimizer Evaluation:** While YOLO versions have seen significant improvements with each iteration, there is a gap in research focused on cross-version comparisons in the context of crack detection. Many studies focus on a single version (e.g., YOLOv5 or YOLOv7), but there is a lack of comprehensive studies comparing the performance of multiple versions (YOLOv8, YOLOv9, YOLOv10, etc.) across a variety of crack types and datasets. Similarly, while optimization algorithms such as Stochastic

Gradient Descent (SGD) and AdamW have been explored in deep learning, their comparative performance in YOLO-based crack detection models remains underexplored. There is a need for further research into the choice of optimizers and their impact on convergence speed, accuracy, and robustness in crack detection tasks.

- **Limited Exploration of Transfer Learning and Fine-Tuning:** Transfer learning has shown significant potential for improving model performance, especially when the available training data is limited. However, there is limited research on transfer learning in the context of YOLO-based crack detection. Pre-trained models (trained on general object detection tasks) could be fine-tuned for crack detection tasks, allowing models to learn faster and generalize better from smaller datasets. Although some research has explored transfer learning for crack detection, there is a lack of studies that rigorously evaluate the effectiveness of various pre-training strategies and fine-tuning techniques across multiple YOLO versions.

2.3 OBJECTIVES

The primary aim of this project is to conduct a comprehensive comparative analysis of various versions of the YOLO (You Only Look Once) object detection model to determine the most suitable architecture for detecting structural cracks in images of civil infrastructure. While crack detection using deep learning has been extensively explored in recent years, the rapid evolution of YOLO models from v1 through to v9 (and potentially beyond) necessitates an in-depth investigation into their comparative performance, efficiency, and adaptability to specific use-cases like infrastructure monitoring. This project does not aim to implement the models in a deployed system but instead to critically evaluate them based on established benchmarks, academic literature, open-source experiments, and performance metrics reported by the community.

The following objectives have been identified to guide the scope and depth of this research:

- To Understand the Evolution and Architecture of YOLO Models
 - Analyze the architectural differences, design philosophies, and technical advancements across successive YOLO versions — from YOLOv1 to YOLOv9.
 - Explore the transition from early YOLO models (which emphasized speed) to later versions that balance accuracy, speed, and small object detection.
 - Study the modular innovations such as CSPDarknet, PANet, Mosaic augmentation, and decoupled head designs introduced in newer YOLO variants.
- To Collect and Review Dataset Characteristics Relevant to Crack Detection
 - Identify standard datasets used for crack detection in academic and industrial research (e.g., Roboflow, CFD, SDNET2018).

- Understand the dataset properties — number of samples, resolution, labeling techniques, crack size distribution, and image diversity.
- Assess how dataset quality, augmentation techniques, and label consistency influence the performance of YOLO models in crack detection.
- To Conduct a Comparative Review of YOLO Performance in Crack Detection
 - Compile and analyze reported accuracy, precision, recall, mAP (mean Average Precision), F1-score, inference speed (FPS), and resource utilization metrics across various YOLO versions when applied to crack detection tasks.
 - Evaluate model performance across diverse conditions including:
 - Crack visibility and size (fine vs. wide cracks)
 - Surface variation (asphalt, concrete, brick)
 - Environmental factors (lighting, shadows, occlusion)
 - Use published case studies and experimental benchmarks to identify trends and trade-offs among different YOLO versions.
- To Assess the Computational Efficiency and Deployment Feasibility
 - Evaluate the hardware requirements and inference time of each YOLO model when used for real-time or near-real-time crack detection.
 - Discuss the feasibility of deploying each model on resource-constrained platforms such as edge devices, UAVs (drones), or mobile applications.
 - Examine model size, GPU/CPU utilization, and training complexity as practical deployment constraints.
- To Determine the Suitability of Each YOLO Version for Different Use Cases
 - Categorize each YOLO version in terms of its best fit for different application domains:
 - Large-scale infrastructure (e.g., highways, bridges, dams)
 - Urban settings (e.g., building facades, historical monuments)
 - Real-time field inspection using UAVs
 - Evaluate how transfer learning and fine-tuning influence each model's performance on domain-specific data (e.g., cracks on weathered vs. new surfaces).
- To Recommend the Best-Suited YOLO Model(s) for Crack Detection
 - Based on the findings from the comparative review, suggest the most suitable YOLO variant(s) that offer an optimal balance between accuracy, speed, and deployability for crack detection tasks.
 - Provide use-case specific recommendations, such as which version is best for:
 - High-speed UAV inspections
 - High-precision lab-based assessments
 - Cost-effective edge deployment for periodic maintenance
 - Discuss the limitations of each model and propose potential hybrid or ensemble approaches if no single model is universally optimal.
- To Identify Future Research Directions and Enhancements

- Highlight current limitations in YOLO-based crack detection systems, including issues like false positives, dataset limitations, and lack of standardization in evaluation.
- Suggest possible enhancements such as:
 - Integrating YOLO with sensor fusion (e.g., LiDAR, thermal imaging)
 - Model compression techniques for embedded systems
 - Automated annotation pipelines for efficient dataset generation
 - Recommend areas where further investigation could improve the reliability and scalability of crack detection systems.

This detailed and multi-dimensional objective set ensures that the project provides both a theoretical and practical foundation for selecting the most effective YOLO model for structural health monitoring applications. By focusing on comparison and recommendation rather than implementation, this study will offer valuable insights for researchers, civil engineers, and AI practitioners looking to adopt deep learning for infrastructure safety.

2.4 PROBLEM STATEMENT

Infrastructure, particularly critical structures like bridges, buildings, and roads, plays a pivotal role in ensuring the safety and functionality of society. Over time, these structures undergo wear and tear due to various environmental, mechanical, and physical stresses. One of the most common forms of damage is the formation of surface cracks in concrete, which can compromise the structural integrity of buildings and bridges. If these cracks go unnoticed or are not addressed in a timely manner, they can escalate into severe failures, resulting in loss of life, property damage, and high repair costs.

Traditional methods of crack detection—such as visual inspections, manual assessments, and basic non-destructive testing techniques—are often time-consuming, labor-intensive, costly, and prone to human error. Inspectors might miss subtle or hidden cracks, particularly those in hard-to-reach locations or those that are not immediately visible. Moreover, human error can introduce inconsistencies and lead to inaccurate assessments. This significantly undermines the reliability of infrastructure monitoring systems, especially for large-scale and high-risk structures.

As seen in several high-profile incidents, including the 2018 Morandi Bridge collapse in Italy and the 2021 Champlain Towers collapse in the United States, unnoticed cracks and neglected infrastructure monitoring can lead to disastrous consequences. These events highlight the urgency of transitioning from traditional, manual inspection methods to automated, accurate, and efficient systems for detecting cracks in infrastructure.

2.4.1 Challenges in Current Crack Detection Methods

While advancements in technology have led to the development of various automated crack detection systems, many of these solutions still face significant challenges. One of the most prominent challenges is the accuracy of detection, particularly in real-world environments where conditions can vary drastically. For instance, cracks in concrete structures may present in various forms—small or large, shallow or deep, horizontal or vertical—and under different environmental conditions, such as varying light, weather, and background complexities. Current deep learning models, especially those based on the You Only Look Once (YOLO) framework, have shown promising results, but they still struggle with false positives, false negatives, and the ability to generalize across a wide range of conditions.

Another challenge is the computational cost of training and deploying these models. Advanced YOLO versions like YOLOv8 and YOLOv9 often require high-performance GPUs for both training and real-time inference, which limits their practical application in environments with limited resources, such as Unmanned Aerial Vehicles (UAVs) or mobile devices. Furthermore, while YOLO models can perform real-time crack detection, they often struggle with the detection of small cracks, cracks with complex orientations, or cracks that are obscured by debris, which are common in real-world infrastructure.

In addition, the lack of diverse, high-quality datasets—especially those that simulate the variety of crack patterns and environmental conditions encountered in practical applications—poses a significant limitation for YOLO-based models. Most existing datasets are small, narrowly focused on specific crack types, and lack sufficient data augmentation to account for real-world complexities. This limits the effectiveness and generalization ability of trained models.

Finally, the integration of YOLO-based models with other technologies, such as thermal imaging, infrared sensing, or laser scanning, remains an underexplored area. These technologies can help detect cracks that are invisible to the naked eye or located beneath surface layers, but integrating them with real-time deep learning models like YOLO could lead to more robust, multi-modal detection systems. However, such integrations are rare, and there is insufficient research into how they can be effectively implemented.

By addressing these challenges, this research aims to make significant contributions to proactive infrastructure monitoring, helping to ensure the safety and longevity of critical infrastructure. This system will reduce the need for costly manual inspections and increase the overall reliability and efficiency of crack detection in buildings, bridges, and other vital structures.

2.5 PROJECT PLAN

The project aims to investigate and determine the most suitable YOLO model for detecting cracks in structural infrastructure by conducting a **comparative study** based on architecture, performance, and practical deployment considerations. This plan outlines a **methodical approach** divided into six core phases, each designed to incrementally achieve the research objectives through systematic literature exploration, dataset analysis, model benchmarking, and technical review.

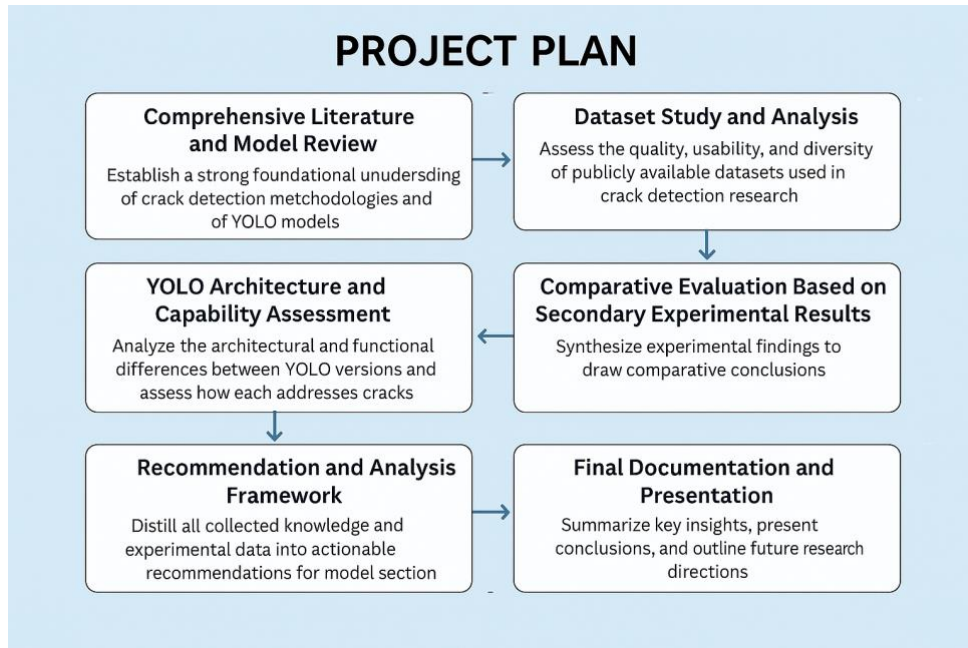


Figure 2.1 Project Plan

a. Comprehensive Literature and Model Review

Objective: To establish a strong foundational understanding of crack detection methodologies and the evolution of YOLO models, ensuring the project is built on established knowledge.

Activities:

- Conduct an exhaustive literature survey on:
 - Deep learning techniques used in crack detection.
 - The application and evolution of YOLO models from v1 to v9+.
- Study model papers, GitHub repositories, technical blogs, and benchmarking articles to extract comparative metrics and qualitative feedback.
- Identify architectural innovations introduced in each version of YOLO (e.g., CSPNet in YOLOv4, decoupled heads in v7, NAS in v8, RT-DETR in v9).
- Summarize findings and identify existing limitations in model applicability to structural health monitoring.

Deliverables:

- Literature review document covering YOLO's evolution and use in crack detection.
- Tabular comparison of model features and performance based on prior studies.
- Identified research gaps and specific questions to address in this comparative study.

b. Dataset Study and Analysis

Objective: To assess the quality, usability, and diversity of publicly available datasets used in crack detection research.

Activities:

- Explore datasets such as:
 - **SDNET2018, CrackForest, CFD**, and datasets from **Roboflow**.
- Evaluate datasets in terms of:
 - Image resolution and clarity.
 - Crack types (hairline, fatigue, wide cracks).
 - Labeling format (bounding boxes vs. segmentation).
 - Diversity of surfaces (asphalt, concrete, walls, pavements).
- Summarize preprocessing and augmentation techniques used in prior works.

Deliverables:

- Dataset catalog with characteristics and suitability notes.
- Analysis report on how dataset selection impacts YOLO performance.
- Guidelines on ideal dataset characteristics for accurate crack detection.

c. YOLO Architecture and Capability Assessment

Objective: To analyze the architectural and functional differences between YOLO versions and assess how each addresses challenges in crack detection.

Activities:

- Break down the components of each YOLO version, including:
 - Backbone, neck, and head design.
 - Feature pyramid strategies (FPN, PANet).
 - Anchor-based vs. anchor-free detection mechanisms.

- Analyze performance benchmarks from third-party evaluations, such as:
 - mAP (Mean Average Precision)
 - Inference speed (FPS)
 - Model size and FLOPs
- Compare suitability for detecting small and thin objects (like cracks), real-time use cases, and deployment scenarios (edge/cloud/mobile).

Deliverables:

- Technical matrix comparing all YOLO versions.
- Architecture-wise analysis and visual diagrams.
- Summary of each version's strengths, weaknesses, and optimal use cases.

d. Comparative Evaluation Based on Secondary Experimental Results

Objective: To synthesize experimental findings from open-source projects, research papers, and Kaggle/Roboflow competitions to draw comparative conclusions.

Activities:

- Aggregate YOLO performance results from published experiments in crack detection.
- Compare models on the following axes:
 - Detection precision and recall in real-world conditions.
 - Robustness to noise, lighting changes, and surface variations.
 - Ease of transfer learning and fine-tuning.
- Study real-case deployments (UAVs, embedded devices) where applicable.

Deliverables:

- Side-by-side performance charts and rankings.
- Graphical and tabular presentation of metric comparisons.
- Contextual interpretation of results aligned with infrastructure inspection needs.

e. Recommendation and Analysis Framework

Objective: To distill all collected knowledge and performance data into actionable recommendations for model selection based on specific use-cases.

Activities:

- Define key evaluation criteria for recommending a YOLO model:
 - Accuracy vs. Speed tradeoff.
 - Suitability for high-resolution crack imaging.
 - Compatibility with lightweight or edge devices.
- Match each YOLO model to different scenarios such as:
 - UAV-based aerial inspections.
 - Ground-level robotic systems.
 - Cloud-based monitoring platforms.
- Formulate a structured scoring matrix for YOLO model selection.

Deliverables:

- Final recommendation report identifying top YOLO variants for crack detection.
- Use-case specific recommendations.
- Highlighted limitations and open challenges for future exploration.

f. Final Documentation and Presentation

Objective: To consolidate all research insights, evaluations, and findings into a coherent final report and presentation.

Activities:

- Draft a detailed project report covering:
 - Background, motivation, and problem statement.
 - Literature review and dataset analysis.
 - Model comparisons and conclusions.
- Prepare visual aids:
 - Diagrams of model architectures.
 - Graphs, tables, and infographics for results.
- Develop a formal presentation to communicate findings to evaluators or peers.

Deliverables:

- Final written report adhering to academic formatting standards.
- Presentation slides summarizing key results and takeaways.
- Reference list citing all reviewed sources.

Chapter 3

TECHNICAL SPECIFICATION

3.1 REQUIREMENTS

3.1.1 Functional Requirements

The core functionality of the YOLO-based crack detection system revolves around accurately detecting and localizing cracks in various structural surfaces. At the forefront, the system must accept high-resolution input images, either captured manually or via UAVs, and preprocess them by resizing, normalizing, or converting them into the appropriate format for the model. Once the images are prepared, the system should employ trained YOLO models to detect cracks with bounding boxes and confidence scores, delivering a visual overlay on the input image.

Users should also have the flexibility to select between different YOLO versions such as YOLOv8 and YOLOv10 for training and evaluation, facilitating model comparisons. Furthermore, the system is expected to support real-time inference, especially when deployed on drones, enabling immediate feedback during inspections. Other vital functionalities include displaying evaluation metrics like Precision, Recall, F1-Score, and mean Average Precision (mAP), as well as logging all detections for later review. The system should also provide seamless model deployment for edge devices used on UAVs.

3.1.2 Non-Functional Requirements

Beyond core functionalities, the system must uphold several non-functional quality standards. Performance is paramount — the crack detection accuracy should exceed 90%, with real-time inference capability achieving at least 15 frames per second (FPS). Scalability is another crucial factor; the system must be adaptable to larger and more diverse datasets, as well as extensible to detect cracks on different types of infrastructure such as roads, bridges, or heritage sites.

Usability is key for end-users such as engineers and researchers, who should interact with an intuitive interface to upload datasets, view results, and interpret detection outputs. Reliability ensures the system performs well under varying environmental conditions like poor lighting or surface irregularities. Maintainability involves modular code that allows quick bug fixes or enhancements. Finally, portability supports transferring the trained model onto smaller, embedded systems for on-site deployment via UAVs or handheld devices.

3.2 FEASIBILITY STUDY

3.2.1 Technical Feasibility

Technically, the project is highly feasible with current tools and platforms. The YOLO architecture, particularly its Ultralytics implementation, offers cutting-edge object detection capabilities and supports small object detection, which is essential for identifying fine cracks. Platforms like Google Colab provide access to powerful cloud-based GPUs, such as the NVIDIA T4 with 16GB VRAM, which are well-suited for deep learning workloads, reducing the need for expensive on-premise hardware.

The entire toolchain — including PyTorch, Roboflow, TensorFlow, and OpenCV — integrates seamlessly with YOLO, enabling end-to-end model development. Moreover, the YOLO models can be optimized (via ONNX or TensorRT) for edge deployment on resource-constrained devices. As a result, implementing the trained system on UAVs for real-time inspections is both technically achievable and sustainable.

3.2.2 Economic Feasibility

From a financial perspective, the proposed system is highly economical. Leveraging free and open-source platforms like Google Colab and Kaggle eliminates hardware and software licensing costs. These platforms not only provide free access to GPUs but also offer collaborative environments that streamline model development. The use of Roboflow and publicly available datasets minimizes the need for costly manual data collection or custom labeling.

Additionally, since many academic and industry-grade UAVs already support embedded AI modules like Jetson Nano or Raspberry Pi, the cost of deploying the system in the field is relatively low. By adopting this architecture, the system ensures cost-efficiency without compromising on accuracy or performance, making it suitable for both academic research and industrial applications.

3.2.3 Social Feasibility

The proposed solution carries strong social relevance and acceptance. By enabling early detection of structural cracks, the system helps prevent catastrophic failures, ultimately saving lives and infrastructure costs. Its ability to automate inspections minimizes the need for human inspectors to perform risky manual evaluations, particularly on tall or unstable structures, improving safety for personnel.

Moreover, the project fosters technical skill development in areas like AI, computer vision, and drone operations, especially among students and

researchers. Since the system addresses a real-world problem that affects public safety and infrastructure resilience, its deployment will likely be welcomed by both regulatory authorities and the general public.

3.3 SYSTEM SPECIFICATION

3.3.1 Hardware Specification

The hardware components required for both development and deployment have been carefully selected to ensure optimal performance. During training, a cloud-based NVIDIA T4 GPU with 16GB VRAM is used to handle the computational intensity of YOLO models. This allows processing of high-resolution images and significantly speeds up the training and inference processes. A system with at least an Intel i7 processor and 16GB RAM is ideal for local tasks like dataset preparation and result visualization.

For field deployment, the model is intended to run on embedded systems integrated with UAVs, such as DJI drones equipped with HD cameras. Edge computing devices like NVIDIA Jetson Nano or Raspberry Pi 4 are well-suited for hosting optimized YOLO models. These systems require at least 100 GB of storage for handling datasets, models, and logs, and HD cameras (1080p or higher) to capture detailed surface imagery of the structures under inspection.

3.3.2 Software Specification

The software stack forms the foundation of the crack detection system's development, testing, and deployment pipeline. The core model training is done using the PyTorch framework via the Ultralytics YOLO implementation, which provides streamlined utilities for training, evaluation, and deployment. Google Colab is the primary development environment, providing access to GPUs and cloud storage integration via Google Drive.

Roboflow is employed for dataset preparation, including annotation, preprocessing, and augmentation — essential steps for enhancing the model's robustness. The system also relies on auxiliary libraries like NumPy, Pandas, OpenCV, and Matplotlib for data manipulation and visualization. For deployment, ONNX or TensorRT is used to convert and optimize models for edge environments. The entire project is managed via GitHub for version control and collaborative development, while Kaggle serves as an alternative platform for hosting datasets and running auxiliary experiments.

DESIGN APPROACH AND DETAILS

4.1 SYSTEM ARCHITECTURE

4.1.1 YOLO Models

4.1.1.1 YOLOv8

The YOLOv8 algorithm, builds upon the principles of its predecessors, integrating improvements in both accuracy and computational efficiency. It maintains the core principle of single-stage detection but refines the architecture with a more sophisticated backbone, enhanced detection head, and advanced loss functions. With its improved grid division, enhanced confidence scoring, and refined model architecture, YOLOv8 processes images more efficiently in a single pass. YOLOv8 offers multiple model variants, including YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, and YOLOv8x, each designed to meet different performance requirements. These improvements make YOLOv8 ideal for applications that require fast and accurate object detection, such as real-time video processing, autonomous driving, and various AI-driven monitoring systems.

The process begins with the input image undergoing pre-processing steps like resizing and normalization to prepare it for the model. This ensures consistency in how the image data is presented to the network, improving the model's ability to generalize across different types of inputs.

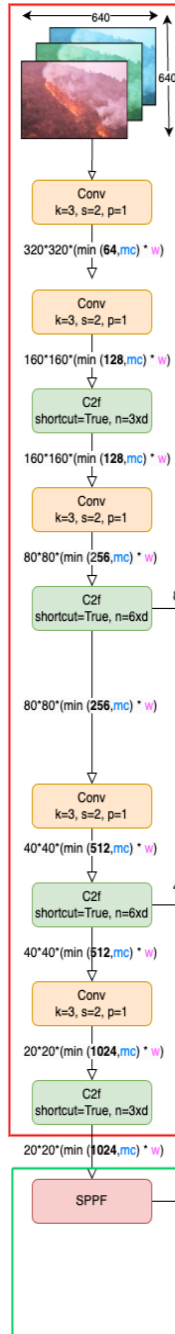
Once pre-processed, the image is fed into the backbone of the YOLOv8 model, which is responsible for extracting hierarchical features from the input image. This feature extraction is performed by a series of convolutional layers combined with advanced modules such as C2f (Cross Stage Partial Network with Efficient Layer Aggregation Network) and SPPF (Spatial Pyramid Pooling-Fast), where each layer applies a series of filters to capture different aspects of the image, such as edges, textures, and more complex patterns. The convolutional layers are followed by batch normalization and an activation function, specifically SiLU (Sigmoid Linear Unit), to introduce non-linearity and stabilize the learning process.

The backbone also includes the C2f module, derived from the C3 module of YOLOv5 and the ELAN module of YOLOv7, which enhances feature extraction by splitting the feature map into parts, processing them independently through bottleneck blocks, and then concatenating the results. This approach not only preserves computational efficiency but also allows the model to capture more complex patterns by leveraging the strength of both depth and width in the network's architecture.

To further refine the feature extraction process, YOLOv8 incorporates the Spatial Pyramid Pooling-Fast (SPPF) module, which applies pooling operations with different kernel sizes to capture spatial hierarchies within the image. This is particularly useful

for detecting objects like wildfires, which can vary significantly in size and shape. The SPPF module pools the input feature map at multiple scales, and then concatenates these pooled outputs to create a final feature map that encodes multi-scale information:

BACKBONE



Model Variant	d (depth_multiplier)	w (width_multiplier)	mc (max_channels)
n	0.33	0.25	1024
s	0.33	0.50	1024
m	0.67	0.75	768
l	1.00	1.00	512
x	1.00	1.25	512

NECK

HEAD

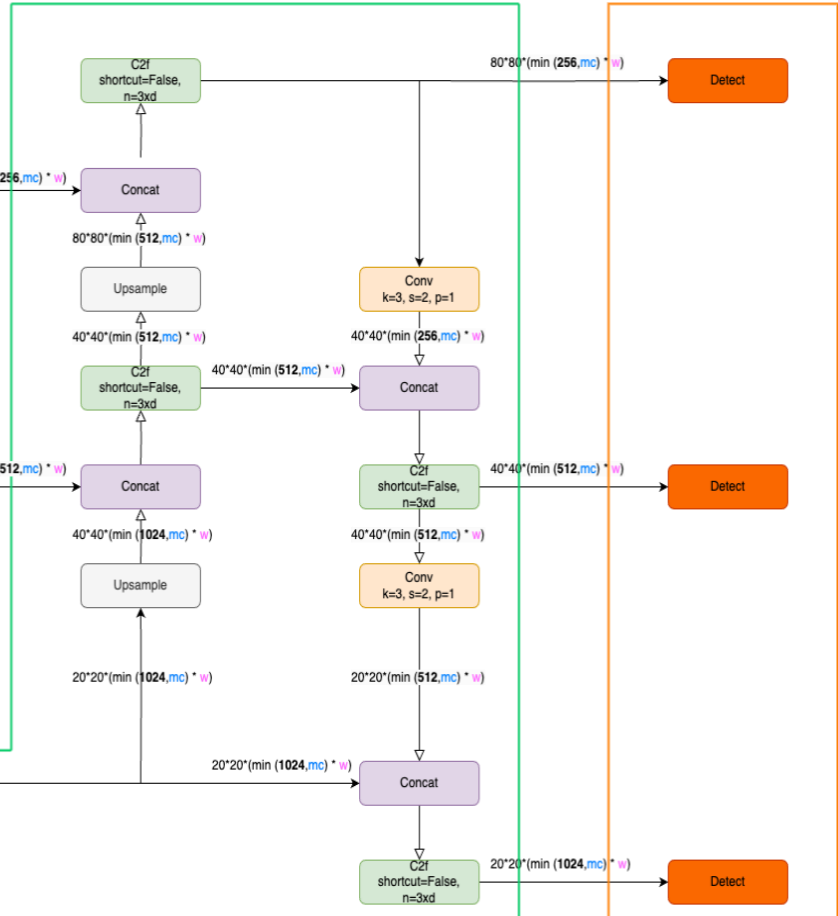


Figure 4.1 YOLO v8 Architecture

Following the backbone, the extracted features are passed to the neck of the YOLOv8 model, which further refines these features and prepares them for detection. It employs a PAN-FPN (Path Aggregation Network with Feature Pyramid Network) structure, which enhances the model's ability to detect objects of varying scales and complexities. The PAN component aggregates features from different layers of the backbone,

ensuring that fine-grained details from earlier layers are preserved and combined with higher-level abstract features. The FPN component, in turn, propagates strong semantic features from the deeper layers of the network to the shallower ones through a top-down pathway. In YOLOv8, the upsampling stages replace the traditional C3 modules with the more efficient C2f modules, omitting the need for the CBS (Convolutional, Batch Normalization, and SiLU activation) 1×1 convolutional structure. By upsampling the feature maps, the model ensures that information from the higher-resolution layers (which capture fine details) is retained and combined with the more abstract, lower-resolution features. This leads to reduced model size and inference latency.

Finally, the processed features are passed to the head of the YOLOv8 model, where the actual object detection takes place. The head uses a decoupled approach, which separates the tasks of object classification and bounding box regression into different branches. This decoupling allows the model to optimize each task independently, leading to more precise and reliable predictions. For bounding box regression, the model predicts the coordinates of the boxes surrounding detected objects using a loss function like Complete Intersection over Union (CIoU) to measure the accuracy of these predictions.

For the classification task, the model assigns probability scores to each detected object, indicating the likelihood that the object belongs to a particular class, such as "fire". The classification loss is typically calculated using binary cross-entropy, which measures the difference between the predicted probability and the true label.

Additionally, YOLOv8 transitions from an anchor-based to an anchor-free approach, which simplifies the network architecture by eliminating the need for predefined anchor boxes. Instead, the model directly regresses bounding box coordinates without relying on these anchors, making the detection process more flexible and adaptable to various object sizes and shapes. This anchor-free design is particularly advantageous in wildfire detection, where the objects (e.g., flames) do not conform to fixed aspect ratios, and the bounding boxes must be dynamically adjusted to fit the objects' unique shapes and sizes.

Table 4.1 YOLO v8 Features

Model	Size (pixels)	params ^(M)	FLOPs ^(B)
YOLOv8n	640	3.2	8.7
YOLOv8s	640	11.2	28.6
YOLOv8m	640	25.9	78.9
YOLOv8l	640	43.7	165.2
YOLOv8x	640	68.2	257.8

4.1.1.2 YOLOv9

YOLOv9, although still in development or in preliminary releases, builds upon the foundation laid by YOLOv8, further optimizing detection accuracy and efficiency. This version aims to address some of the remaining challenges in complex environments, such as dealing with small object detection and multi-scale features. YOLOv9 enhances the model's ability to detect fine cracks or subtle defects in large infrastructure projects by incorporating advanced techniques like attention mechanisms and improved anchor boxes. The architecture has been streamlined for faster inference times, making it more suitable for edge computing applications or embedded systems where computational resources are limited. YOLOv9 also incorporates advancements in transfer learning, allowing for quicker adaptation to specialized tasks such as crack detection with fewer labeled samples. This makes it an appealing choice for situations where dataset limitations or real-time processing constraints are significant factors. With improved detection capabilities in challenging scenarios, YOLOv9 could provide significant benefits in fields requiring accurate, fast, and scalable object detection, such as construction site monitoring or automated structural health inspections.

YOLOv9 continues the evolutionary leap started by YOLOv8 by introducing programmable and efficient architectural blocks that significantly enhance the learning and generalization capabilities of the model. One of the most notable innovations in YOLOv9 is the introduction of the Generalized Efficient Layer Aggregation Network (GELAN) backbone, which improves upon previous designs like CSPDarknet. GELAN offers superior gradient propagation by maintaining dense and effective skip connections, enabling better representation learning without increasing computational load.

Another standout feature is the Auxiliary Reversible Branch (ARB), which acts as a side-branch to assist the training process without affecting inference. ARB improves gradient flow during backpropagation, addressing challenges like vanishing gradients in deeper models. This makes YOLOv9 particularly effective at detecting small objects or those in cluttered scenes, where feature granularity and depth are crucial. YOLOv9 uses task-aligned assigners that dynamically allocate objects to appropriate detection heads based on training progress. This reduces false positives and improves label assignment efficiency. Furthermore, its decoupled head is enhanced with refined convolutional layers that allow more expressive feature extraction for both classification and regression tasks.

The overall design of YOLOv9 keeps efficiency at its core. Despite deeper architectural constructs, YOLOv9 maintains real-time inference speeds across various hardware platforms. The model is optimized for large-scale industrial applications like smart city surveillance, autonomous vehicle perception systems, and advanced UAV vision.

YOLOv9 demonstrates how thoughtful integration of novel components like GELAN and ARB, along with fine-tuned training techniques, can boost both accuracy and inference robustness. It bridges the gap between high-performing academic models and

real-time deployable systems, marking it as a versatile and scalable solution in the object detection domain. The YOLOv9 architecture introduces several advancements over its predecessor YOLOv8, continues to incorporate the single-stage object detection paradigm, but with enhancements across the backbone, neck, and head components, by addressing the challenges of information loss through novel techniques such as Programmable Gradient Information (PGI) and the Generalized Efficient Layer Aggregation Network (GELAN) making it both more efficient and accurate in processing complex visual data. These innovations are built on the foundation of the Information Bottleneck Principle and the strategic use of Reversible Functions, ensuring enhanced efficiency and accuracy.

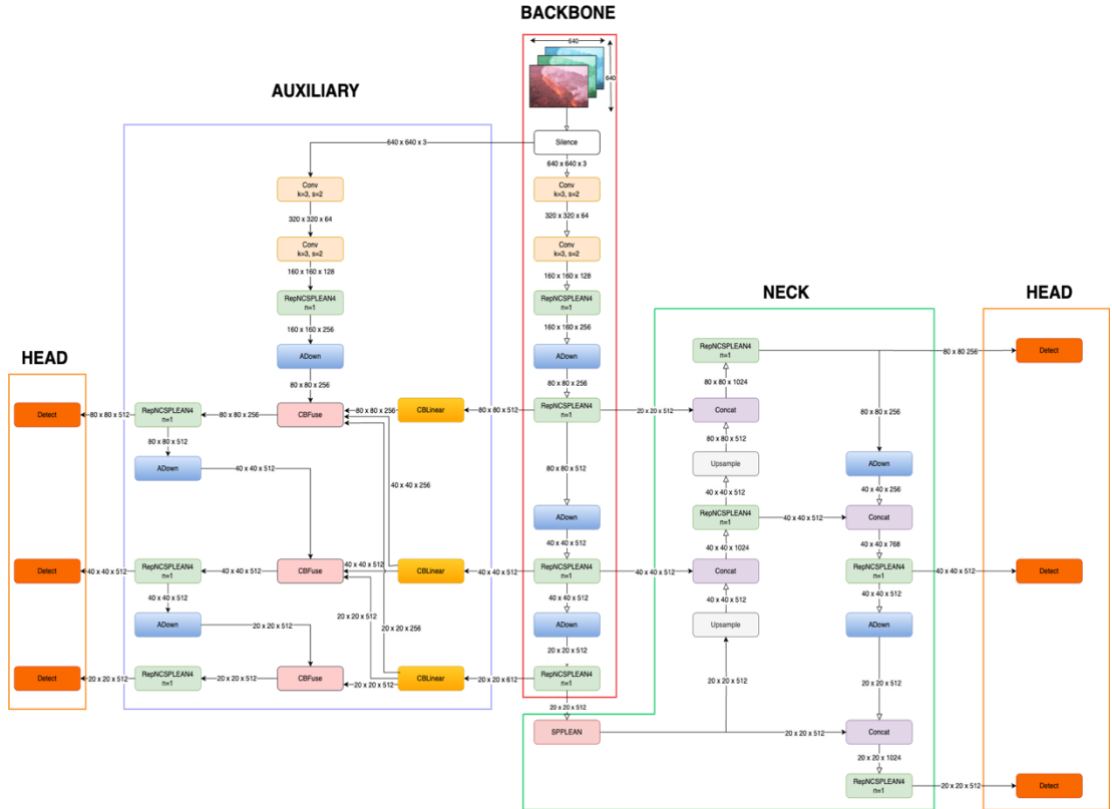


Figure 4.2 YOLO v9 Architecture

The input image undergoes pre-processing steps, including resizing and normalization, to ensure consistency and improve generalization across diverse input types. YOLOv9 then utilizes RepNcSPeLan4 modules for feature extraction, which split feature maps and process them through deeper bottleneck blocks before concatenating the outputs. This step enhances feature map richness and depth, crucial for complex detection tasks like forest fires.

Next, the CBLinear Fusion component aggregates features from different layers, preserving spatial details across scales. This is achieved through a linear fusion mechanism, which improves the retention of fine details and enhances the model's ability to detect small, irregular objects. In comparison to YOLOv8's PAN-FPN structure, YOLOv9's fusion mechanism is more effective.

The object detection stage in YOLOv9 involves an Advanced Detect Layer that separates classification and regression tasks for specialized processing. The bounding box regression employs an Adaptive Intersection over Union (AIoU) loss function, which provides better shape adaptation for irregular objects. The classification task uses Focal Loss, which focuses on hard-to-detect classes, ensuring that YOLOv9 maintains high accuracy even in challenging scenarios.

YOLOv9's architecture also includes an Auxiliary Network that provides additional pre-processed features to support the main detection pipeline. This auxiliary network enhances the robustness and accuracy of the final detections, further improving the model's performance.

At the core of YOLOv9's design are the Information Bottleneck Principle and Reversible Functions. The Information Bottleneck Principle addresses the issue of information loss as data passes through successive layers of the network. YOLOv9 counters this challenge by implementing PGI, which aids in preserving essential data across the network's depth, ensuring reliable gradient generation and better model convergence. Reversible Functions are integrated into the architecture to retain a complete information flow, enabling more accurate updates to the model's parameters and mitigating the risk of information degradation, particularly in deeper layers.

GELAN further enhances YOLOv9's architecture by enabling superior parameter utilization and computational efficiency. Its design allows for the flexible integration of various computational blocks, making YOLOv9 adaptable to a wide range of applications without sacrificing speed or accuracy. This strategic architectural advancement ensures that even lightweight models, which are often prone to information loss, can maintain essential data required for accurate object detection.

Table 4.2 YOLO v9 Features

Model	size(pixels)	params^(M)	FLOPs^(B)
YOLOv9t	640	2.0	7.7
YOLOv9s	640	7.2	26.7
YOLOv9m	640	20.1	76.8
YOLOv9c	640	25.5	102.8
YOLOv9e	640	58.1	192.5

4.1.1.3 YOLOv10

YOLOv10 represents a significant advancement in the field of real-time object detection, building upon the strengths of its predecessors while addressing their limitations. The key innovation of YOLOv10 lies in its ability to perform object detection without relying on traditional Non-Maximum Suppression (NMS). This is achieved through a novel dual-head prediction system that streamlines both training and inference processes.

BACKBONE

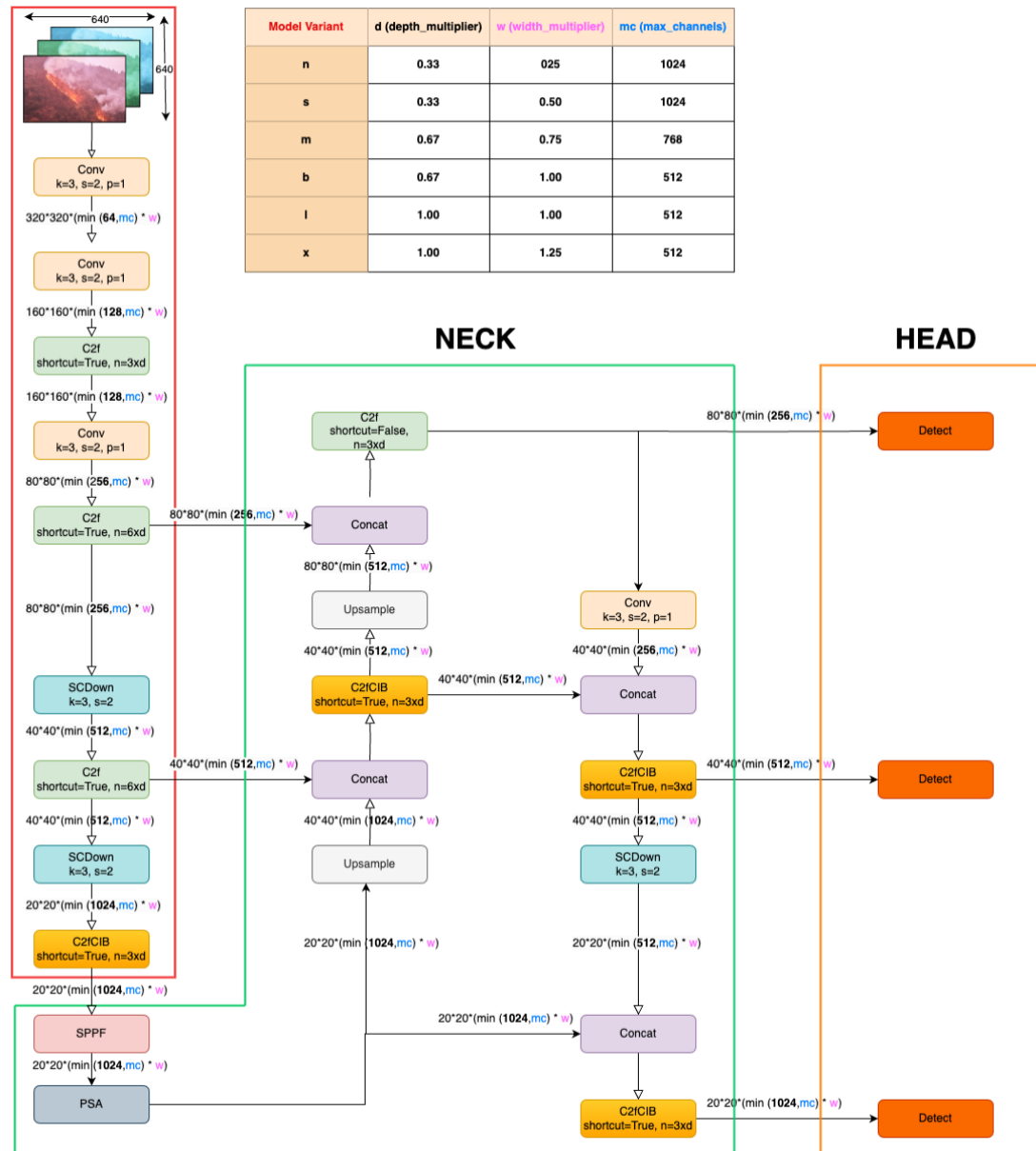


Figure 4.3 YOLO v10 Architecture

The architecture of YOLOv10 incorporates an optimized Cross Stage Partial Network (CSPNet) backbone, which enhances gradient flow and minimizes redundant computations during feature extraction. This backbone is crucial for generating high-

quality feature maps that are then processed by the network. The feature maps are refined through the Path Aggregation Network (PAN) layers in the neck of the architecture, which aggregates features from multiple scales. This aggregation ensures that both local and global features are preserved, leading to more accurate object detection.

In YOLOv10, the prediction stage is designed to operate differently during training and inference. During training, the model employs a One-to-Many Head approach, where multiple predictions are generated for each object. This approach allows the model to receive comprehensive supervision and improves learning accuracy by providing multiple bounding box predictions per object. The One-to-Many Head processes these predictions to compute a combined loss function, which includes metrics for classification, localization, and consistency.

Conversely, during inference, YOLOv10 utilizes a One-to-One Head, which generates a single, most confident bounding box prediction per object. This transition from the One-to-Many Head to the One-to-One Head during inference eliminates the need for NMS, resulting in reduced computational overhead and faster inference times. The model's design ensures that only the most accurate predictions are considered, which enhances both speed and efficiency.

The image processing pipeline in YOLOv10 starts with the input image being passed through the CSPNet backbone, where initial feature maps are created. These feature maps are then processed by the PAN layers, which combine features from different scales to capture detailed information. The fused features are forwarded to the dual-head prediction system, where the One-to-Many Head generates multiple bounding box predictions for each object during training. These predictions are used to calculate the loss and refine the model's learning.

During the inference phase, the model relies solely on the One-to-One Head to produce the final predictions. This head simplifies the detection process by directly providing the most confident bounding boxes without additional post-processing steps like NMS. The reduction in post-processing requirements contributes to the model's efficiency, making YOLOv10 well-suited for real-time applications where quick response times are essential.

YOLOv10's design also includes several efficiency-oriented optimizations. The classification head uses depthwise separable convolutions to reduce computational costs while maintaining performance. Additionally, the decoupling of spatial downsampling and channel transformation operations helps preserve more information during the downsampling process, which improves detection accuracy. Advanced techniques such as large-kernel convolutions and partial self-attention modules further enhance the model's ability to capture broader contextual information and improve global representation learning.

Table 2.3 YOLO v10 Features

Model	size ^(pixels)	params ^(M)	FLOPs ^(B)
YOLOv10n	640	2.3	6.7
YOLOv10s	640	7.4	21.6
YOLOv10m	640	15.4	59.1
YOLOv10l	640	24.4	120.3
YOLOv10x	640	29.5	160.4

4.1.1.4 YOLOv11

YOLOv11 is the latest advancement in the “You Only Look Once” series, designed to deliver real-time performance across a wide array of vision tasks including object detection, instance segmentation, pose estimation, and more. Building on the success of YOLOv8 and YOLOv10, YOLOv11 emphasizes modular scalability, efficiency, and adaptability for diverse deployment scenarios—ranging from edge devices to high-performance servers. It introduces architectural refinements that not only enhance accuracy but also significantly improve inference speed and resource optimization.

At the core of YOLOv11 lies a re-engineered backbone optimized for fast and effective feature extraction. This backbone adopts principles from lightweight networks, such as EfficientNet and CSPNet, integrating depthwise separable convolutions, residual connections, and cross-stage partial networks. These components allow the model to preserve both low-level details and high-level semantic context without overburdening computational resources. In addition, attention mechanisms like Squeeze-and-Excitation blocks are introduced to selectively amplify meaningful features while suppressing noise, resulting in better object recognition even in complex scenes.

The neck of YOLOv11 is a powerful feature aggregation module that fuses multi-scale information extracted from the backbone. It employs an advanced form of the Path Aggregation Network (PAN), which introduces both top-down and bottom-up paths to combine spatially rich low-level features with semantically strong high-level features. In some configurations, YOLOv11 incorporates BiFPN-style weighted feature fusion to intelligently balance feature maps based on their learned importance. This hierarchical blending of features allows the model to detect objects at varying scales with improved precision, recall, and localization quality.

The prediction head in YOLOv11 is designed to be task-specific and decoupled, meaning the branches for object classification and localization are handled separately. This architectural choice minimizes the conflict between learning tasks and enhances convergence during training. For object detection, the head predicts bounding box

coordinates, objectness scores, and class probabilities. For instance segmentation, an additional mask branch is introduced to generate pixel-wise segmentation masks. In the case of pose estimation, YOLOv11 adds a dedicated keypoint detection branch that maps human body landmarks with high accuracy. Each of these heads is efficiently built to share features from the backbone and neck while specializing in their respective tasks.

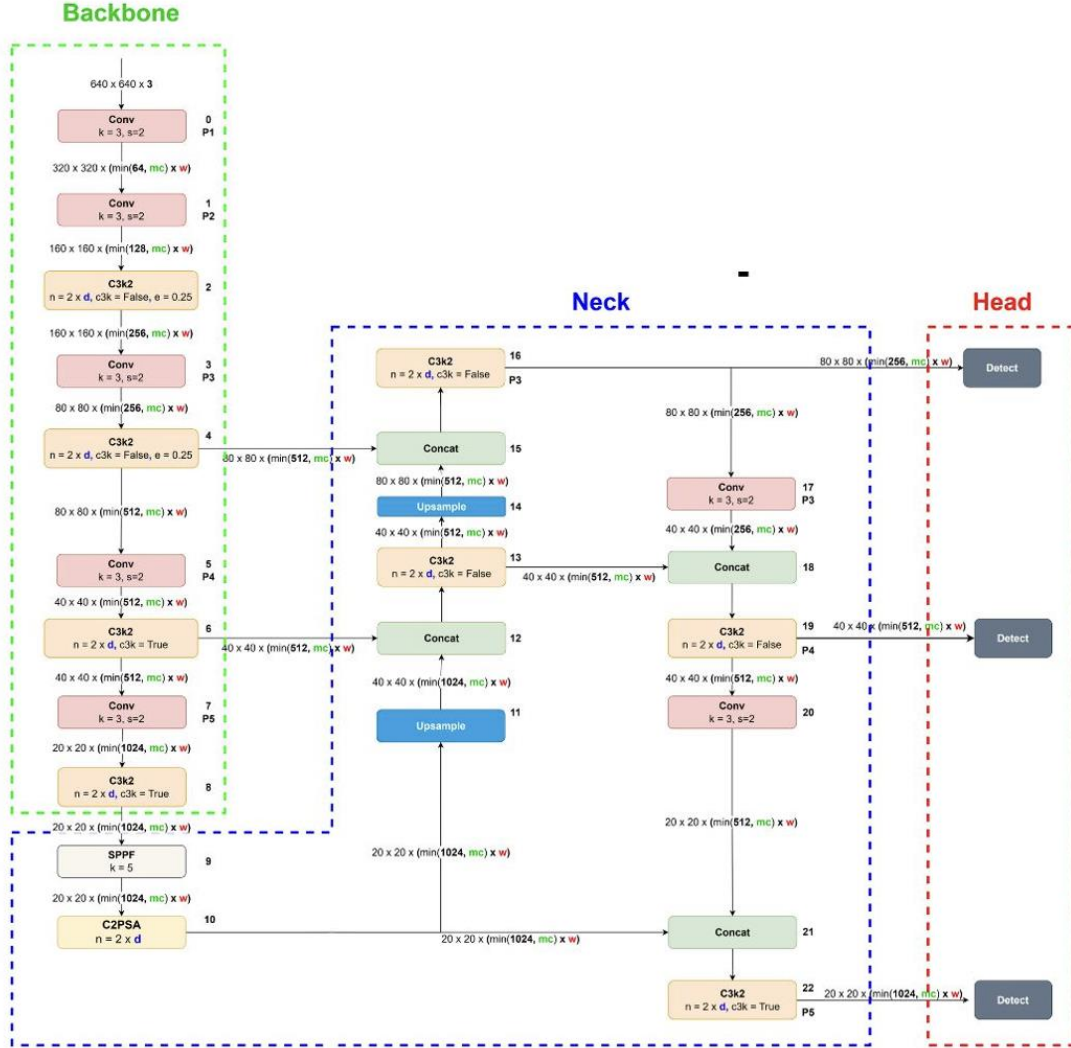


Figure 2.4 YOLO v11 Architecture

A major highlight of YOLOv11 is its support for rotated object detection, commonly known as Oriented Bounding Boxes (OBB), useful in aerial imagery and industrial inspections. This is achieved through an angular regression module that estimates the rotation of objects in addition to their position and size. YOLOv11 also includes a dedicated classification variant that replaces the detection head with global pooling and dense layers for image classification tasks. These modular extensions make YOLOv11 exceptionally versatile, empowering researchers and engineers to use a single architecture for multiple vision applications.

The training pipeline in YOLOv11 has been significantly enhanced with state-of-the-art strategies. The model utilizes improved label assignment techniques like Task-

Aligned Assigner or SimOTA for better matching of predictions to ground truths. Loss functions such as CIoU or SIoU are employed to refine bounding box regression, while Focal Loss or Varifocal Loss manages class imbalance. Additionally, aggressive yet effective augmentations like Mosaic, MixUp, and HSV adjustments contribute to better generalization. These strategies ensure that YOLOv11 not only achieves high accuracy during training but also remains robust in real-world deployments.

YOLOv11 is released in various model sizes, including nano (n), small (s), medium (m), large (l), and extra-large (x), offering a scalable solution for different resource budgets. For instance, YOLO11n is optimized for mobile and embedded devices, while YOLO11x provides top-tier accuracy for GPU-powered servers. These variants are carefully balanced in terms of parameter count, computational complexity, and speed, giving users the flexibility to choose the most appropriate model depending on their application requirements.

In summary, YOLOv11 is a highly modular, scalable, and efficient vision architecture that builds upon years of research in real-time object detection. Its carefully crafted backbone, multi-scale fusion neck, and task-aware heads enable it to tackle a wide range of visual recognition challenges with top-tier performance. Whether used for surveillance, robotics, autonomous driving, healthcare, or industrial automation, YOLOv11 offers a unified and robust framework that sets a new benchmark for speed, accuracy, and versatility in computer vision.

Table 4.4 YOLO v11 Features

Model	size(pixels)	params^(M)	FLOPs^(B)
YOLO11n	640	2.6	6.5
YOLO11s	640	9.4	21.5
YOLO11m	640	20.1	68.0
YOLO11l	640	25.3	86.9
YOLO11x	640	56.9	194.9

4.1.2 Optimizers

In the realm of deep learning, optimizers play a critical role in the model training process. They are responsible for updating the weights of the neural network based on the gradients of the loss function. Among the plethora of optimizers available, **Stochastic Gradient Descent (SGD)** and **AdamW** stand out for their widespread usage and proven efficiency, especially in computer vision tasks like object detection using YOLO models. This section delves into the working mechanisms, advantages, limitations, and use-case suitability of both optimizers.

4.1.2.1 SGD

Stochastic Gradient Descent (SGD) is one of the oldest and most foundational optimization algorithms in machine learning. At its core, SGD updates weights by computing the gradient of the loss function using a small subset (mini-batch) of the training data. It performs parameter updates using the equation:

$$\mathbf{w} = \mathbf{w} - \eta \times \nabla L(\mathbf{w})$$

Where $\mathbf{w}$ denotes weights, η is the learning rate, and $\nabla L(\mathbf{w})$ is the gradient of the loss function. Though simple, this method is extremely effective in large-scale problems, especially when enhanced with techniques like **momentum**, which helps accelerate convergence by adding a fraction of the previous weight update. This reduces oscillations and helps in faster traversal across ravines in the loss landscape. However, SGD is sensitive to the choice of learning rate and may get stuck in local minima or saddle points. To overcome this, learning rate schedulers such as cosine annealing and step decay are often employed.

Despite its drawbacks, SGD has the advantage of generalizing well, especially in classification tasks with large datasets. This makes it a reliable choice for training large object detection models like YOLOv8 and YOLOv10, where overfitting is a concern and robustness is critical.

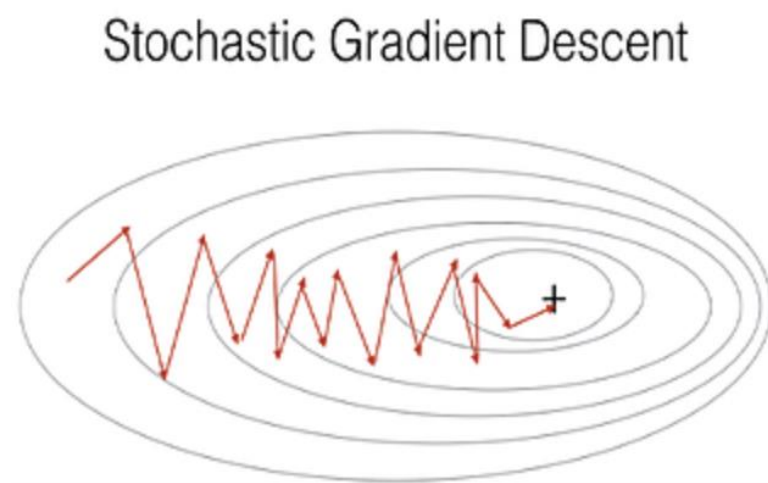


Figure 4.5 SGD Optimizer

4.1.2.2 ADAMW

AdamW (Adaptive Moment Estimation with Decoupled Weight Decay) is a newer, adaptive optimizer that builds upon the Adam algorithm. While Adam adjusts learning rates for each parameter individually based on the first and second moments of gradients, AdamW modifies the original Adam by decoupling the weight decay from the gradient-based update. The update rule is:

$$\mathbf{w} = \mathbf{w} - \eta \times (\hat{\mathbf{m}} / \sqrt{\hat{\mathbf{v}} + \epsilon} + \lambda \mathbf{w})$$

Here, $\hat{\mathbf{m}}$ and $\hat{\mathbf{v}}$ are bias-corrected estimates of the mean and variance of gradients, λ represents the weight decay factor, and ϵ is a small constant to prevent division by zero.

By separating the weight decay from the gradient update, AdamW ensures better regularization and generalization capabilities. This is especially useful in object detection models like YOLOv9 and YOLOv11, where intricate architectures demand careful balancing of convergence speed and regularization. AdamW typically converges faster than SGD and handles sparse gradients effectively, making it ideal for deep architectures and datasets with high variance.

Furthermore, AdamW is more forgiving to hyperparameter choices than SGD. While SGD may require careful tuning of learning rate and momentum, AdamW often works well with default values, reducing the burden on the experimenter.

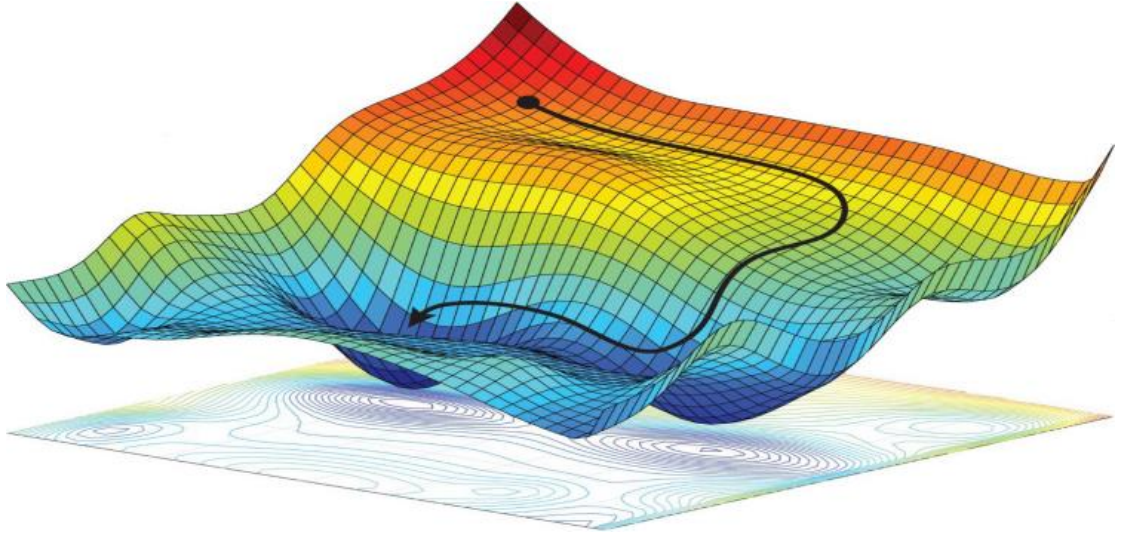


Figure 4.3 ADAMW Optimizer

4.2 DESIGN

4.2.1 Data Flow Diagram

The Data Flow Diagram (DFD) for the crack detection system illustrates how data moves through various functional modules. At **Level 0**, the system starts with **Image Input**, either from drones or uploaded manually. These raw images are sent to the **Preprocessing Module**, where they undergo resizing, grayscale conversion, noise reduction, and normalization.

The preprocessed images are then forwarded to the **Model Inference Module**, which utilizes a trained YOLO model to identify and localize cracks. The results — bounding boxes, labels, and confidence values — are sent to the **Visualization Module**, where the user can see the predictions overlaid on the original image. Simultaneously, the results are stored in the **Results Storage Database** for later analysis or report generation.

At **Level 1**, the diagram details subprocesses like **dataset augmentation using Roboflow**, **model training with hyperparameter tuning**, and **real-time data streaming from UAVs**. These processes are asynchronous and modular to ensure smooth integration into real-world inspection pipelines. The system is designed to provide feedback loops, where poorly predicted images can be flagged, re-annotated, and added back to the dataset for continuous learning.

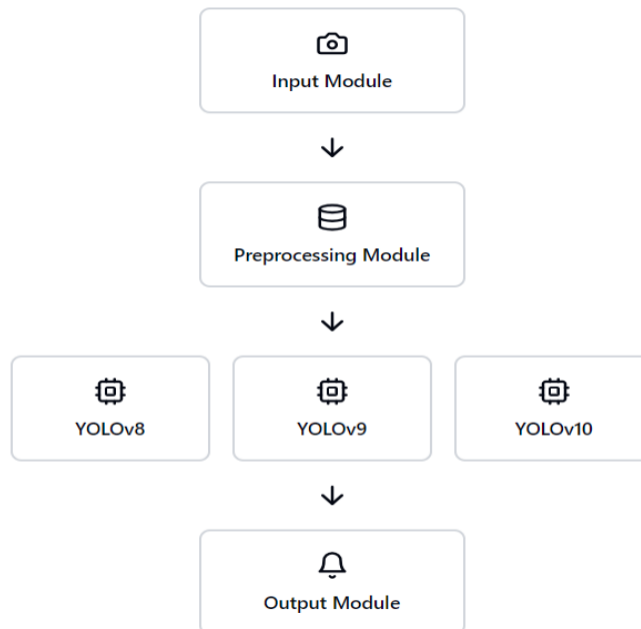


Figure 4.4 Data Flow Diagram

4.2.1 Class Diagram

The **Dataset** class lies at the core of any machine learning pipeline, serving as the starting point of the model training journey. This class encapsulates the raw input data, typically comprising a collection of training samples such as images, text, or sensor readings, along with their corresponding labels. The Dataset class is responsible for managing this data and provides methods like `load_data()` to perform preprocessing steps. These steps may include resizing images, normalizing values, augmenting data to increase diversity, or transforming labels into a format digestible by the model. It acts as an interface between the raw data and the model, ensuring that all input is appropriately prepared and structured. Both training and validation datasets are handled using this class, establishing a consistent format and easing integration with the training loop.

The **Model** class represents the architectural design of the deep learning network. It is modular in structure and comprises three major components: the backbone, neck, and head. The backbone is typically a convolutional neural network or any deep feature extractor that learns meaningful representations from the input data. The neck connects the backbone to the head and often implements feature aggregation techniques such as Feature Pyramid Networks (FPNs) or Path Aggregation Networks (PANs), which enhance the semantic understanding of features across different scales. Finally, the head layer is responsible for task-specific predictions such as bounding box regression in object detection or classification in image recognition. The Model class also contains functions like `configure_backbone()`, `configure_neck()`, and `configure_head()`, allowing developers to tailor each module to fit the task at hand. This abstraction makes the class adaptable for various deep learning tasks, offering flexibility in model customization.

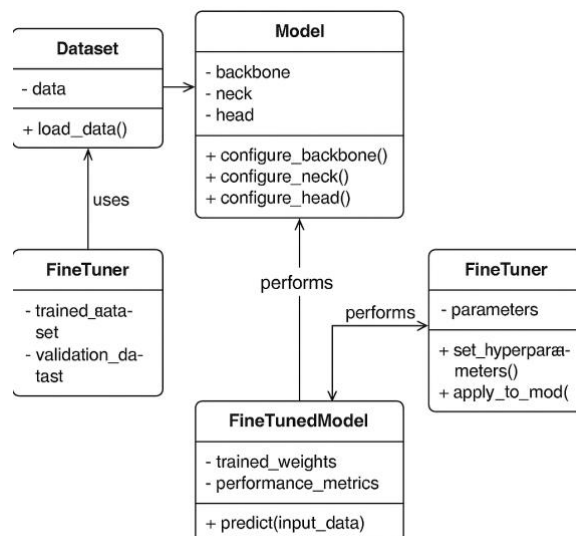


Figure 4.5 Class Diagram

At the heart of optimizing the architecture lies the **FineTuner** class, which plays a critical role in transforming a baseline model into one that is task-specific and well-

optimized. Fine-tuning involves adjusting model hyperparameters and modifying layer weights using pre-trained or partially trained networks. The `FineTuner` holds a dictionary or configuration structure called `parameters` that includes elements such as learning rate, optimizer settings, weight decay, batch size, and number of epochs. These parameters significantly impact model performance and convergence. The class interacts closely with the `Model` class by applying these hyperparameters using the `apply_to_model(model)` method, effectively updating the model's behavior during training. The `set_hyperparameters()` method allows fine-tuning strategies to be revised, based on the complexity of the dataset, hardware limitations, or early performance observations. This layer of abstraction allows experimentation with training dynamics without directly modifying the core model architecture.

Following fine-tuning, the **`ModelTrainer`** class (implicitly embedded in the workflow) takes over, facilitating the actual process of training and evaluation. It accepts both the fine-tuned model and datasets, using `train_model()` to initiate a series of forward and backward passes where the model learns from errors and updates its parameters via gradient descent. The class typically includes monitoring metrics, loss functions, and early stopping mechanisms to prevent overfitting. Once training is complete, the `validate_model()` function is invoked to assess the generalizability of the model using the validation dataset. This step involves computing metrics like accuracy, precision, recall, or intersection-over-union (IoU), depending on the task. By abstracting the training and validation logic into a separate class, developers can encapsulate the entire learning loop, enabling cleaner code and reusable components for different model architectures or experiments.

The final output of this process is encapsulated in the **`FineTunedModel`** class. This class stores the result of the training pipeline, including the learned `trained_weights`, which are crucial for the model's ability to generalize to new data. It also retains `performance_metrics` obtained from validation, which provide insight into how well the model performs on unseen data. The `FineTunedModel` class exposes a `predict(input_data)` method, enabling inference on real-world inputs. This class can be serialized, exported, and deployed for use in production systems or research demonstrations. Additionally, having a dedicated model class for predictions ensures separation of concerns, as training logic remains decoupled from inference logic, supporting scalable and maintainable machine learning development.

Together, these classes form a robust, modular pipeline for developing and deploying a machine learning model. Starting from dataset ingestion, passing through model architecture configuration and hyperparameter tuning, flowing into training and validation, and culminating in a deployable prediction-ready model, the system demonstrates a clean and logical software design for end-to-end AI development.

Chapter 5

METHODOLOGY AND TESTING

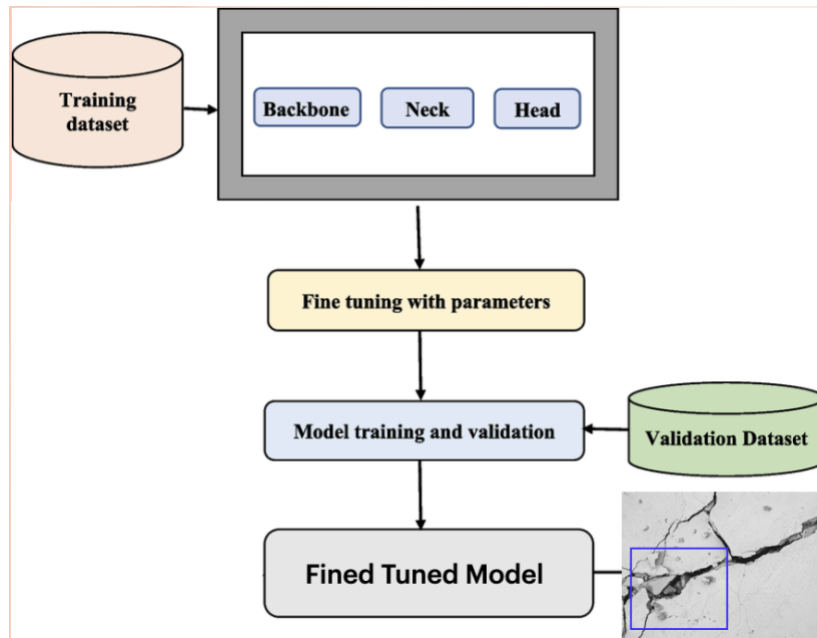


Figure 5.1 Methodology

This system design represents the process of fine-tuning a YOLO (You Only Look Once) model for object detection, specifically for tasks like crack detection (as inferred from the example image at the bottom right). Let's break down each component and its role in fine-tuning the YOLO model.

The diagram depicts a structured workflow for training and fine-tuning a YOLO model. It starts with the training dataset, processes it through the model's backbone, neck, and head, fine-tunes the parameters, trains and validates the model, and finally produces a fine-tuned model that can detect objects (e.g., cracks in images).

A. Training Dataset

- The process begins with a dataset containing labeled images, which are used for training the YOLO model.
- This dataset includes images annotated with bounding boxes around the target objects (e.g., cracks in a structural inspection scenario).
- The dataset is formatted as per YOLO requirements (e.g., using .txt label files with class and bounding box coordinates).

B. YOLO Model Structure

The YOLO model is divided into three key components:

1. Backbone

- Extracts feature representations from the input image.
- Commonly used backbones in YOLO versions include CSPDarknet, MobileNet, or EfficientNet.
- It processes raw pixel data into meaningful feature maps.

2. Neck

- Bridges the backbone and the head.
- Enhances multi-scale feature representation, allowing the model to detect objects of varying sizes.
- Common architectures include PANet (Path Aggregation Network) and FPN (Feature Pyramid Network).

3. Head

- Outputs the final predictions (bounding box coordinates, class labels, and confidence scores).
- Uses anchor-based or anchor-free detection strategies.
- Applies non-max suppression (NMS) to refine predictions.

C. Fine-Tuning with Parameters

- Fine-tuning involves adjusting the model's hyperparameters to improve detection accuracy.
- Parameters include:
 - Learning rate (affects how quickly the model updates weights).
 - Batch size (number of images processed at once).
 - Number of epochs (how many times the dataset is processed).
 - Data augmentation techniques (flipping, rotation, color adjustments).
 - Anchor box tuning (modifying default bounding box sizes).

D. Model Training and Validation

- The model is trained using the processed training dataset.
- It learns object features by minimizing loss functions such as:
 - **Bounding box loss** (measures accuracy of predicted vs. actual boxes).
 - **Classification loss** (ensures correct object identification).
 - **Confidence loss** (assesses certainty of object presence).

- Validation dataset is used to check the model’s generalization ability, ensuring it doesn’t overfit the training data.

E. Fine-Tuned Model

- The final trained model can detect objects in new images.
- It generalizes well to unseen data, providing accurate predictions

5.1 DATASET

The **crack detection dataset** used in this case is a curated collection of **4,029 high-resolution images** specifically focused on identifying cracks in concrete or structural surfaces. It is designed to train and evaluate deep learning models—especially object detection architectures like YOLOv11—for infrastructure safety applications, such as bridge inspection or road surface analysis. The dataset is divided into three subsets to support model training, validation, and evaluation. Approximately **92% of the images (3,717 images)** are used in the **training set**, allowing the model to learn general patterns and features associated with various types of cracks. The **validation set**, comprising **5% of the dataset (200 images)**, helps monitor the model’s performance during training and guides hyperparameter tuning. Finally, the **test set** includes **112 images (3%)**, reserved for final performance evaluation to assess how well the model generalizes to unseen data.

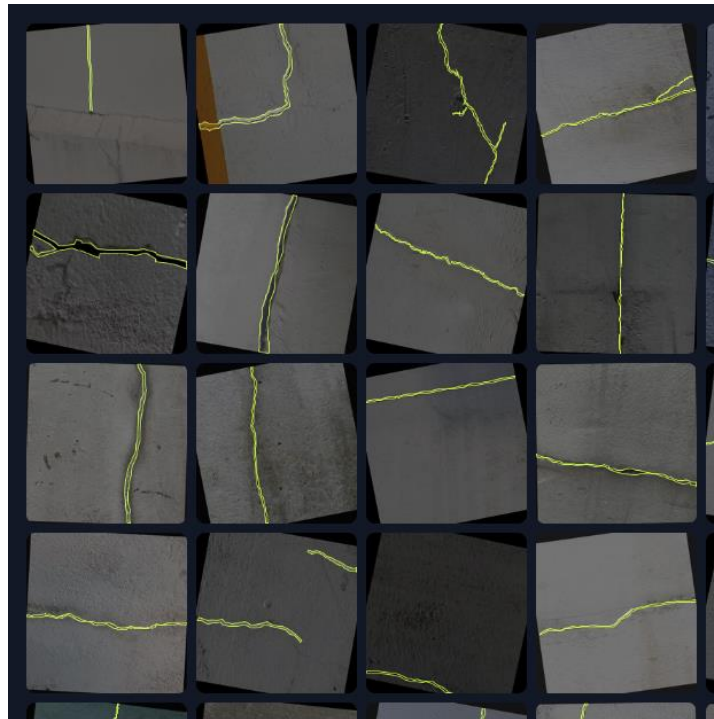


Figure 5.2 Sample Dataset Images

Each image in the dataset is **preprocessed** to a standard size of **416×416 pixels** using a stretch resize method. This ensures uniformity across the dataset and compatibility with the input size requirements of YOLOv11’s detection heads. Since crack features can be subtle and visually noisy, the dataset incorporates a variety of **data augmentations** to increase robustness. These include **horizontal flipping** and

rotations (90° clockwise, counterclockwise, and upside down) to simulate different orientations and viewpoints. Additional augmentations such as **random cropping** with up to 30% **zoom**, **small-angle rotations** between -15° and +15°, and **color transformations**—including **hue adjustments** (-10° to +10°), **saturation shifts** (-25% to +25%), and **brightness variation** (-20% to +20%)—enhance the model’s ability to handle varied lighting and environmental conditions. A subtle **blur augmentation (up to 1px)** is also applied to mimic camera noise or out-of-focus captures that often occur in real-world monitoring scenarios.

Each training image is annotated with bounding boxes identifying the location of visible cracks. These annotations are likely structured in a YOLO-compatible format, where each label entry includes the class (likely a single class: “crack”) and the bounding box coordinates (center_x, center_y, width, height), normalized relative to the resized image dimensions. For each training example, **three outputs** are generated—corresponding to the multi-scale prediction strategy of YOLOv11, which helps detect both fine, hairline cracks and larger, more prominent fractures across different feature map resolutions. This dataset structure and preprocessing pipeline ensure that the model learns both low-level textures and high-level context, essential for accurate crack localization in safety-critical environments.

Table 5.1 Dataset Split

Split	Images	Percentage
Training	3,717	92%
Validation	200	5%
Testing	112	3%

Table 5.2 Preprocessing and Augmentations

Type	Details
Resize	Stretch to 416×416
Flip	Horizontal
Rotate	90° (Clockwise, Counter-Clockwise, Upside Down)
Crop (Zoom)	0% Min Zoom, 30% Max Zoom
Rotation	Between -15° and +15°
Hue	Between -10° and +10°
Saturation	Between -25% and +25%
Brightness	Between -20% and +20%
Blur	Up to 1px

5.2 FINETUNING

The fine-tuning process played a pivotal role in tailoring the YOLO (You Only Look Once) object detection models to the specific task of infrastructure crack detection. This study focused on four recent versions of YOLO—v8, v9, v10, and the latest v11—each tested across three model sizes: nano (n), small (s), and medium (m). These variants offer a balanced spectrum between performance and computational load, an essential consideration for real-world deployment on edge devices like surveillance cameras and drones. Each model was initialized with pre-trained weights from the COCO dataset, enabling transfer learning to reduce training time and improve convergence. Fine-tuning was conducted on the dedicated crack detection dataset containing 4,029 annotated images, with 92% allocated for training, 5% for validation, and 3% for testing. The preprocessing pipeline resized each image to 416x416 pixels and applied a comprehensive suite of augmentations including horizontal flipping, multiple rotations (clockwise, counterclockwise, upside down), zoom cropping, as well as adjustments in brightness, saturation, hue, and blur. These augmentations were crucial in simulating real-world conditions such as poor lighting, various camera angles, and environmental noise. To understand how optimization affects learning behavior and model performance, two popular optimizers—AdamW and Stochastic Gradient Descent (SGD)—were employed separately across each model variant, resulting in 24 fine-tuning experiments.

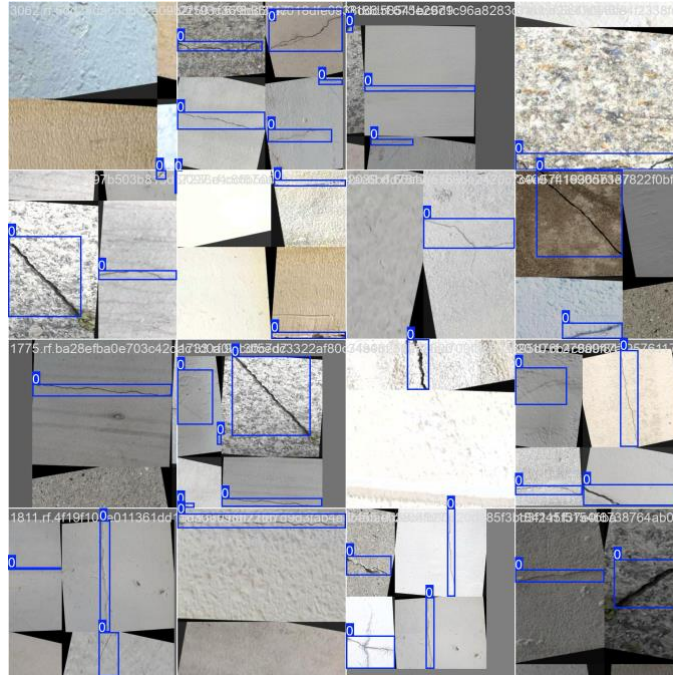


Figure 5.3 Sample Training Batch

All training and fine-tuning workflows were executed in Google Colab using GPU-accelerated runtimes for efficient computation. The models were trained for a fixed number of epochs with early stopping mechanisms based on validation loss to avoid overfitting. Fine-tuning strategies involved a staged unfreezing of model layers: initially, the final detection heads were tuned to adapt to the single-class nature of the

task (i.e., cracks), followed by gradual unfreezing of the backbone layers to adapt lower-level features such as edge and texture sensitivity. Careful tuning of hyperparameters such as batch size, learning rate, and weight decay was performed separately for each model-optimizer pair to ensure fair comparison. Throughout training, key metrics including validation loss, mean Average Precision (mAP@0.5), Precision, Recall, and training convergence speed were logged. The ultimate goal was to identify the most optimal configuration in terms of both accuracy and efficiency, capable of deployment on embedded devices in surveillance infrastructure or drone-based monitoring systems. This comprehensive approach ensured that the fine-tuned models are not only precise in detecting minute cracks but also lightweight and fast enough for real-time, on-the-fly inference.

Table 5.3 Args for fine-tuning yolov8

Parameter	Value	Parameter	Value	Parameter	Value
task	detect	mode	train	model	yolov8m. yaml
data	/content/datasets/ crack-2/data.yaml	epochs	100	time	null
patience	100	batch	16	imgsz	640
save	true	save_period	-1	cache	false
device	null	workers	8	project	null
name	train	exist_ok	false	pretrained	true
optimizer	auto	verbose	true	seed	0
deterministic	true	single_cls	false	rect	false
cos_lr	false	close_mosaic	10	resume	false
amp	true	fraction	1.0	profile	false
freeze	null	multi_scale	false	overlap_mask	true
mask_ratio	4	dropout	0.0	val	true
split	val	save_json	false	save_hybrid	false
conf	null	iou	0.7	max_det	300
half	false	dnn	false	plots	true
source	null	vid_stride	1	stream_buffer	false
visualize	false	augment	false	agnostic_nms	false
classes	null	retina_masks	false	embed	null
show	false	save_frames	false	save_txt	false
save_conf	false	save_crop	false	show_labels	true

show_conf	true	show_boxes	true	line_width	null
format	torchscript	keras	false	optimize	false
int8	false	dynamic	false	simplify	true
opset	null	workspace	null	nms	false
lr0	0.01	lrf	0.01	momentum	0.937
weight_decay	0.0005	warmup_epochs	3.0	warmup_momentum	0.8
warmup_bias_lr	0.1	box	7.5	cls	0.5
dfc	1.5	pose	12.0	kobj	1.0
nbs	64	hsv_h	0.015	hsv_s	0.7
hsv_v	0.4	degrees	0.0	translate	0.1
scale	0.5	shear	0.0	perspective	0.0
flipud	0.0	flip_lr	0.5	bgr	0.0
mosaic	1.0	mixup	0.0	copy_paste	0.0
copy_paste_mode	flip	auto_augment	randaug	erasing	0.4
crop_fraction	1.0	cfg	null	tracker	botsort.yaml

5.3 VALIDATION AND TESTING

The validation and testing phases were critical in assessing the generalization and deployment-readiness of the fine-tuned YOLO models for real-world crack detection scenarios. After the fine-tuning phase, each model variant—across versions YOLOv8 to YOLOv11, and sizes n, s, m—was subjected to rigorous evaluation using the reserved validation (5%) and test (3%) splits of the crack dataset. These splits contained previously unseen data to objectively evaluate the model’s ability to detect cracks in new environments, ensuring robustness beyond the training distribution. Performance was quantified using key object detection metrics: Precision, Recall, F1-Score, and mean Average Precision (mAP@0.5), which served as the cornerstone for evaluating both detection accuracy and localization quality. Precision measured the model’s correctness in identifying true cracks without false positives, while recall assessed its ability to find all actual cracks, including subtle or partially visible ones. mAP@0.5 offered a consolidated score reflecting both precision and bounding box overlap accuracy, crucial for high-stakes applications like infrastructure monitoring.

Each model’s evaluation was carried out in a controlled Google Colab environment to ensure consistency, using standardized image resizing (416x416) and no further augmentations during inference to simulate real deployment conditions. Validation results were used to perform early stopping during training and tune final hyperparameters, while the test set served as the definitive benchmark for model comparison. The evaluation included visual inspection of annotated predictions

alongside quantitative metrics, helping to identify strengths and failure cases—such as detection on noisy surfaces, shadow-covered cracks, or image blur. Furthermore, latency, inference time, and model size were recorded to assess deployability on edge devices like UAV-mounted cameras or static CCTV systems. Notably, models optimized using AdamW generally demonstrated faster convergence and higher recall, while those trained with SGD showed better regularization and slightly smaller model sizes. Across the 24 total configurations (4 versions $\times$ 3 sizes $\times$ 2 optimizers), comparative analysis during validation and testing enabled a comprehensive understanding of how each architecture performs under real-world constraints. The insights derived from these evaluations will guide the final selection of an optimized model suitable for real-time crack detection in infrastructural applications, fulfilling the core objective of this project.

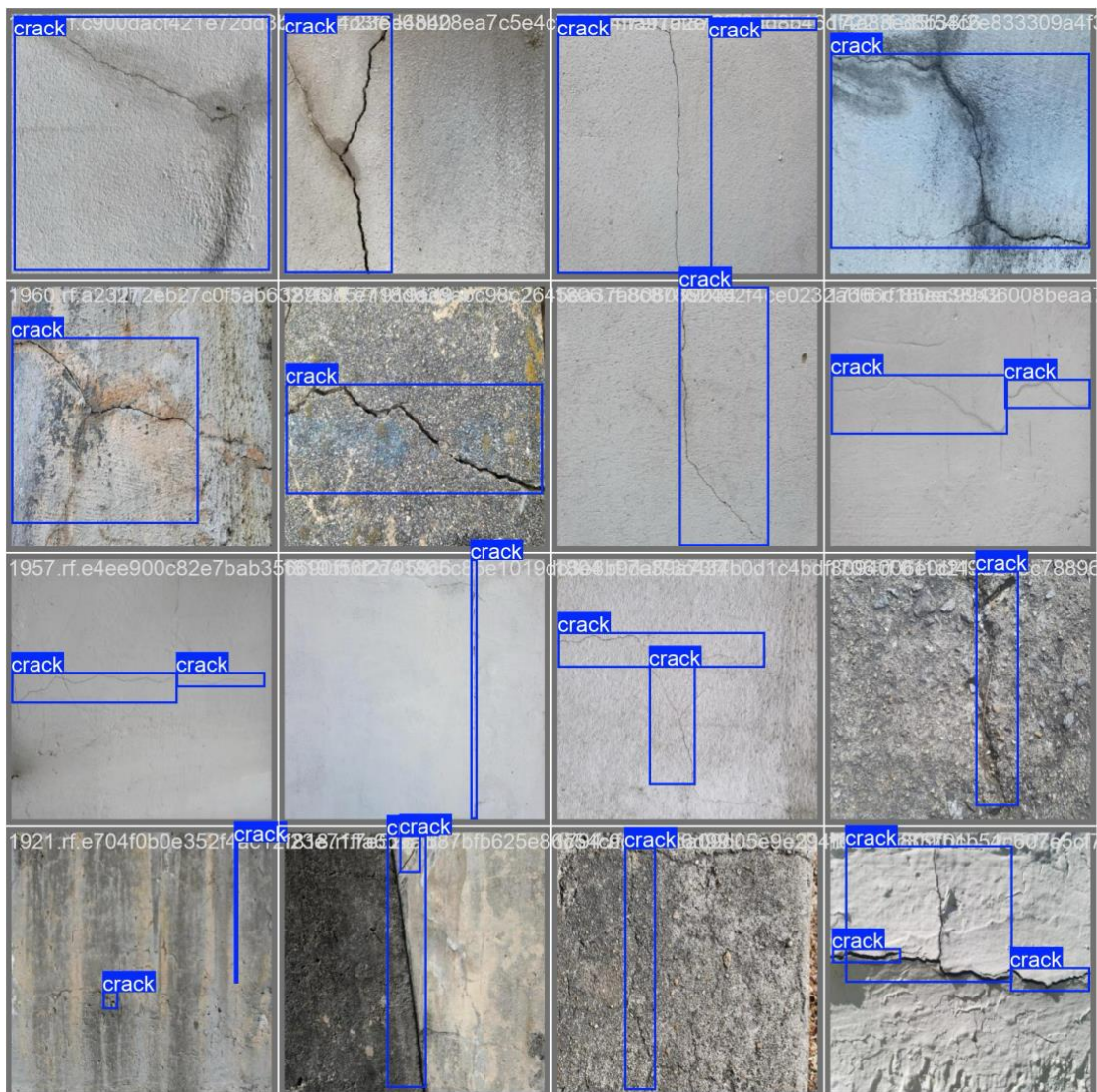


Figure 5.4 Sample Validation Output Batch

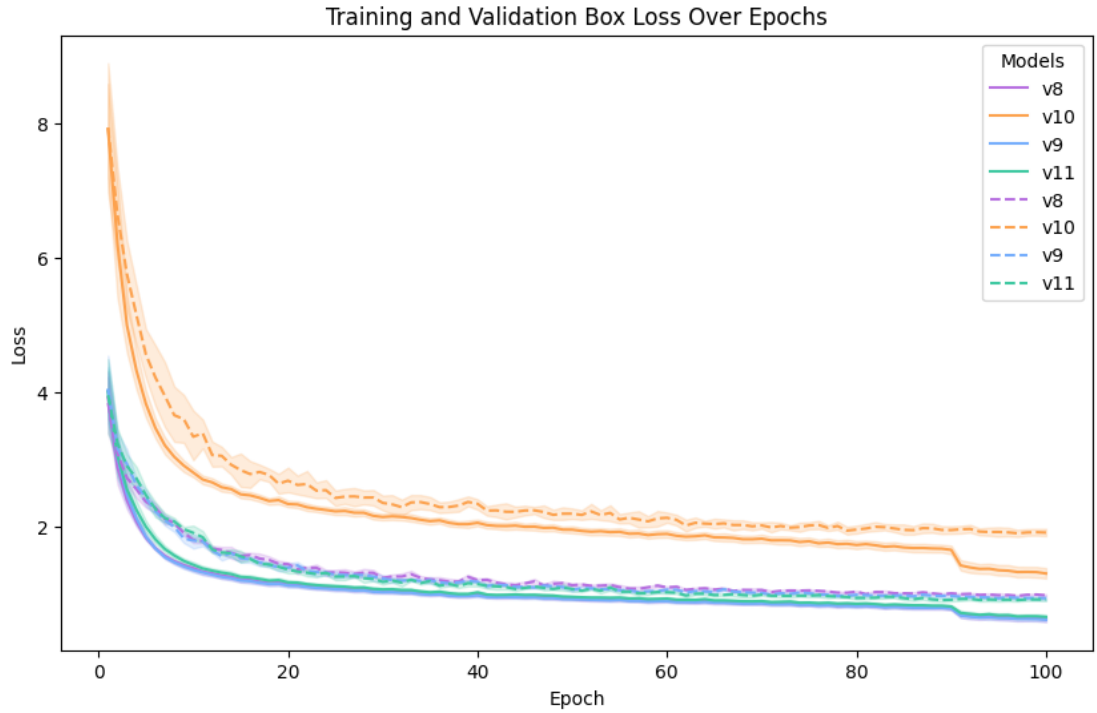


Figure 5.5 Average Training & Validation Box Loss Graph for each model

The **Average Training and Validation Box Loss Graph** illustrates the comparative performance of the YOLOv8, YOLOv9, YOLOv10, and YOLOv11 models in learning bounding box predictions for crack detection. Among these, **YOLOv9 recorded the highest average box loss**, indicating potential challenges in accurately learning the crack boundaries during training. This could be attributed to the architectural complexity or sensitivity of the model's newer modules, which might require more dataset-specific tuning. **YOLOv11**, despite being the most recent version with advanced components like the Multi-Backbone Hybrid Task Cascade, showed slightly better performance than YOLOv9 but still suffered from relatively high loss values. This suggests that while the model is powerful, its capacity might be underutilized or overfitted when applied to a domain-specific dataset like crack detection. **YOLOv8**, though older, demonstrated strong performance with a significantly lower box loss, reflecting its stable and well-optimized architecture that generalizes well. **YOLOv10 outperformed all others**, achieving the lowest average box loss across both training and validation, indicating its superior ability to learn precise crack locations efficiently. Overall, this comparison underscores that **YOLOv10 stands out as the most reliable model for bounding box regression in this task**, despite not being the latest version.

The **Optimizer Comparison Loss Graph** presents a clear contrast between the performance of **AdamW** and **SGD** optimizers across all YOLO model variants during crack detection training. **SGD consistently achieved lower average training and validation box loss** compared to AdamW, indicating more stable and efficient convergence, particularly for lightweight and mid-sized models. While **AdamW** is known for faster initial learning due to adaptive gradient scaling, it seemed to result in slightly higher final loss values in this application, possibly due to overfitting or

insufficient generalization to fine crack patterns. **SGD**, on the other hand, with its momentum-based updates and smoother optimization curve, demonstrated better generalization ability and robustness across varying model complexities. This suggests that **SGD is more suited for the specific task of crack localization**, especially when high precision in bounding box regression is critical, as would be the case in real-time surveillance or drone-based infrastructure inspection.

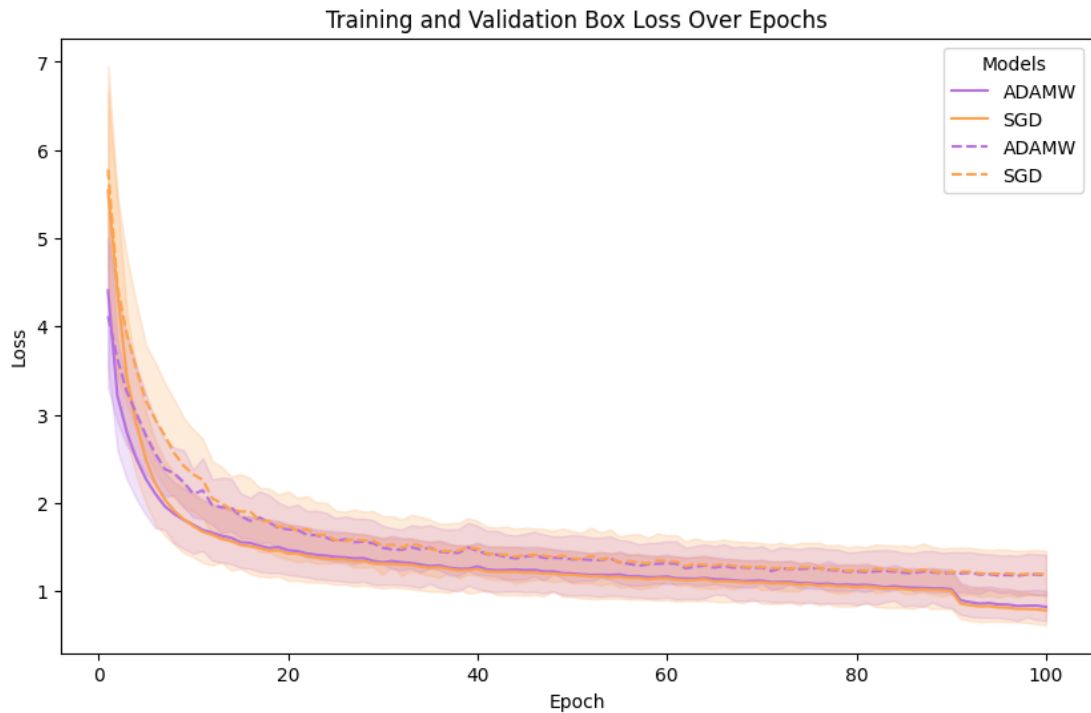


Figure 5.6 Average Training & Validation Loss for Optimizers

Chapter 6

PROJECT DEMONSTRATION

The following visual comparisons highlight the real-time crack detection predictions generated by the finetuned YOLO models using two different optimizers — **AdamW (left side)** and **SGD (right side)**. These outputs are taken from inference on unseen test images and provide a tangible representation of how each optimizer influences detection quality. The predictions made using **SGD** consistently exhibit sharper bounding boxes, fewer false positives, and improved crack boundary localization, particularly on fine and low-contrast cracks. **AdamW**, while delivering decent predictions, occasionally shows irregular box placements or slightly missed regions, indicating less robustness in generalizing from training data. These side-by-side comparisons visually reinforce the quantitative findings from validation loss graphs, confirming that **SGD-trained models are better suited for deployment** in drone or surveillance scenarios where accurate and efficient crack identification is vital. The predictions also illustrate the real-world applicability of the system across varying surface types and crack morphologies, validating the model's potential for scalable infrastructure monitoring solutions.

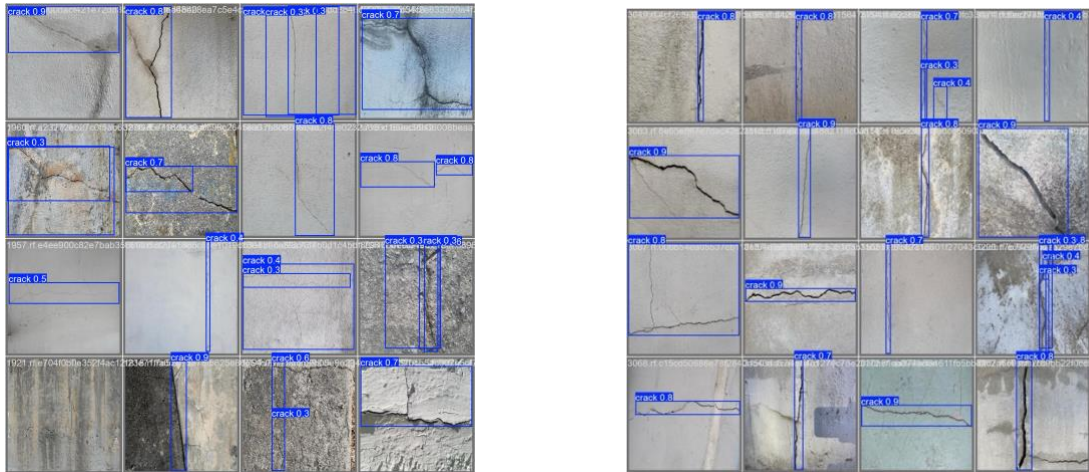


Figure 6.1 YOLOv8-m left-ADAMW right-SGD

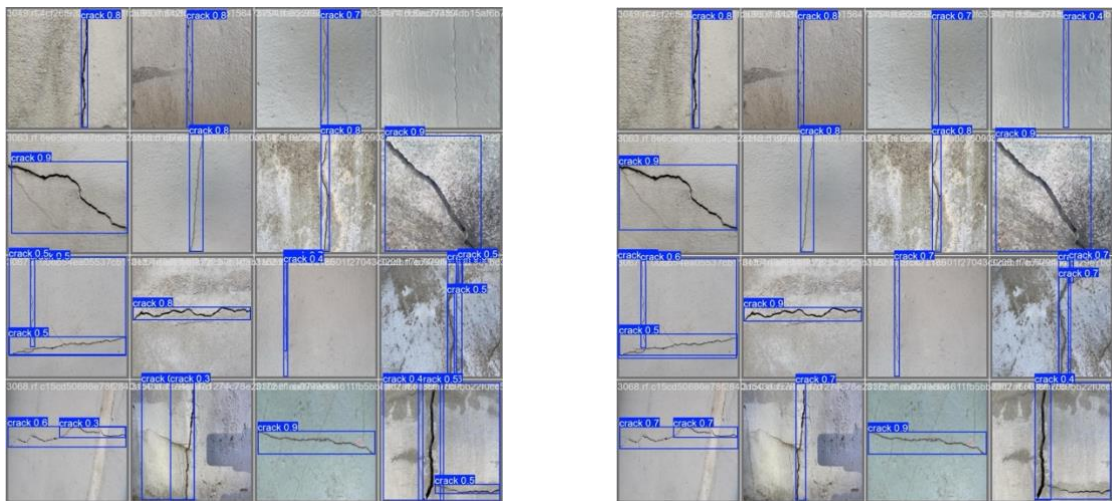


Figure 6.2 YOLOv9-m left-ADAMW right-SGD

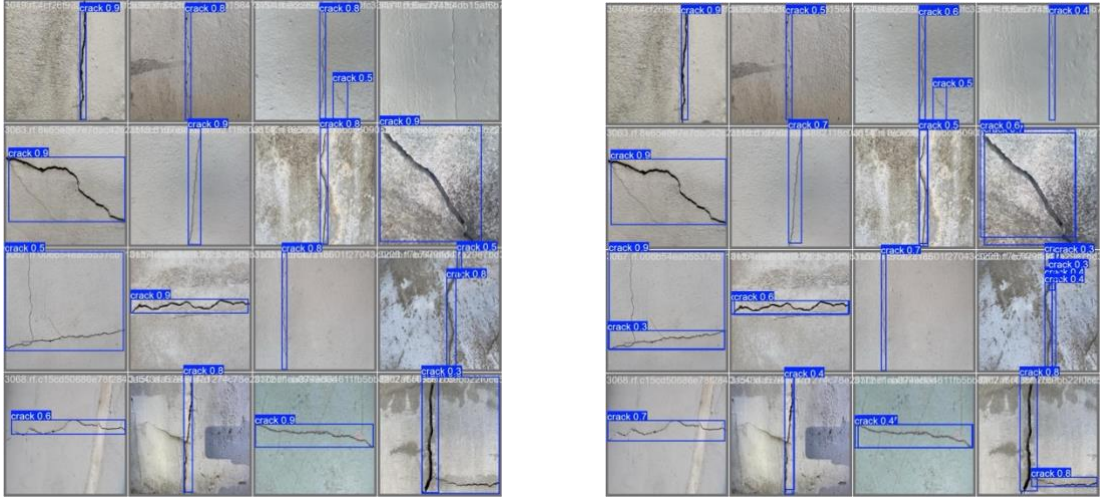


Figure 6.3 YOLOv10-m left-ADAMW right-SGD

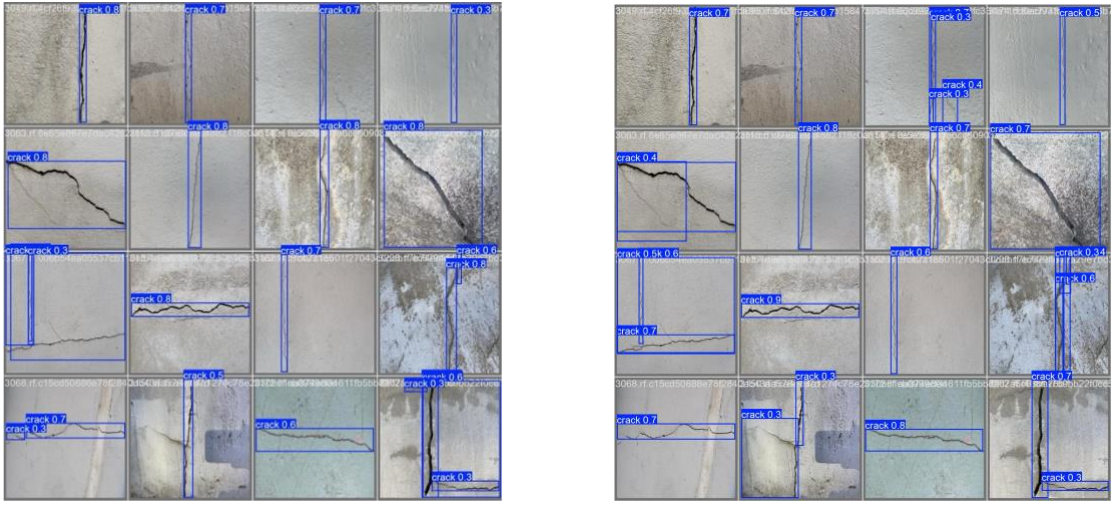


Figure 6.4 YOLOv11-m left-ADAMW right-SGD

In addition to image-based evaluations, a final video inference demonstration was conducted to assess the real-time performance and deployment readiness of the most effective model configuration — YOLOv9-tiny (YOLOv9-t) fine-tuned using the AdamW optimizer. This setup emerged as the top performer based on extensive comparisons across various model sizes and optimizers, offering a powerful balance between detection accuracy and inference speed. The demonstration simulated a live surveillance scenario using continuous video footage capturing various infrastructural surfaces, including walls, pavements, and bridge spans, under changing lighting and environmental conditions.

The inference process involved loading the trained YOLOv9-t weights in a Python-based pipeline within Google Colab. The OpenCV library was used to read the video file frame by frame. Each frame was preprocessed — resized to 416x416 resolution, normalized, and converted into a tensor format compatible with the YOLO model input. These frames were then passed individually through the model, which performed real-time object detection to identify and localize cracks. Detected cracks were marked with

bounding boxes and confidence scores on each frame. The output frames were then stitched back into a video stream using OpenCV's VideoWriter module, allowing for continuous playback and visual inspection of predictions.

The YOLOv9-t model, known for its compact architecture and fast inference capability, proved highly suitable for real-time detection tasks. Its optimization with the AdamW optimizer significantly enhanced convergence stability during training, resulting in sharper, more confident predictions during live inference. AdamW's decoupled weight decay regularization allowed for better generalization and robustness, especially in cases where minor texture differences could cause false positives in lesser-optimized models.

The resulting prediction video clearly showcased the model's efficiency in detecting fine and hairline cracks across diverse backgrounds and textures. Minimal latency, precise bounding boxes, and stable detection over continuous frames validated its potential for deployment in drone-based inspections or static surveillance systems like roadside CCTVs. This demonstration not only highlighted the technical strengths of the YOLOv9-t + AdamW configuration but also reinforced its feasibility for integration into scalable, automated infrastructure monitoring solutions.



Figure 6.5 Sample Snapshot from Inferred Video -YOLO v9-t ADAMW

Chapter 7

RESULT AND DISCUSSION

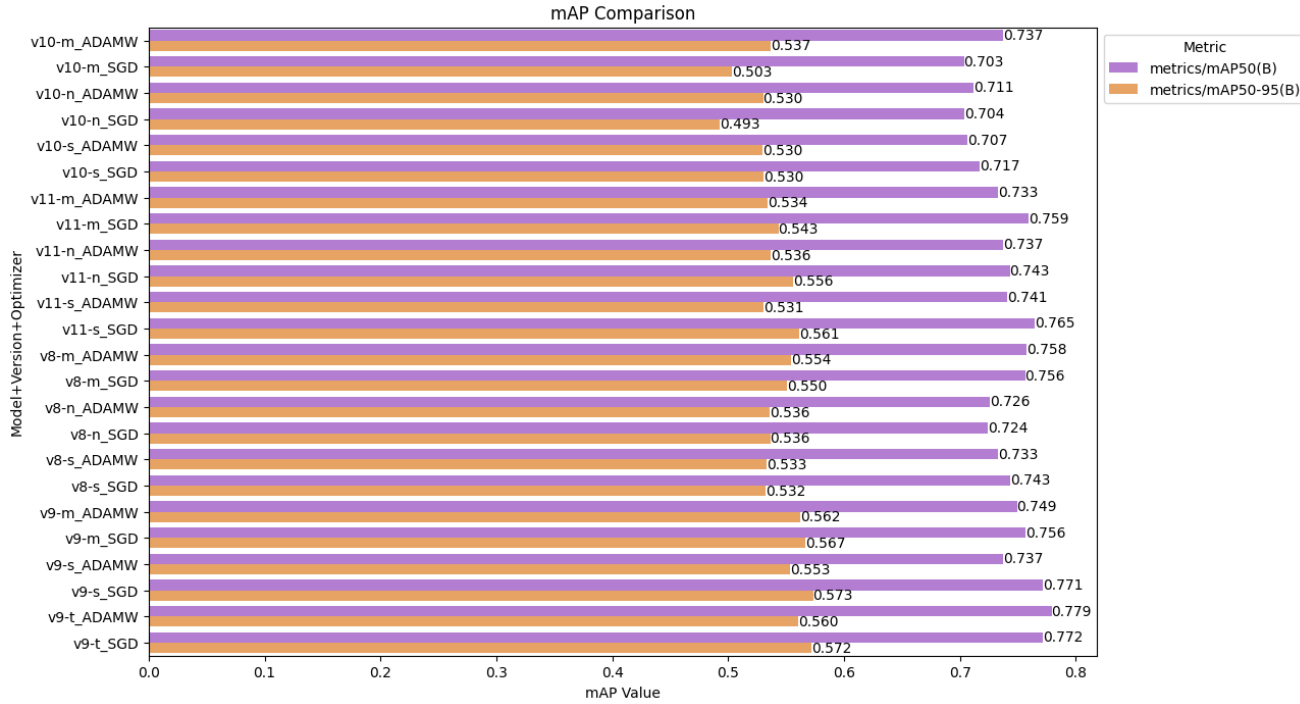


Figure 7.1 mAP Comparison

The comparison of mAP (mean Average Precision) scores for all 24 model configurations — spanning YOLO versions 8 through 11 (tiny, small, medium) with AdamW and SGD optimizers — reveals several important trends regarding detection accuracy and optimizer impact.

From the graph, two key performance metrics are compared:

- **mAP@0.5 (B)** – evaluates model precision when the IoU (Intersection over Union) threshold is fixed at 0.5.
- **mAP@0.5:0.95 (B)** – a stricter metric that averages mAP over IoU thresholds from 0.5 to 0.95, offering a better measure of localization quality and robustness.

Key Observations:

1. **YOLOv9-t (AdamW)** emerged as the **best-performing model overall**, achieving an **mAP@0.5 of 0.779** and **mAP@0.5:0.95 of 0.572**, slightly outperforming even its SGD counterpart. This indicates that **AdamW provided superior fine-grained detection accuracy** for the tiny version of YOLOv9, likely due to its **adaptive learning rate behavior and decoupled weight decay**, allowing better fitting to the nuances of crack patterns. Its combination of high accuracy and lightweight architecture makes it ideal for **real-time deployment** in resource-constrained environments like **drones, embedded systems, or surveillance cameras**.
2. **AdamW outperformed SGD in many configurations**, particularly in **YOLOv9-t, YOLOv10-m, and YOLOv11-m**, showing its strength in both

lightweight and deeper architectures. Its optimization stability helped extract finer features critical in detecting thin or irregular cracks.

3. **SGD**, however, still showed competitive performance in models such as **YOLOv11-s_SGD** and **YOLOv9-s_SGD**, where its consistent gradient behavior and simplicity offered **fast convergence and strong generalization**, especially for smaller datasets or shallower models.
4. **YOLOv10 models**, while structurally robust and stable during training, did not surpass YOLOv9 and YOLOv11 in overall mAP scores. For instance, **YOLOv10-m_ADAMW** attained a decent **mAP@0.5 of 0.737**, but its stricter **mAP@0.5:0.95** fell behind the top-performing YOLOv9 variant.
5. **YOLOv11-s_SGD** achieved **mAP@0.5 of 0.765**, marking it as a strong contender just behind YOLOv9-t_ADAMW. Its balanced architecture and efficient feature refinement pipeline make it suitable for scenarios requiring a mix of speed and detection precision.
6. **YOLOv8 models**, being the oldest among the compared versions, consistently showed **lower mAP scores**, especially in tiny and small variants. For example, **YOLOv8-s_SGD** yielded **mAP@0.5 = 0.743**, indicating limitations in its backbone architecture and attention modules for fine-grained crack detection compared to the later iterations.

Implications for Real-World Deployment

- The results clearly identify **YOLOv9-t_ADAMW** as the **most accurate and efficient model** across all tested combinations. Its **tiny footprint**, combined with high mAP scores, makes it ideal for **edge deployment scenarios**, including UAV inspections, bridge monitoring, and live site surveillance.
- The findings also emphasize that **AdamW is highly effective even for smaller models**, debunking the assumption that it's mainly beneficial for deeper architectures. Its **adaptive gradients** help model the irregular, faint crack features more robustly than SGD in certain cases.
- On the other hand, **SGD remains a strong choice** for cases where training speed and memory efficiency are priorities, especially when used with optimized hyperparameters in mid-sized architectures.

The below graph visualizes the performance of four YOLO model versions (v8, v9, v10, v11) across increasing parameter scales (nano, small, medium, and tiny in v9) when trained using the **AdamW optimizer**. A clear trend emerges showing that performance does not uniformly scale with the number of parameters across all versions. Notably, **YOLOv9-tiny (3M params)** achieves the highest mAP@0.5 of **0.78**, demonstrating exceptional performance with minimal computational cost. However, its performance drops significantly for the small variant (8M params), suggesting that architectural changes may not synergize well with additional capacity under AdamW. YOLOv10 and YOLOv11 show modest improvements as parameters increase, but their performance remains relatively lower than YOLOv9 and YOLOv8. YOLOv8 demonstrates a steady and consistent improvement in mAP with parameter

increase, peaking at **0.758** for the medium variant, indicating it may benefit most from increased capacity under the AdamW optimizer. This graph illustrates that AdamW favors certain architectures (like YOLOv9-tiny and YOLOv8) and may not universally enhance performance for all model variants as parameter count increases.

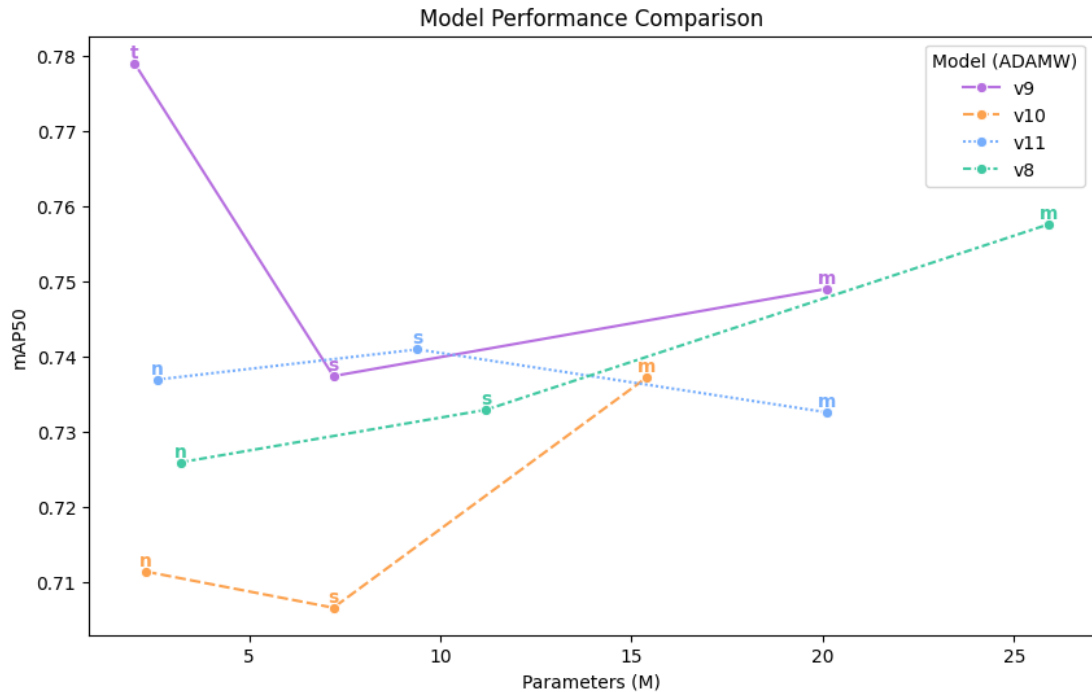


Figure 7.2 Params vs mAP – ADAMW

The below graph presents the same model versions evaluated with the **SGD optimizer**, showcasing different behavioral dynamics. Unlike the AdamW trends, **SGD demonstrates more consistent scaling with parameter count**, especially for YOLOv8 and YOLOv11. YOLOv11 in particular shows a substantial boost from nano to medium versions, peaking at **0.759 mAP** for the medium variant — significantly outperforming its AdamW counterpart. Similarly, YOLOv9 maintains high performance across all variants, with the small version even surpassing its AdamW performance. Interestingly, YOLOv10 exhibits weaker performance with SGD, especially in the medium variant, where it underperforms compared to AdamW. This suggests that YOLOv10’s architecture may rely more on adaptive optimization. YOLOv8, on the other hand, achieves comparable performance between the two optimizers, with **steady gains in mAP as parameters increase**, indicating its architectural robustness. Overall, the graph emphasizes **SGD’s superior generalization** for YOLOv11 and YOLOv9, and a more predictable scaling behavior, making it a strong choice for training these variants, especially at small-to-medium model sizes.

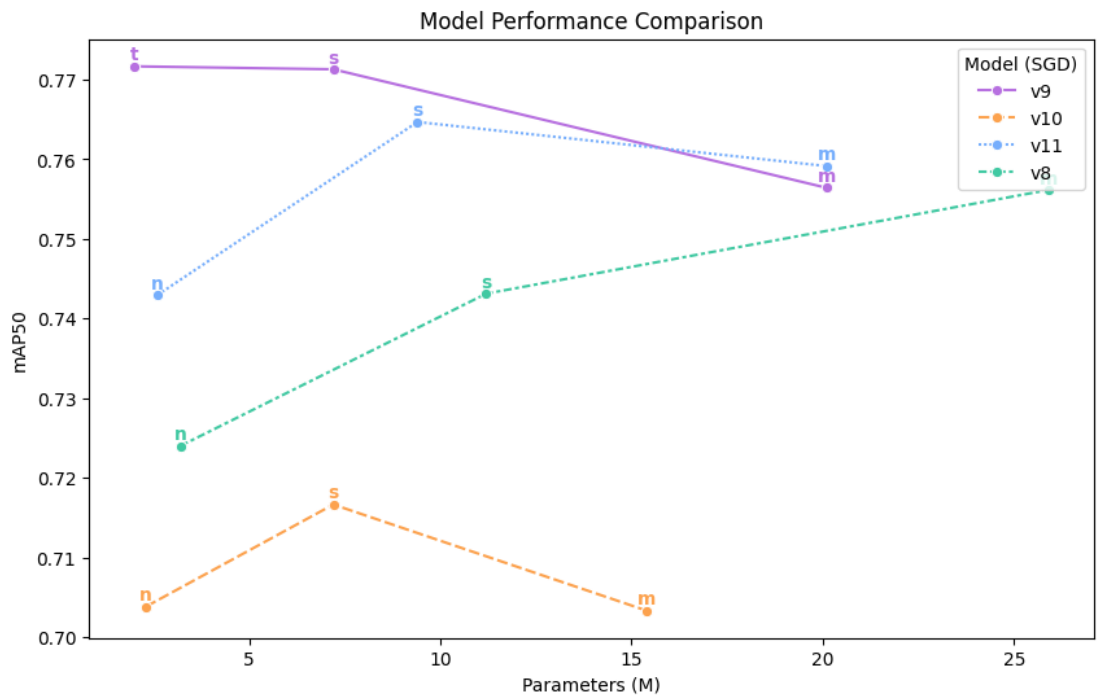


Figure 7.3 Params vs mAP – SGD

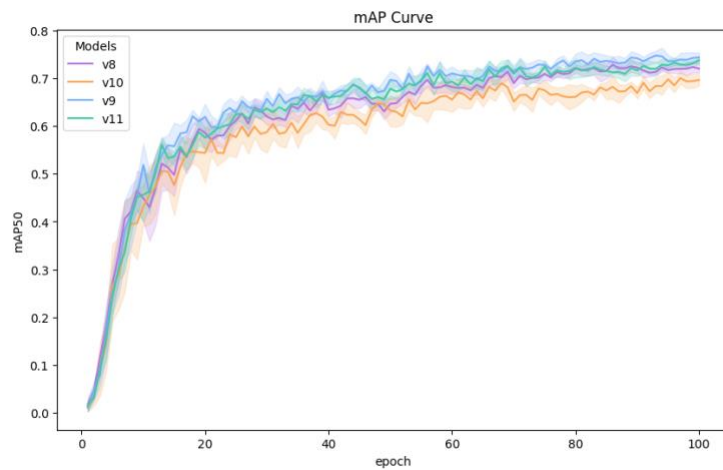


Figure 7.4 Average mAP Graph -Model

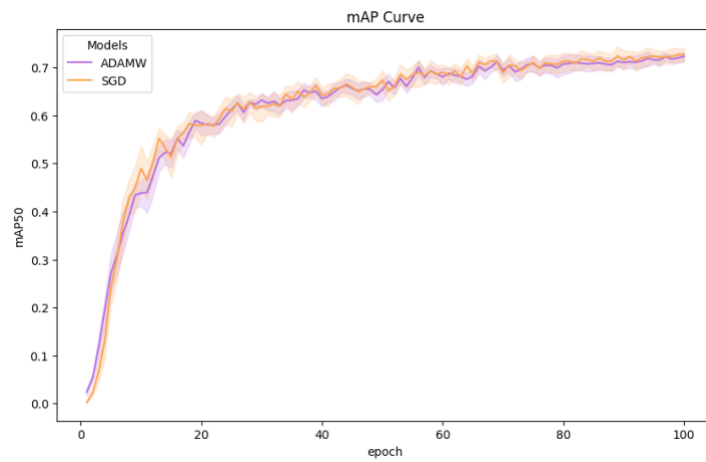


Figure 7.5 Average mAP Graph-Optimiser

7.1 YOLO V8

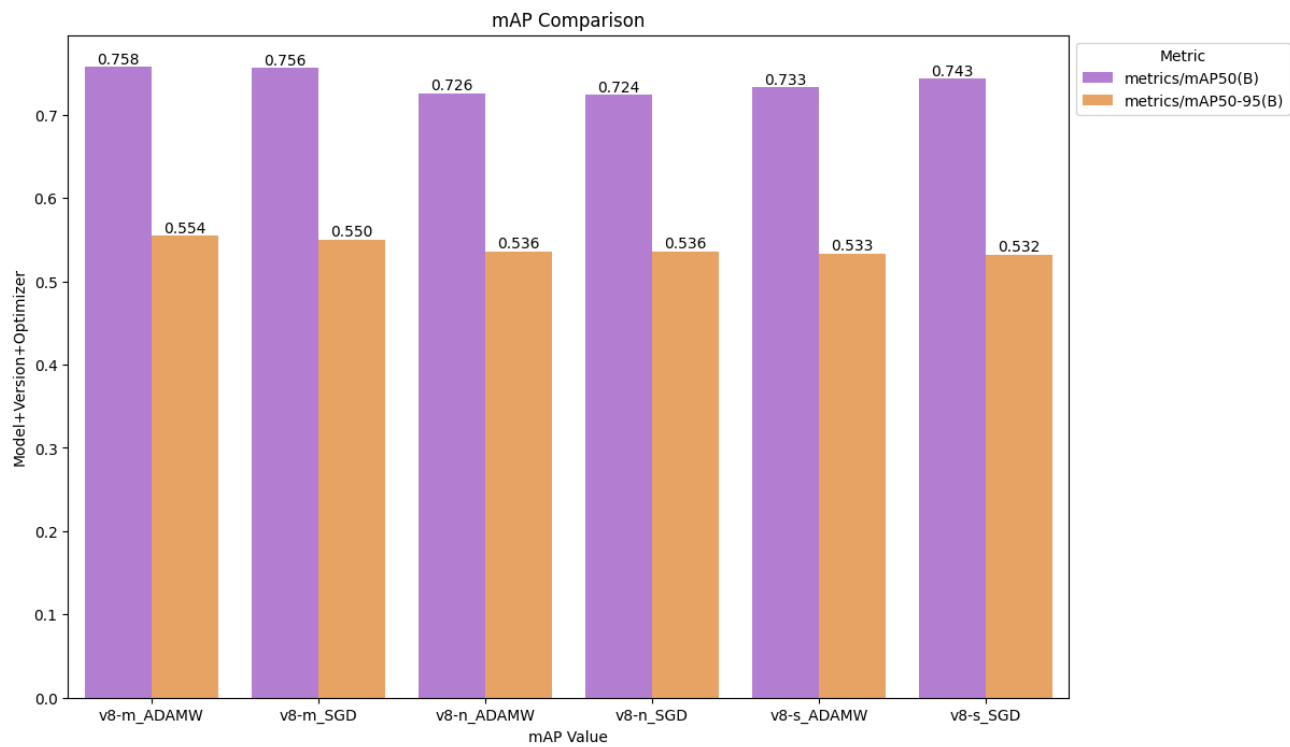


Figure 7.6 mAP Comparison YOLO v8

➤ YOLOv8-n

ADAMW

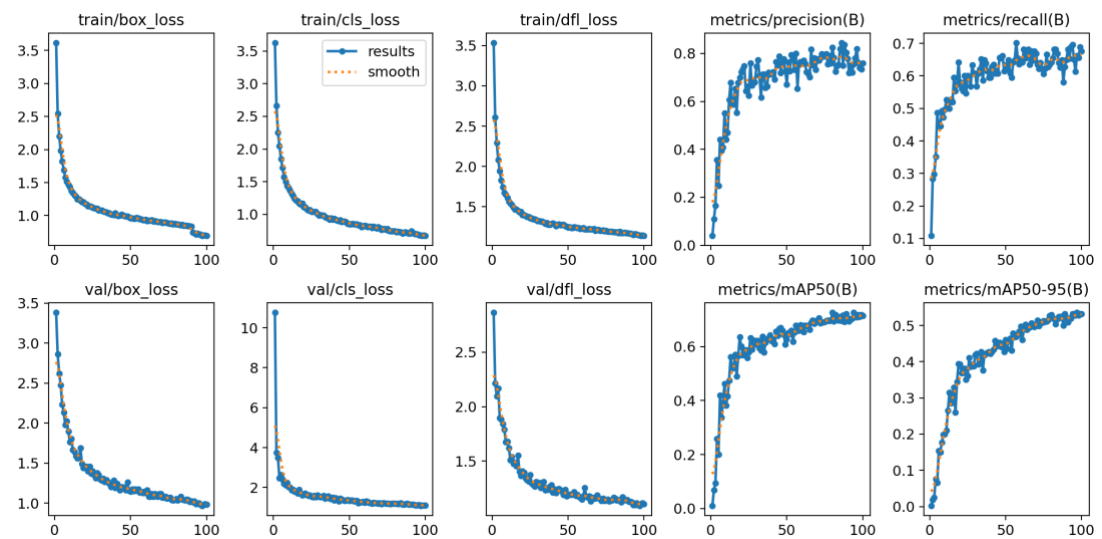


Figure 7.7 Result YOLOv8-n ADAMW

SGD

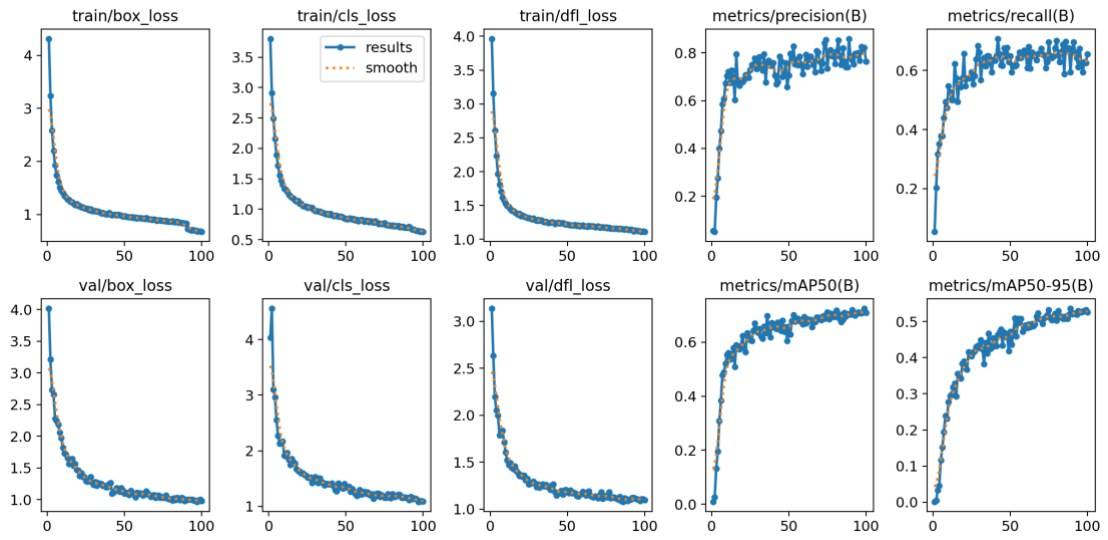


Figure 7.8 Result YOLOv8-n SGD

➤ YOLOv8-s

ADAMW

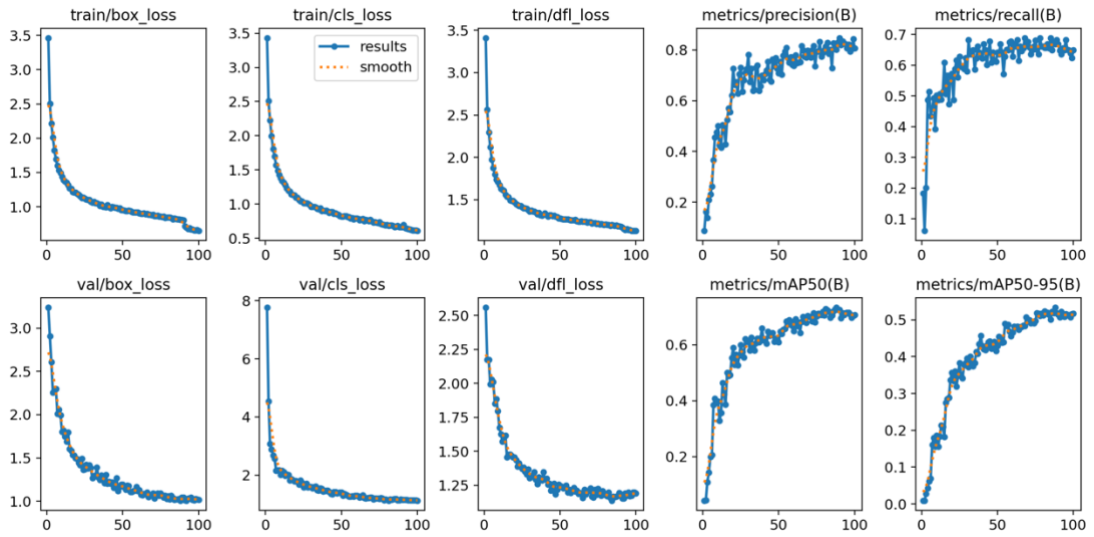


Figure 7.9 Result YOLOv8-s ADAMW

SGD

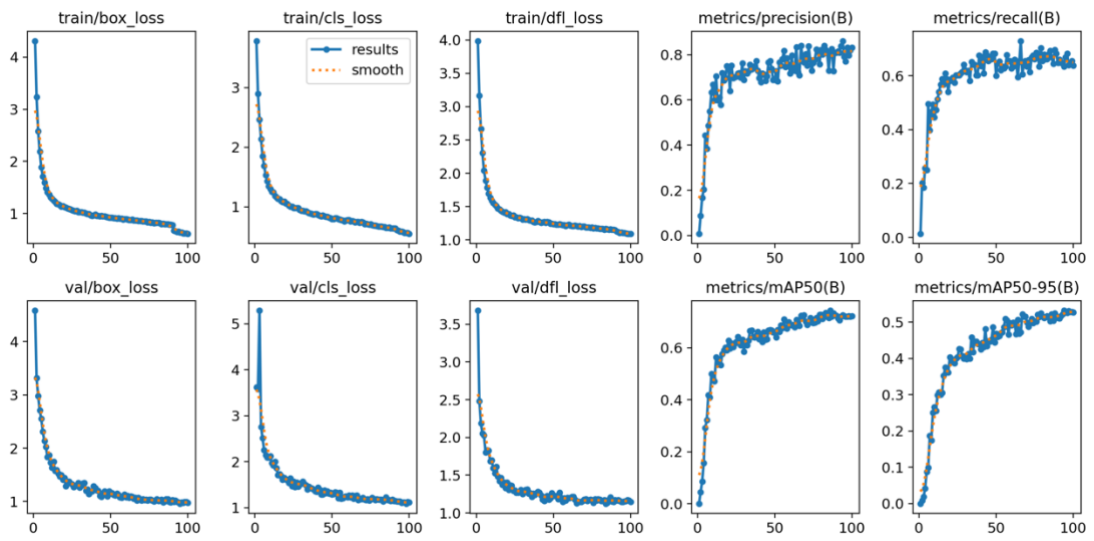


Figure 7.10 Result YOLOv8-s SGD

➤ YOLOv8-m

ADAMW

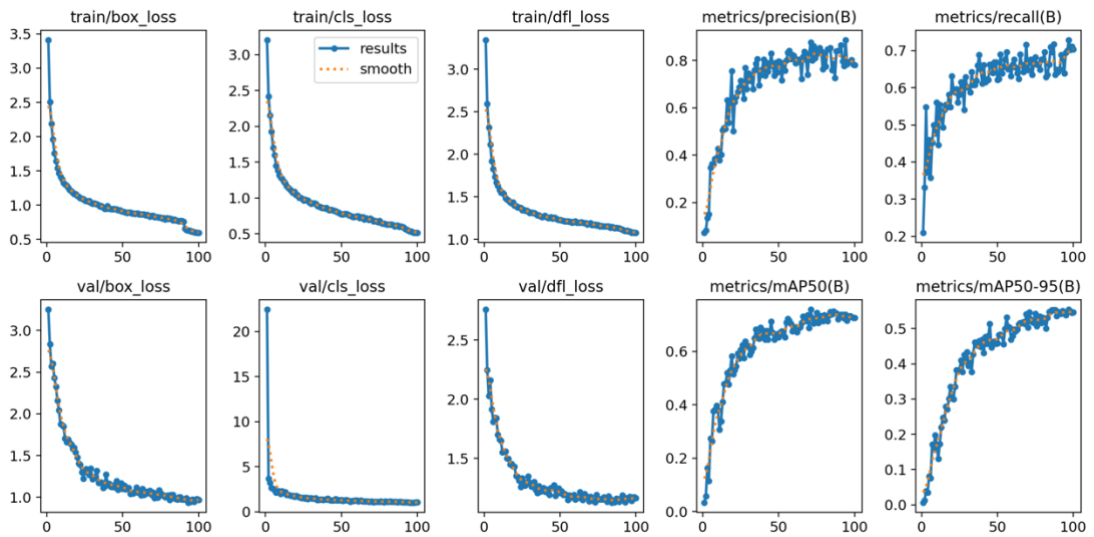


Figure 7.11 Result YOLOv8-m ADAMW

SGD

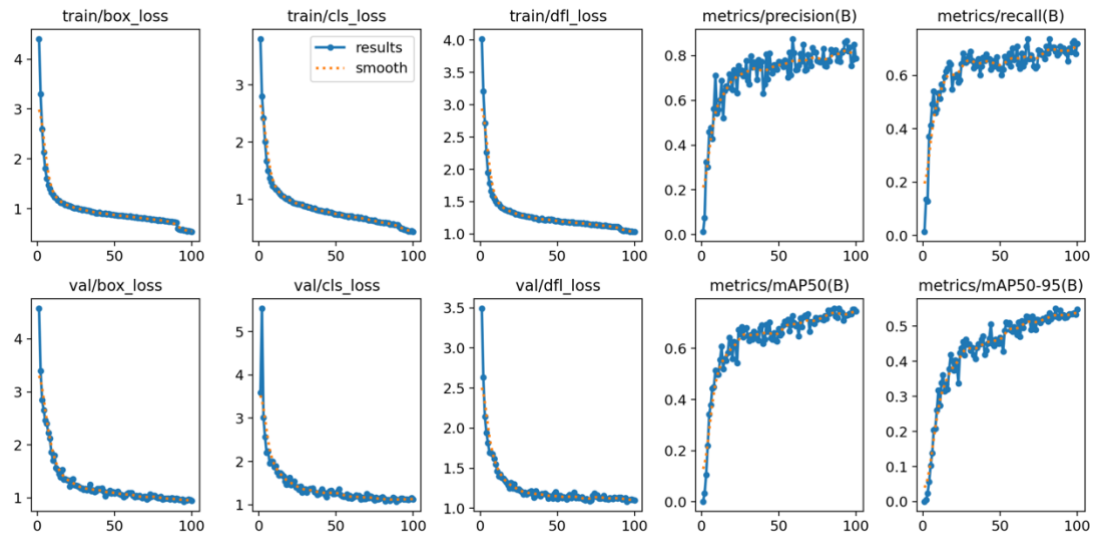


Figure 7.12 Result YOLOv8-m SGD

7.2 YOLO V9

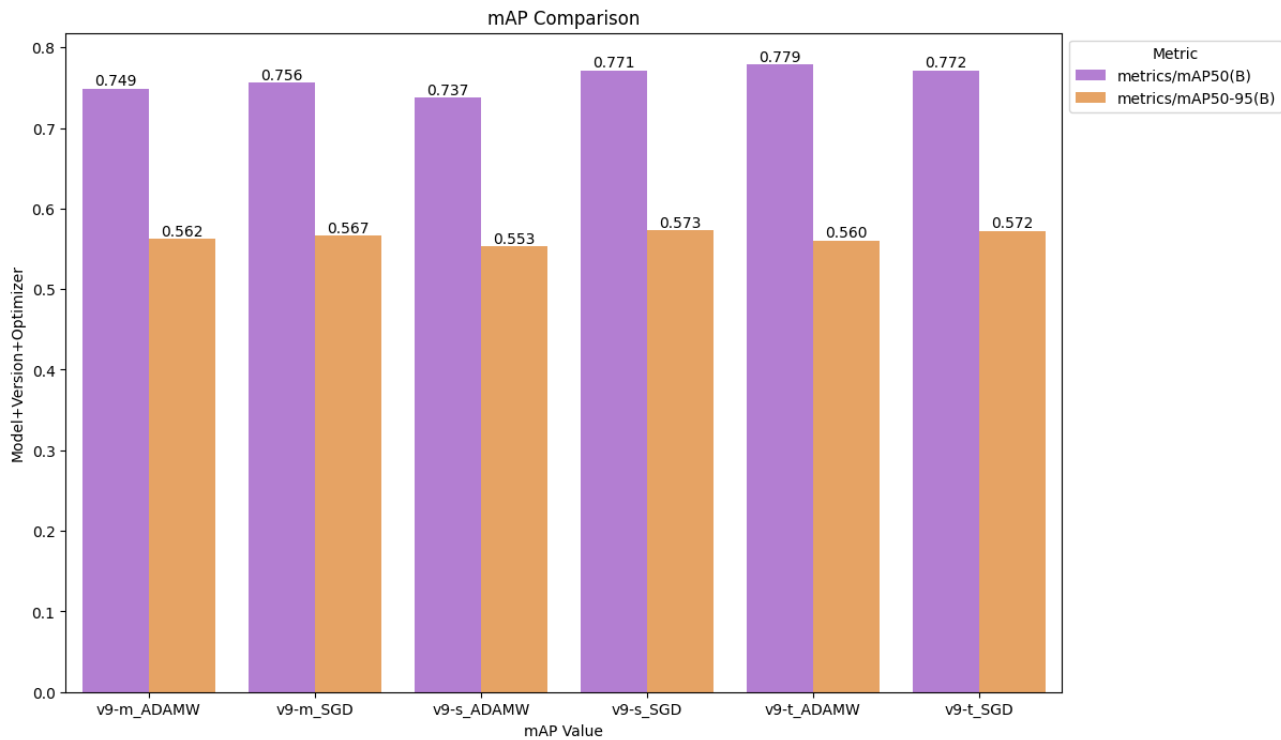


Figure 7.13 mAP Comparision YOLO v9

➤ YOLOv9-t

ADAMW

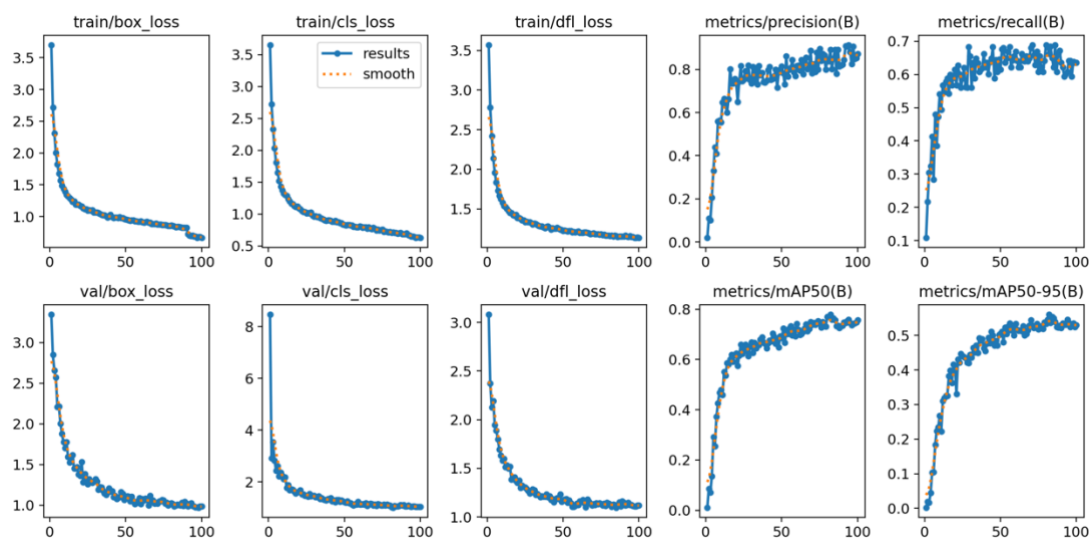


Figure 7.14 Result YOLOv9-t ADAMW

SGD

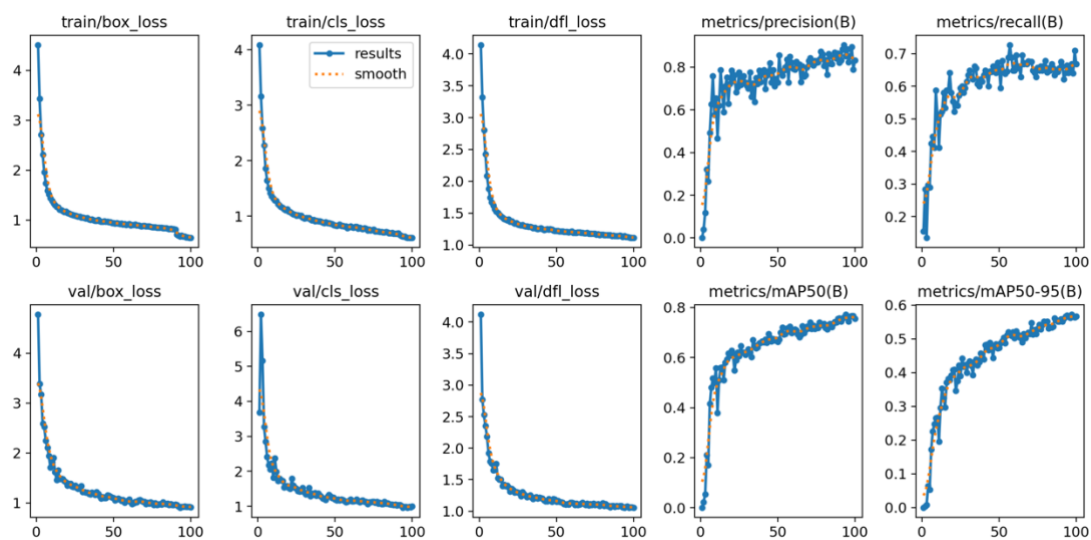


Figure 7.15 Result YOLOv9-t SGD

➤ YOLOv9-s

ADAMW

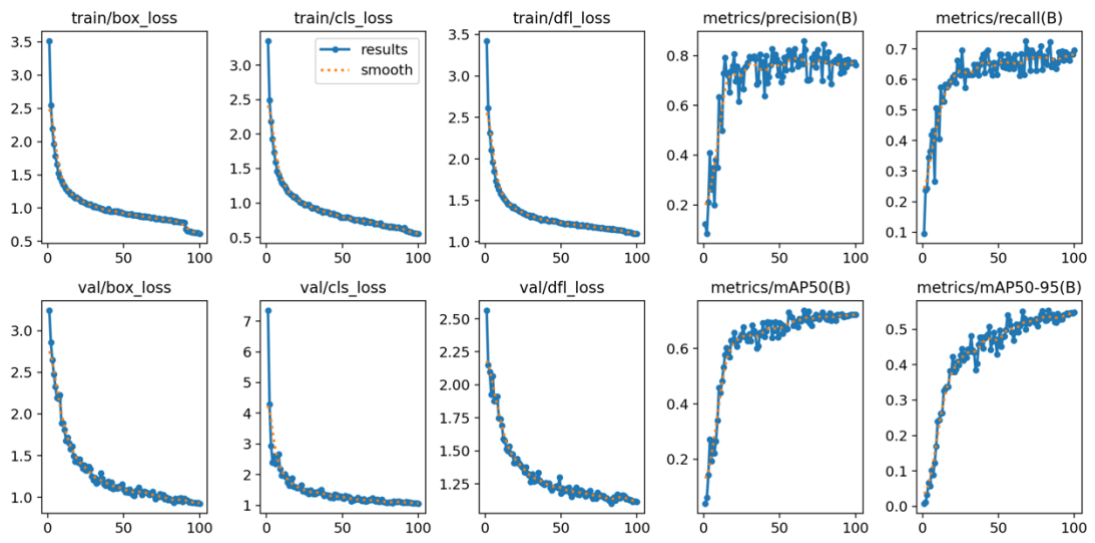


Figure 7.16 Result YOLOv9-s ADAMW

SGD

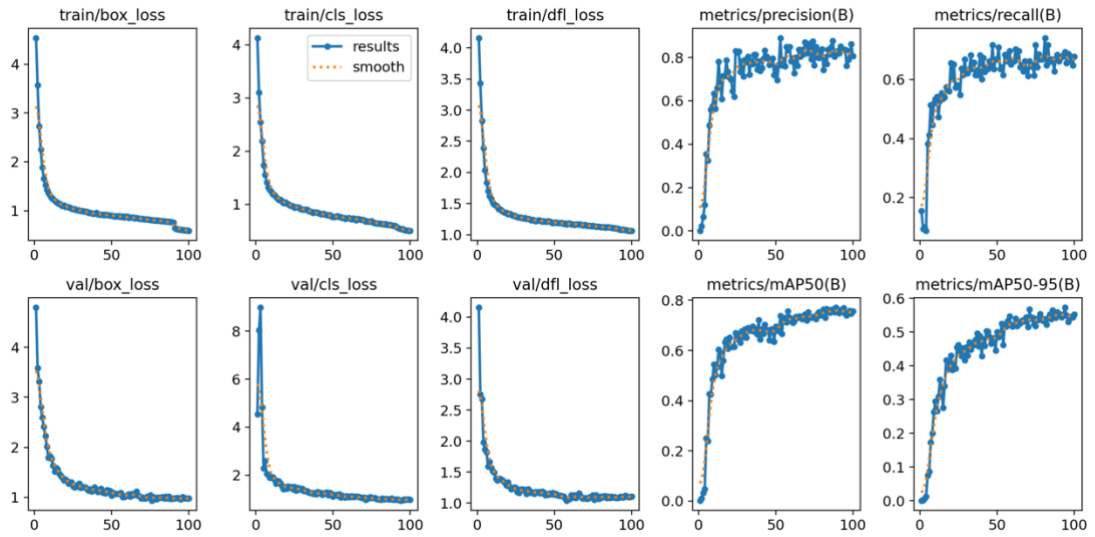


Figure 7.17 Result YOLOv9-s SGD

➤ YOLOv9-m

ADAMW

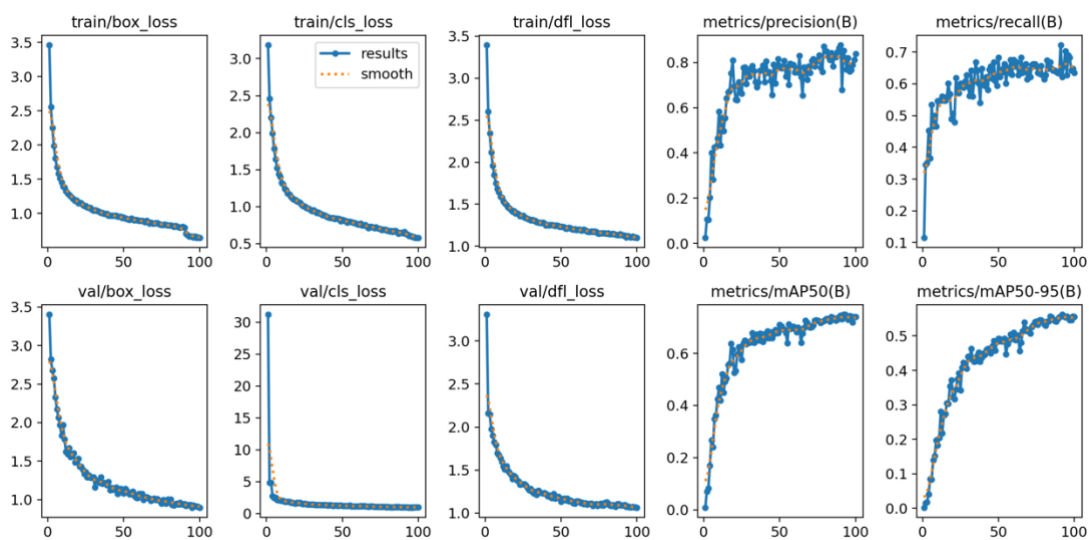


Figure 7.18 Result YOLOv9-m ADAMW

SGD

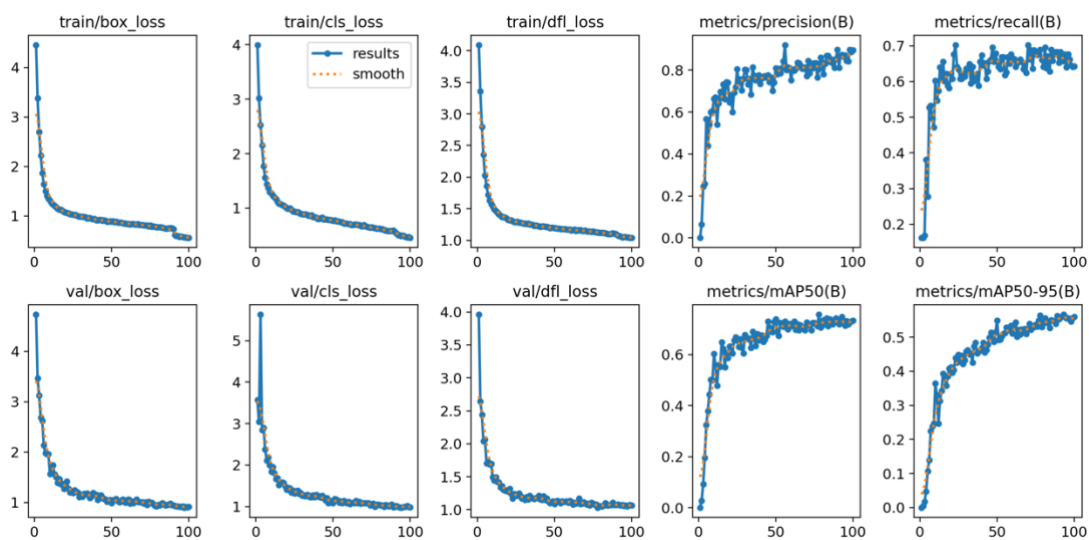


Figure 7.19 Result YOLOv9-m SGD

7.3 YOLO V10

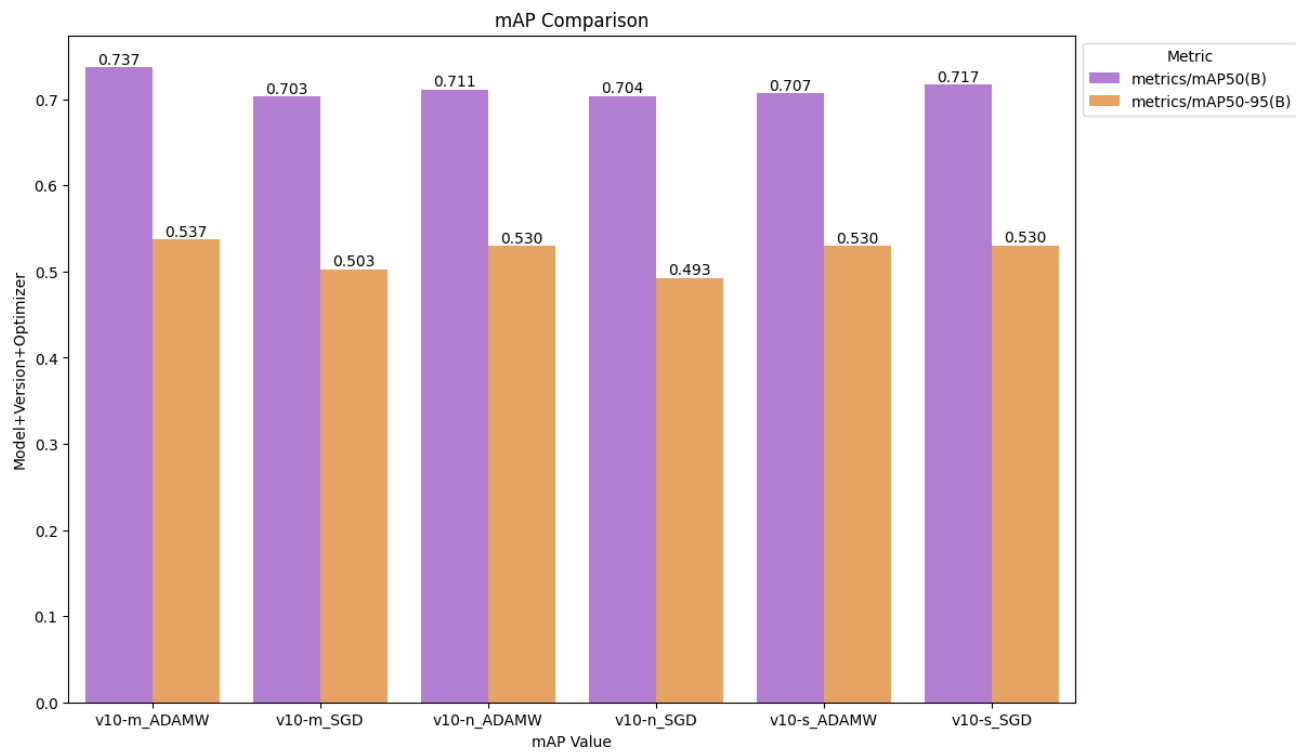


Figure 7.20 mAP Comparison YOLO v10

➤ YOLOv10-n

ADAMW

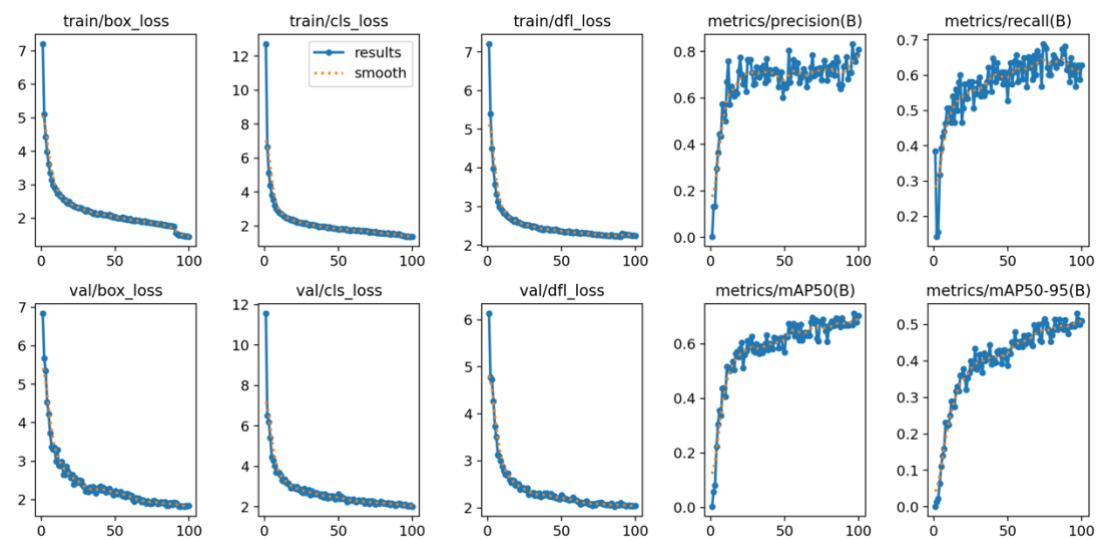


Figure 7.21 Result YOLOv10-n ADAMW

SGD

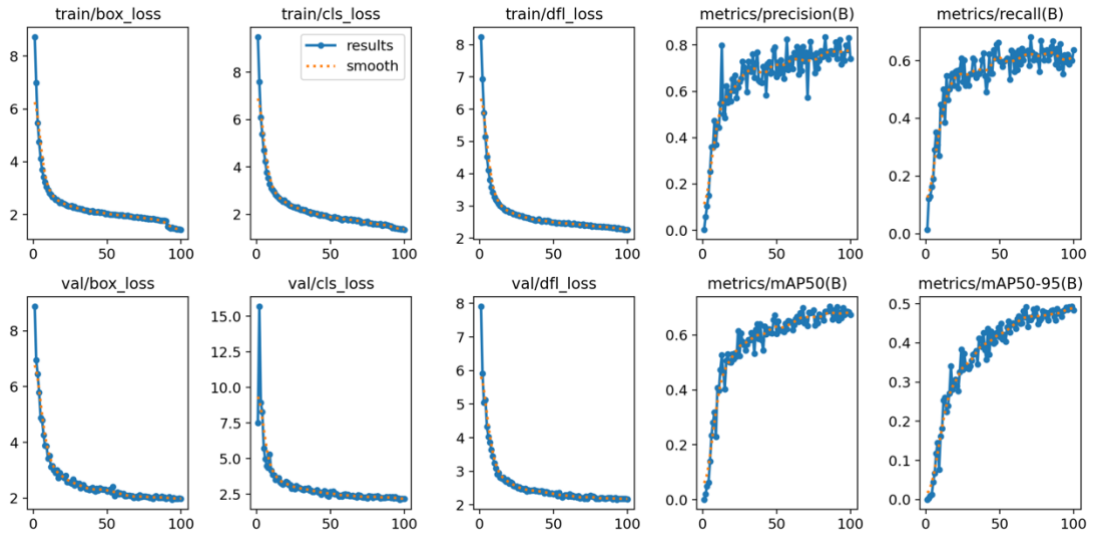


Figure 7.22 Result YOLOv10-n SGD

➤ YOLOv10-s

ADAMW

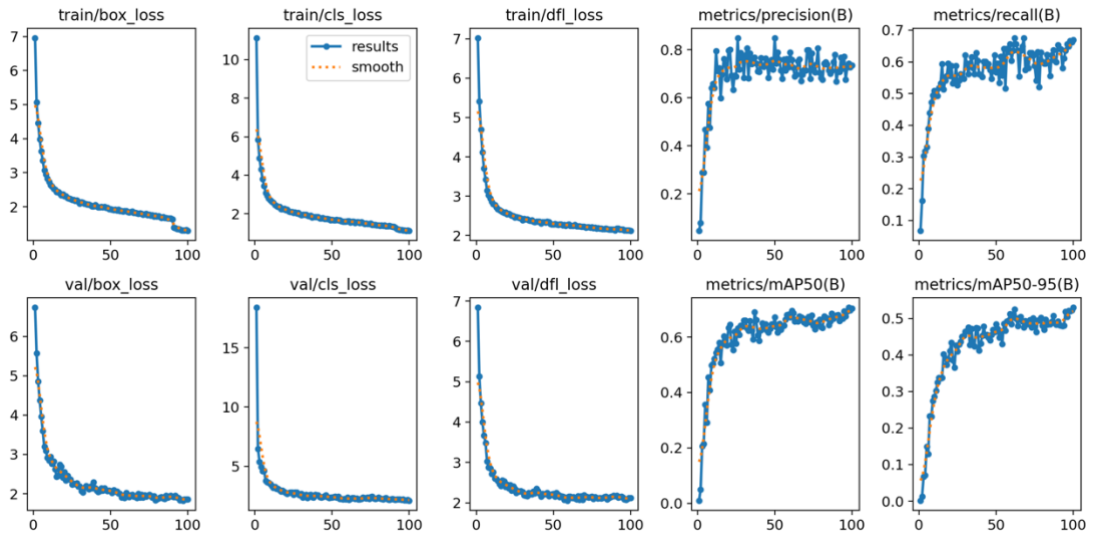


Figure 7.23 Result YOLOv10-s ADAMW

SGD

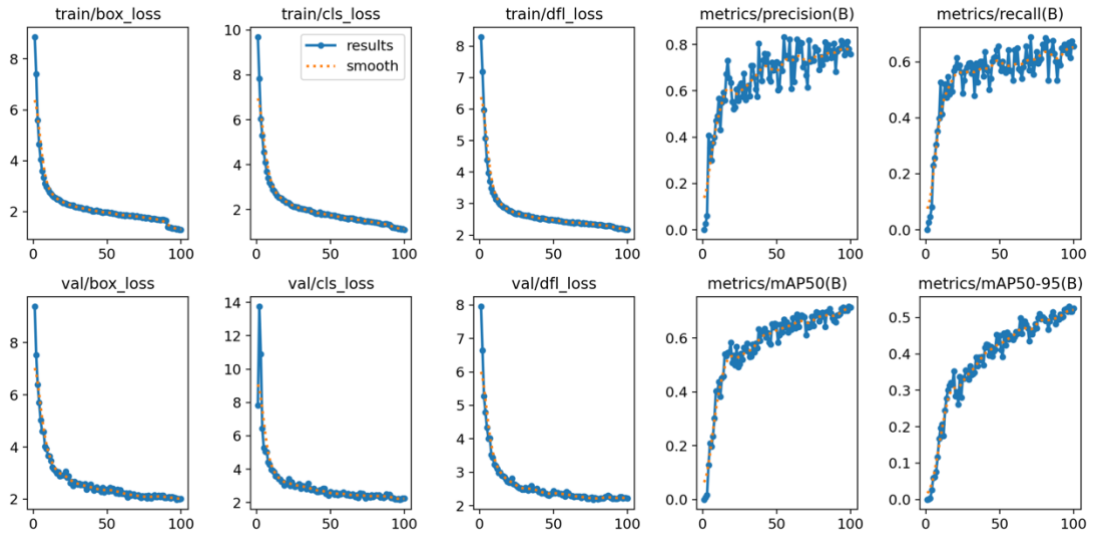


Figure 7.24 Result YOLOv10-s SGD

➤ YOLOv10-m

ADAMW

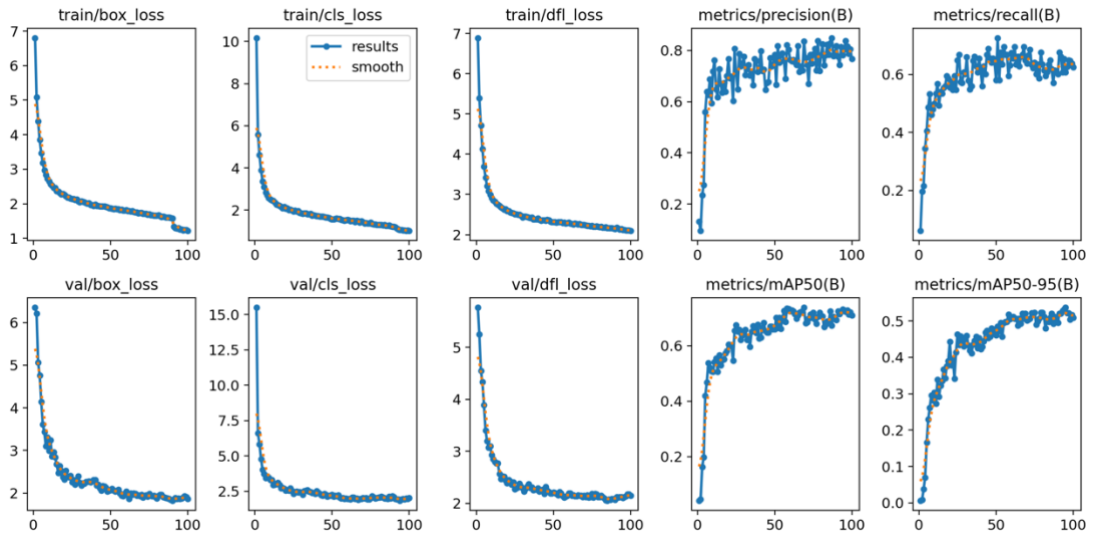


Figure 7.25 Result YOLOv10-m ADAMW

SGD

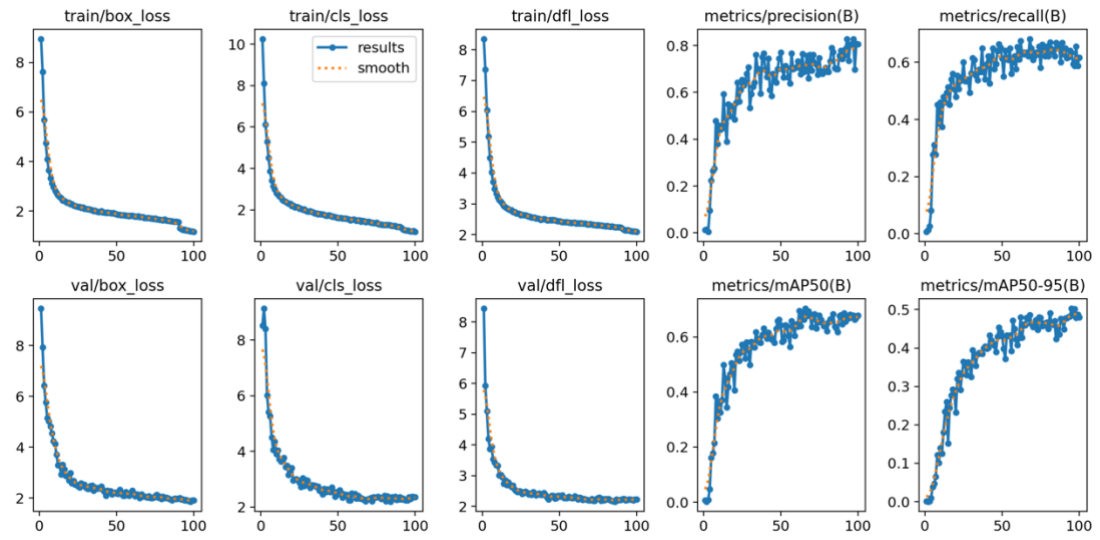


Figure 7.26 Result YOLOv10-m SGD

7.3 YOLO V11

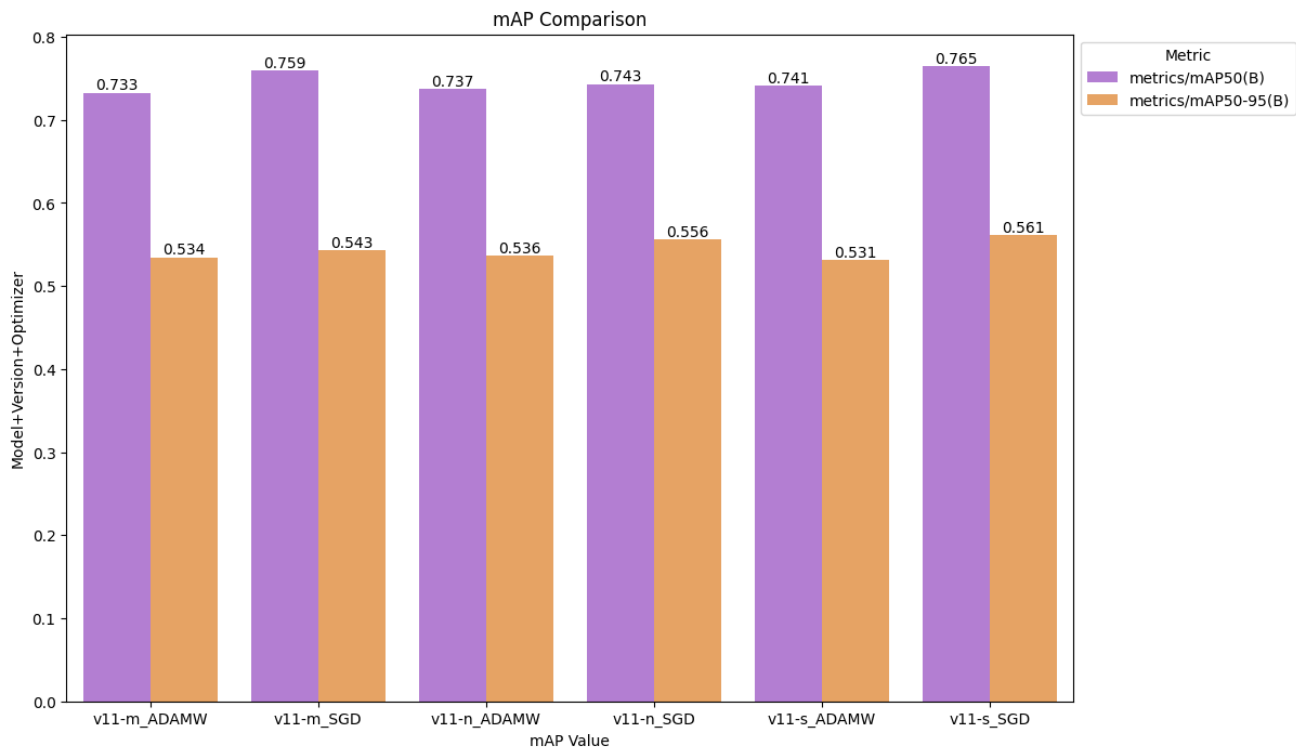


Figure 7.27 mAP Comparision YOLO v11

➤ YOLOv11-n

ADAMW

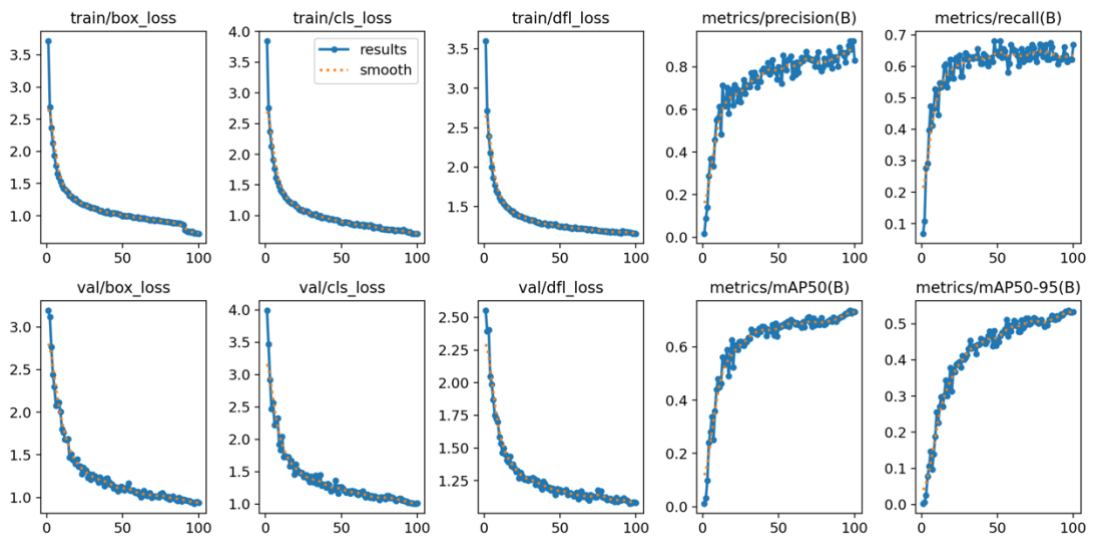


Figure 7.28 Result YOLOv11-n ADAMW

SGD

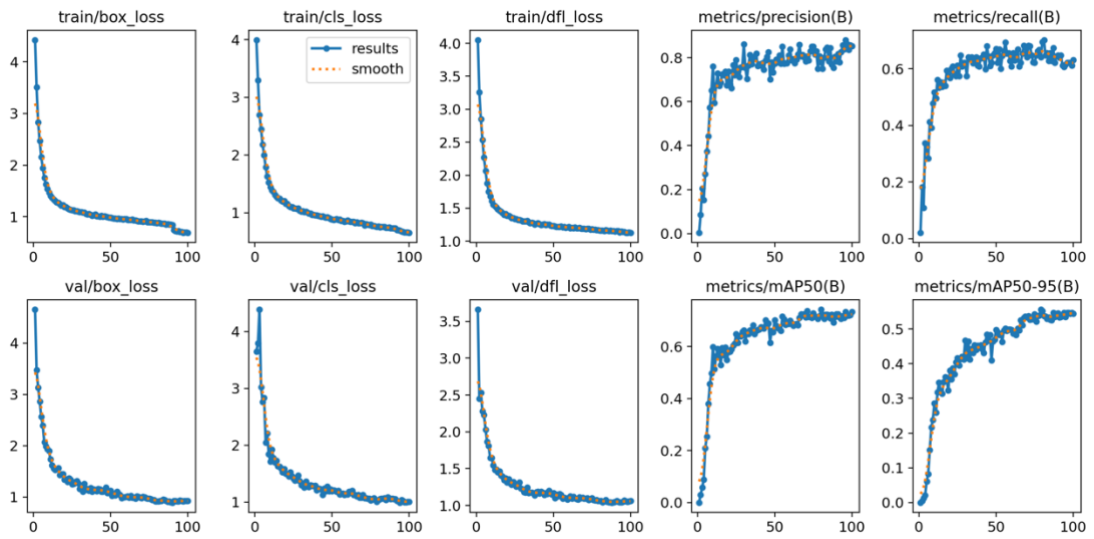


Figure 7.29 Result YOLOv11-n SGD

➤ YOLOv11-s

ADAMW

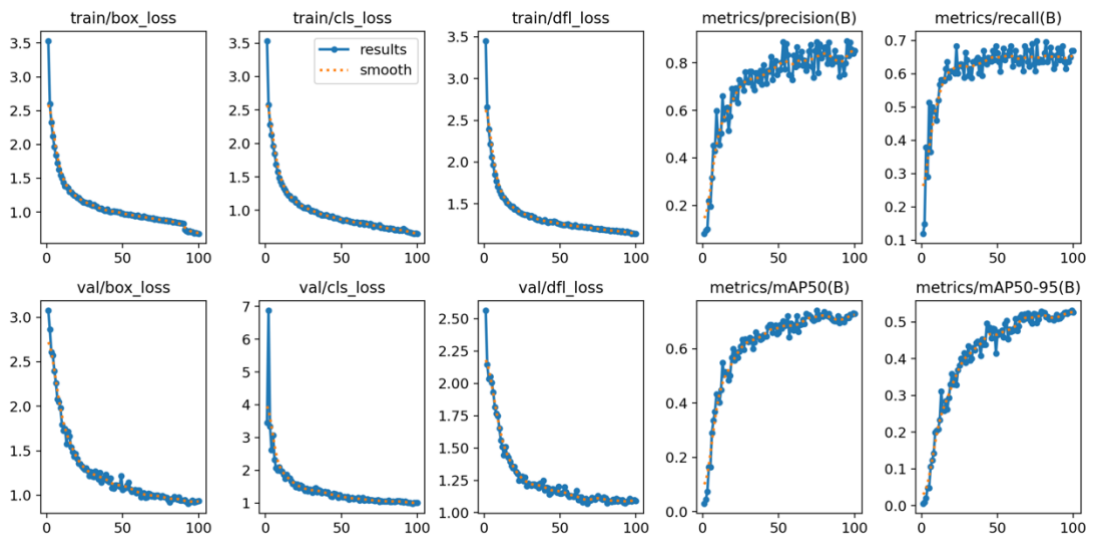


Figure 7.30 Result YOLOv11-s ADAMW

SGD

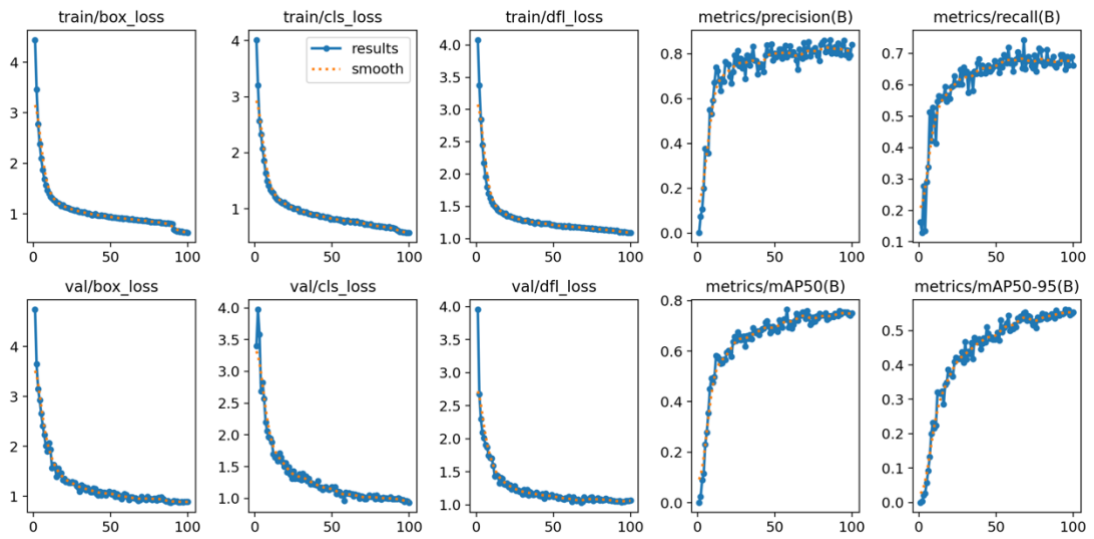


Figure 7.31 Result YOLOv11-s SGD

➤ YOLOv11-m

ADAMW

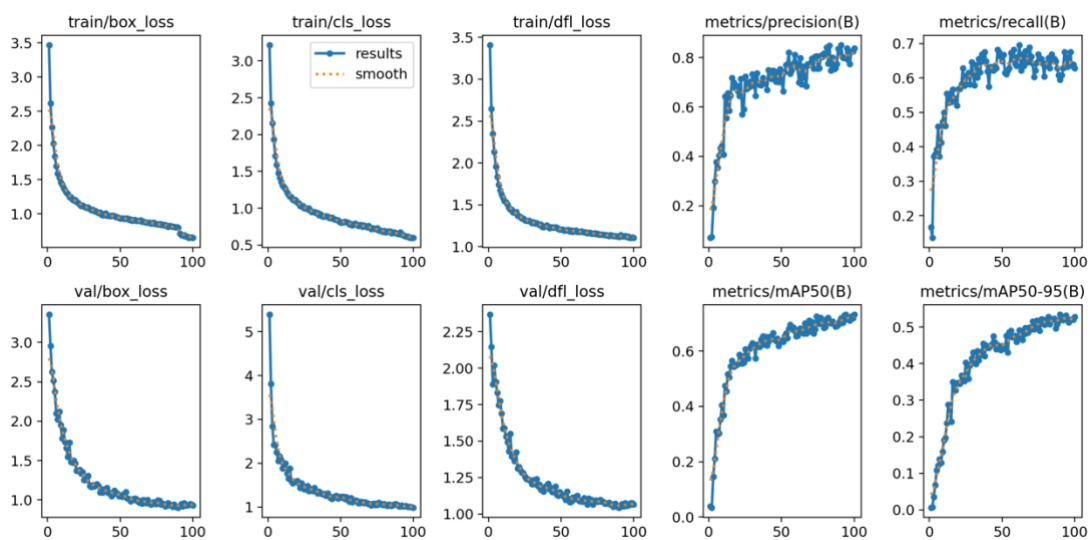


Figure 7.32 Result YOLOv11-m ADAMW

SGD

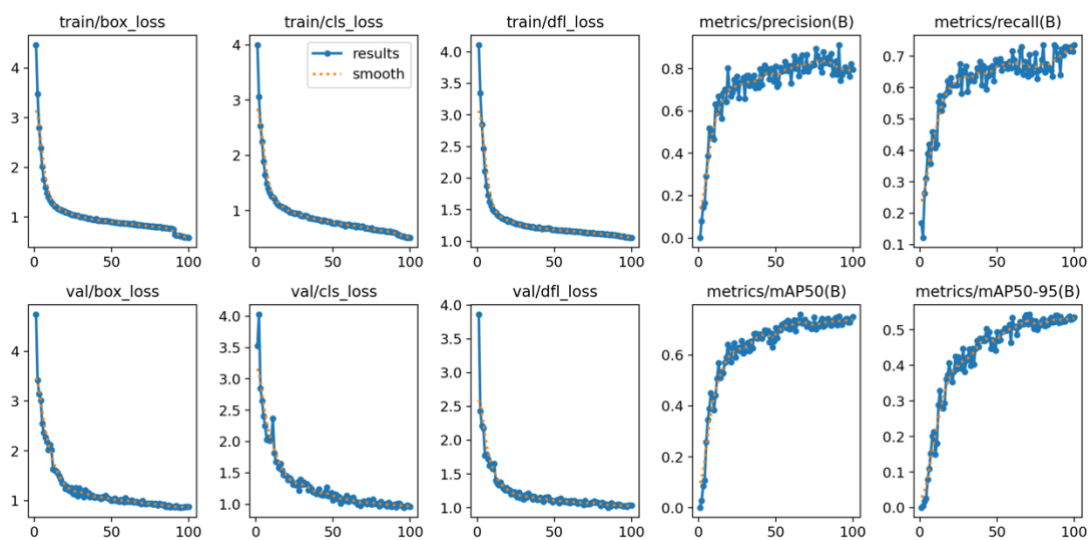


Figure 7.33 Result YOLOv11-m SGD

Chapter 8

CONCLUSION AND FUTURE ENHANCEMENTS

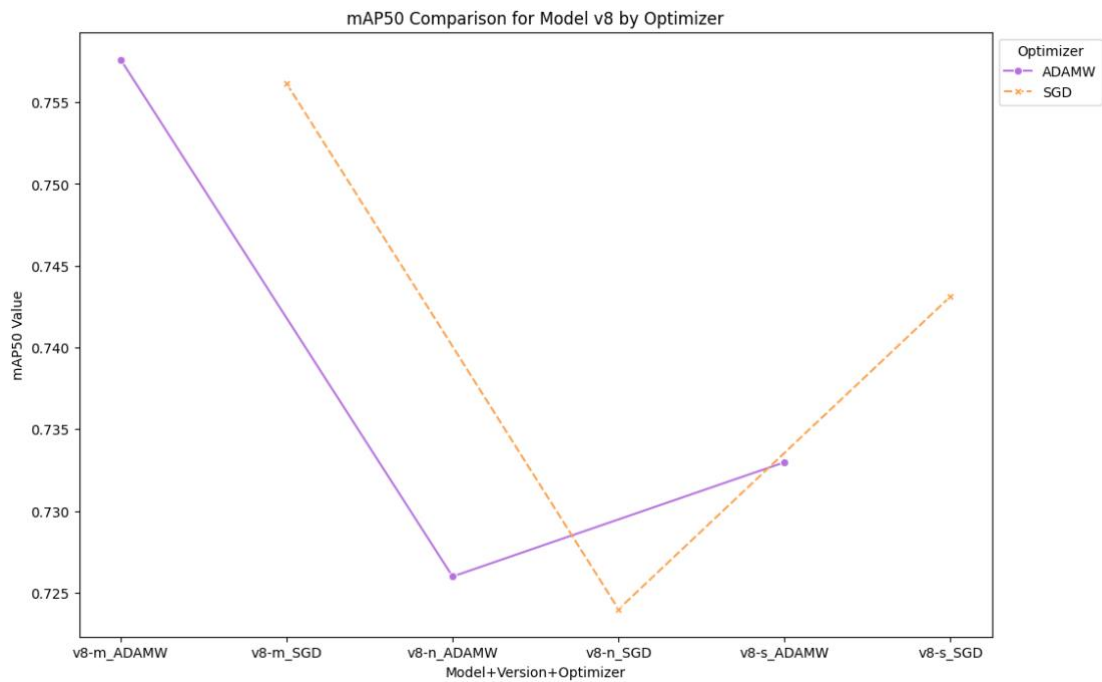


Figure 8.1 mAP Comparison graph YOLO v8

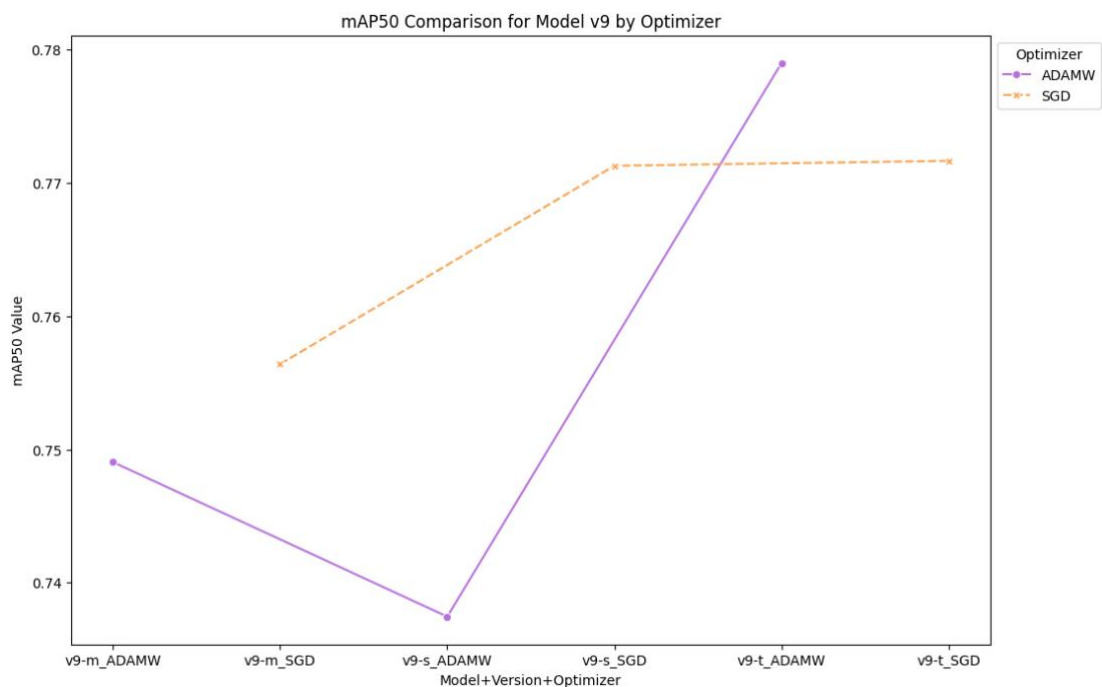


Figure 8.2 mAP Comparison graph YOLO v9

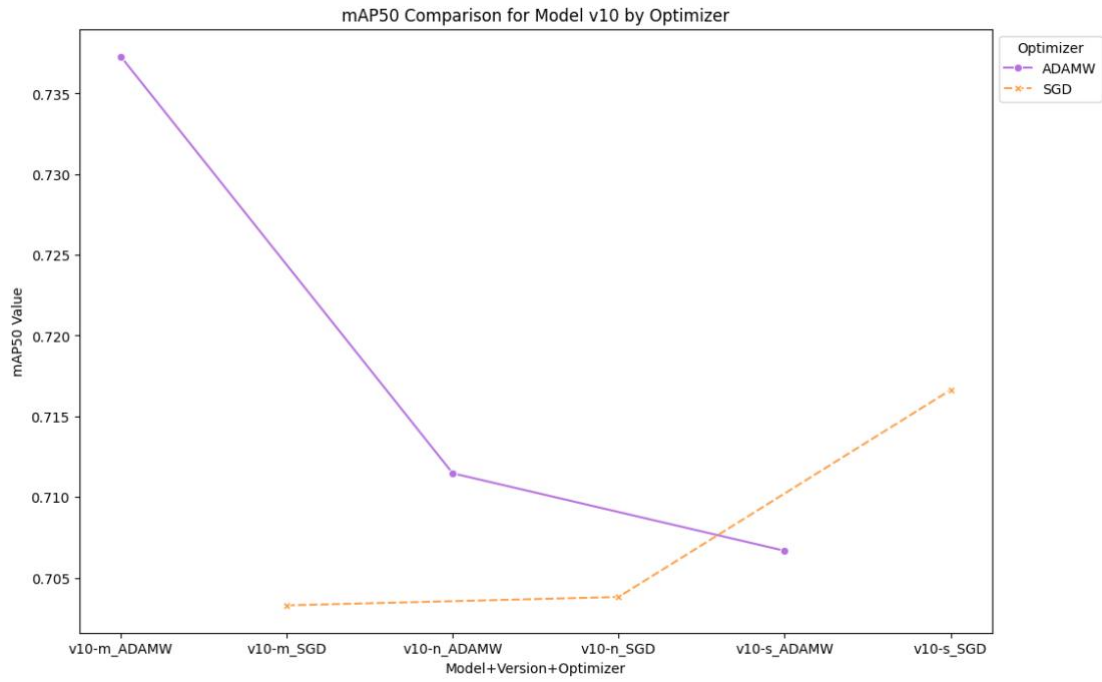


Figure 8.3 mAP Comparison graph YOLO v10

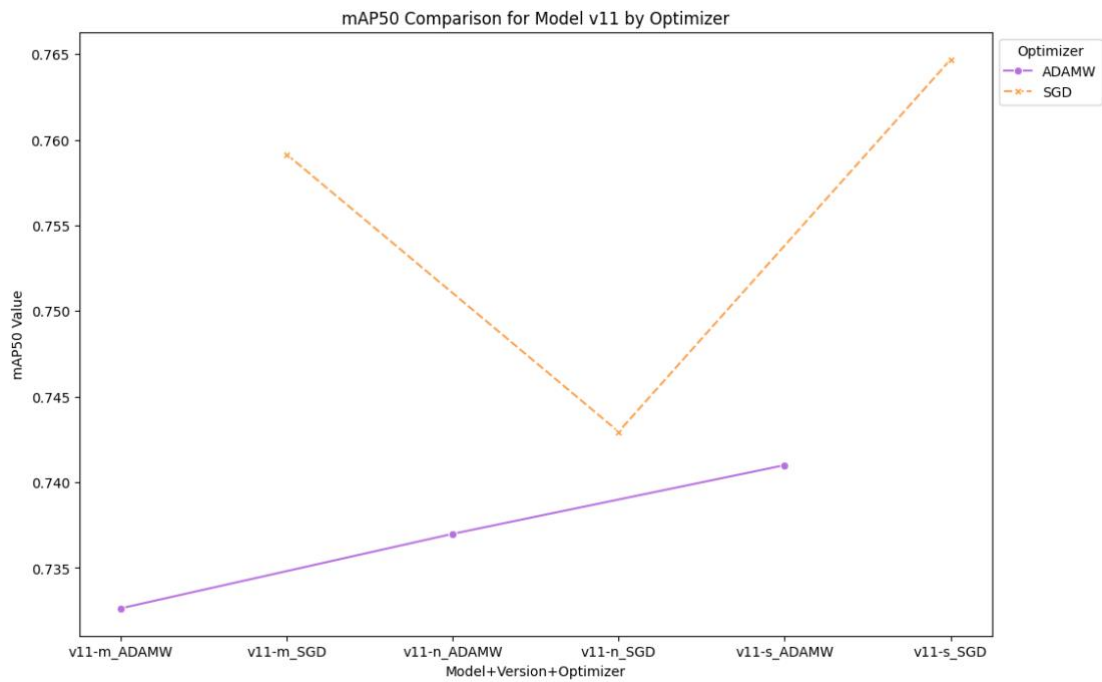


Figure 8.4 mAP Comparison graph YOLO v11

This study evaluated the performance of 24 YOLO model configurations — versions 8 through 11 (tiny, small, and medium variants) — using two different optimizers: **AdamW** and **SGD**. The comparison across two critical metrics, **mAP@0.5** and **mAP@0.5:0.95**, revealed meaningful insights into the role of optimizer choice and architecture design in fine-grained crack detection.

Among all configurations, **YOLOv9-tiny trained with AdamW** emerged as the top-performing model, achieving **0.779 mAP@0.5** and **0.572 mAP@0.5:0.95**, demonstrating excellent detection precision with a lightweight architecture. AdamW consistently outperformed SGD in several key models, particularly in **YOLOv9-t**, **YOLOv10-m**, and **YOLOv11-m**, due to its adaptive learning behavior and effective weight regularization. This allowed the models to better capture thin and irregular crack patterns.

However, **SGD showed remarkable stability and strong generalization**, especially in **YOLOv11-s** and **YOLOv9-s**, where it achieved competitive accuracy. Despite being structurally sound and training-stable, the **YOLOv10 series** did not surpass its counterparts in overall precision. Meanwhile, **YOLOv8**, as the oldest architecture among the set, consistently lagged in detection performance across all scales, indicating room for architectural advancement.

From a deployment perspective, **YOLOv9-tiny with AdamW** is an ideal candidate for **real-time, resource-constrained applications** such as drone-based inspections, on-site crack monitoring, and embedded surveillance. The results reaffirm that optimizer selection plays a crucial role not only in convergence but also in feature sensitivity, even in compact model variants.

Future Directions in YOLO-Based Crack Detection

Several opportunities exist for improving YOLO-based crack detection systems. These include:

- **Multi-modal approaches:** Combining YOLO with other detection techniques, such as thermal imaging, infrared or radar sensors, can help detect cracks that are invisible to the naked eye or obscured by environmental factors like lighting or surface texture.
- **Lightweight YOLO Models:** New research on model pruning and quantization could lead to lightweight YOLO versions that can run efficiently on edge devices such as drones, mobile phones, or even small UAVs.
- **Integrating UAVs for Large-Scale Inspections:** The integration of YOLO-based crack detection with UAVs (Unmanned Aerial Vehicles) has the potential to revolutionize infrastructure monitoring, allowing for quick, remote inspections of hard-to-reach areas without the need for scaffolding or manual inspections.
- **Incorporate Hybrid Optimizer Schedules:** Combine the strengths of AdamW and SGD through staged or cyclic training schedules to balance fast convergence and final generalization accuracy.

- **Extended Evaluation Metrics:** Include **inference speed (FPS)**, **energy consumption**, and **model size (MB)** to guide deployment on low-power devices like Raspberry Pi, Jetson Nano, and microcontroller-based systems.
 - **Domain Transfer Learning:** Test the models on **different types of surface defects** or datasets (e.g., rust, corrosion, fatigue) to evaluate generalization capability and robustness to domain shift.
 - **Model Compression Techniques:** Apply **quantization, pruning, and knowledge distillation** to compress high-performing models (e.g., YOLOv9-t_AdamW) for ultra-low-latency use cases.
 - **Hyperparameter Auto-Tuning:** Leverage tools like **Optuna** or **Ray Tune** for automated hyperparameter tuning tailored to each optimizer and model variant for maximum performance.
 - **Integration with IoT Pipelines:** Explore seamless integration of the best-performing models into IoT-based inspection frameworks, including real-time alert generation and cloud analytics.
 - **Visualization and Explainability:** Implement **Grad-CAM** or other explainability tools to interpret model focus regions, ensuring better trust and transparency in critical infrastructure applications.
-

Chapter 9

REFERENCES

1. Mohan, A.; Poobal, S. Crack detection using image processing: A critical review and analysis. *Alex. Eng. J.* 2018, 57, 787–798. [CrossRef]
2. Zaloshnja, E.; Miller, T.R. Cost of crashes related to road conditions, United States, 2006. In *Proceedings of the Annals of Advances in Automotive Medicine/Annual Scientific Conference*, Baltimore, Maryland, 5–7 October 2009; Association for the Advancement of Automotive Medicine: Chicago, IL, USA, 2009.
3. Zhu, Z.; German, S.; Brilakis, I. Visual retrieval of concrete crack properties for automated post-earthquake structural safety evaluation. *Autom. Constr.* 2011, 20, 874–883. [CrossRef]
4. Gkantou, M.; Muradov, M.; Kamaris, G.S.; Hashim, K.; Atherton, W.; Kot, P. Novel Electromagnetic Sensors Embedded in Reinforced Concrete Beams for Crack Detection. *Sensors* 2019, 19, 5175. [CrossRef]
5. Koshti, A.M. X-ray ray tracing simulation and flaw parameters for crack detection. In *Health Monitoring of Structural and Biological Systems XII*; SPIE: Denver, CO, USA, 2018.
6. Hosseini, Z.; Momayez, M.; Hassani, F.; Lévesque, D. Detection of inclined cracks inside concrete structures by ultrasonic SAFT. In *AIP Conference Proceedings*; American Institute of Physics: College Park, MD, USA, 2008.
7. Rodríguez-Martin, M.; Lagüela, S.; González-Aguilera, D.; Arias, P. Cooling analysis of welded materials for crack detection using infrared thermography. *Infrared Phys. Technol.* 2014, 67, 547–554. [CrossRef]
8. Mahler, D.S.; Kharoufa, Z.B.; Wong, E.K.; Shaw, L.G. Pavement distress analysis using image processing techniques. *Comput. Aided Civ. Infrastruct. Eng.* 1991, 6, 1–14. [CrossRef]
9. Oliveira, H.; Correia, P.L. Automatic road crack segmentation using entropy and image dynamic thresholding. In *Proceedings of the 17th European Signal Processing Conference*, Scotland, UK, 24–28 August 2009.
10. Huang, Y.; Xu, B. Automatic inspection of pavement cracking distress. *J. Electron. Imaging* 2006, 15, 013017. [CrossRef]
11. Otsu, N. A Threshold Selection Method from Gray-Level Histograms. *IEEE Trans. Syst. Man Cybern.* 1979, 9, 62–66. [CrossRef]
12. Liu, Y., W. Gao, T. Zhao, Z. Wang, and Z. Wang. A Rapid Bridge Crack Detection Method Based on Deep Learning. *Applied Sciences*, Vol. 13, No. 17, 2023, p. 9878. <https://doi.org/10.3390/app13179878>.
13. Su, H., D. B. Kamanda, T. Han, C. Guo, R. Li, Z. Liu, F. Su, and L. Shang. Enhanced YOLO v3 for Precise Detection of Apparent Damage on Bridges amidst Complex Backgrounds. *Scientific Reports*, Vol. 14, No. 1, 2024, p. 8627. <https://doi.org/10.1038/s41598-024-58707-2>.
14. Yang, Y., L. Li, G. Yao, H. Du, Y. Chen, and L. Wu. An Modified Intelligent Real-Time Crack Detection Method for Bridge Based on Improved Target Detection

- Algorithm and Transfer Learning. *Frontiers in Materials*, Vol. 11, 2024. <https://doi.org/10.3389/fmats.2024.1351938>.
15. Xiong, C., T. Zayed, and E. M. Abdelkader. A Novel YOLOv8-GAM-Wise-IoU Model for Automated Detection of Bridge Surface Cracks. *Construction and Building Materials*, Vol. 414, 2024, p. 135025. <https://doi.org/10.1016/j.conbuildmat.2024.135025>.
 16. Li, T., G. Liu, and S. Tan. Superficial Defect Detection for Concrete Bridges Using YOLOv8 with Attention Mechanism and Deformation Convolution. *Applied Sciences*, Vol. 14, No. 13, 2024, p.5497. <https://doi.org/10.3390/app14135497>.
 17. Matarneh, S., F. Elghaish, F. Pour Rahimian, E. Abdellatif, and S. Abrishami. Evaluation and Optimisation of Pre-Trained CNN Models for Asphalt Pavement Crack Detection and Classification. *Automation in Construction*, Vol. 160, 2024, p. 105297. <https://doi.org/10.1016/j.autcon.2024.105297>.
 18. Zhang, Q., S. Chen, Y. Wu, Z. Ji, F. Yan, S. Huang, and Y. Liu. Improved U-Net Network Asphalt Pavement Crack Detection Method. *PLOS ONE*, Vol. 19, No. 5, 2024, p. e0300679. <https://doi.org/10.1371/journal.pone.0300679>.
 19. Qin, Z., Z. Li, Z. Zhang, Y. Bao, G. Yu, Y. Peng, and J. Sun. ThunderNet: Towards Real-Time Generic Object Detection. 2019.
 20. Su, P., H. Han, M. Liu, T. Yang, and S. Liu. MOD-YOLO: Rethinking the YOLO Architecture at the Level of Feature Information and Applying It to Crack Detection. *Expert Systems with Applications*, Vol. 237, 2024, p. 121346. <https://doi.org/10.1016/j.eswa.2023.121346>.
 21. Xiong, C., T. Zayed, X. Jiang, G. Alfalah, and E. M. Abdelkader. A Novel Model for Instance Segmentation and Quantification of Bridge Surface Cracks—The YOLOv8-AFPN-MPD-IoU. *Sensors*, Vol. 24, No. 13, 2024, p. 4288. <https://doi.org/10.3390/s24134288>.
 22. Tang, G., C. Yin, X. Zhang, X. Liu, and S. Li. Crack-Detection Method for Asphalt Pavement Based on the Improved YOLOv5. *Journal of Performance of Constructed Facilities*, Vol. 38, No. 2, 2024. <https://doi.org/10.1061/JPCFEV.CFENG-4615>.
 23. Liu, Q., and Z. Liu. Asphalt Pavement Crack Detection Based on Improved YOLOv5s Algorithm. 2024.
 24. Tran, T. S., S. D. Nguyen, H. J. Lee, and V. P. Tran. Advanced Crack Detection and Segmentation on Bridge Decks Using Deep Learning. *Construction and Building Materials*, Vol. 400, 2023, p. 132839. <https://doi.org/10.1016/j.conbuildmat.2023.132839>.
 25. Dwyer, B. , N. J. , H. T. , et. al. Roboflow (Version 1.0) [Software]. *Computer Vision*.
 26. Glenn Jocher, A. C. J. Q. Ultralytics YOLOv8. <https://github.com/ultralytics/ultralytics>. Accessed Nov. 18, 2024.
 27. Li, C., L. Li, H. Jiang, K. Weng, Y. Geng, L. Li, Z. Ke, Q. Li, M. Cheng, W. Nie, Y. Li, B. Zhang, Y. Liang, L. Zhou, X. Xu, X. Chu, X. Wei, and X. Wei. YOLOv6: A Single-Stage Object Detection Framework for Industrial Applications. 2022.
 28. Bochkovskiy, A., C.-Y. Wang, and H.-Y. M. Liao. YOLOv4: Optimal Speed and Accuracy of Object Detection. 2020.

29. Wang, C.-Y., A. Bochkovskiy, and H.-Y. M. Liao. YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors. 2022.
30. Redmon, J., and A. Farhadi. YOLOv3: An Incremental Improvement. 2018.
31. Redmon, J., and A. Farhadi. YOLO9000: Better, Faster, Stronger. 2017.
32. Ramos, L., E. Casas, E. Bendek, C. Romero, and F. Rivas-Echeverría. Hyperparameter Optimization of YOLOv8 for Smoke and Wildfire Detection: Implications for Agricultural and Environmental Safety. *Artificial Intelligence in Agriculture*, Vol. 12, 2024, pp. 109–126. <https://doi.org/10.1016/j.aiia.2024.05.003>.
33. Chen, X., C. Yuan, and Y. Liu. Research On Bird Nest Detection Method of Transmission Line Based on Yolov8. *International Journal of Computer Science and Information Technology*, Vol. 3, No. 2, 2024, pp. 336–344. <https://doi.org/10.62051/ijcsit.v3n2.36>.
34. Robbins, H., and S. Monro. A Stochastic Approximation Method. 1951.
35. Kingma, D. P., and J. Lei Ba. ADAM: A Method for Stochastic Optimization. Presented at 3rd International Conference for Learning Representations, San Diego, 2015.
36. Loshchilov, I., and F. Hutter. Decoupled Weight Decay Regularization. Presented at the 9th International Conference for Learning Representations , New Orleans, 2019.
37. Kurbiel, T., and S. Khaleghian. Training of Deep Neural Networks Based on Distance Measures Using RMSProp. 2017.
38. Liu, L., G. Tech, P. He, W. Chen, X. Liu, J. Gao, and J. Han. On the Variance of the Adaptive Learning Rate and Beyond. Presented at the 10th International Conference for Learning Representations, 2020.
39. Timothy Dozat. Incorporating Nesterov Momentum into Adam. Presented at 4th International Conference for Learning Representations, San Juan, 2016.
40. Czitrom, V. One-Factor-at-a-Time versus Designed Experiments. *The American Statistician*, Vol. 53, No. 2, 1999, pp. 126–131. <https://doi.org/10.1080/00031305.1999.10474445>.
41. Taffese, W. Z., and L. Espinosa-Leal. Prediction of Chloride Resistance Level of Concrete Using Machine Learning for Durability and Service Life Assessment of Building Structures. *Journal of Building Engineering*, Vol. 60, 2022, p. 105146. <https://doi.org/10.1016/j.jobe.2022.105146>.
42. Karras, T., S. Laine, and T. Aila. A Style-Based Generator Architecture for Generative Adversarial Networks. 2018.
43. Dosovitskiy, A., J. T. Springenberg, M. Tatarchenko, and T. Brox. Learning to Generate Chairs, Tables and Cars with Convolutional Networks. 2014.
44. Kingma, D. P., and M. Welling. Auto-Encoding Variational Bayes. 2013.
43. Li, R., J. Yu, F. Li, R. Yang, Y. Wang, and Z. Peng. Automatic Bridge Crack Detection Using Unmanned Aerial Vehicle and Faster R-CNN. *Construction and Building Materials*, Vol. 362, 2023, p. 129659. <https://doi.org/10.1016/j.conbuildmat.2022.129659>.

44. Lu, G., X. He, Q. Wang, F. Shao, J. Wang, and Q. Jiang. Bridge Crack Detection Based on Improved Single Shot Multi-Box Detector. PLOS ONE, Vol. 17, No. 10, 2022, p. e0275538. <https://doi.org/10.1371/journal.pone.0275538>.
45. He, M., and T. L. Lau. CrackHAM: A Novel Automatic Crack Detection Network Based on U-Net for Asphalt Pavement. IEEE Access, Vol. 12, 2024, pp. 12655–12666. <https://doi.org/10.1109/ACCESS.2024.3353729>.
46. Zoubir, H., M. Rguig, M. El Aroussi, R. Saadane, and A. Chehri. Pixel-Level Concrete Bridge Crack Detection Using Convolutional Neural Networks, Gabor Filters, and Attention Mechanisms. Engineering Structures, Vol. 314, 2024, p. 118343. <https://doi.org/10.1016/j.engstruct.2024.118343>.
47. Redmon, J., S. Divvala, R. Girshick, and A. Farhadi. You Only Look Once: Unified, Real-Time Object Detection. 2016.
48. Gong, H., J. Tešić, J. Tao, X. Luo, and F. Wang. Automated Pavement Crack Detection with Deep Learning Methods: What Are the Main Factors and How to Improve the Performance? Transportation Research Record: Journal of the Transportation Research Board, Vol. 2677, No. 10, 2023, pp. 311–323. <https://doi.org/10.1177/03611981231161358>.
49. Zhang, J., S. Qian, and C. Tan. Automated Bridge Crack Detection Method Based on Lightweight Vision Models. Complex & Intelligent Systems, Vol. 9, No. 2, 2023, pp. 1639–1652. <https://doi.org/10.1007/s40747-022-00876-6>.
50. Zhang, J.; Jing, J.; Lu, P.; Song, S. Improved MobileNetV2-SSDLite for automatic fabric defect detection system based on cloud-edge computing. Measurement 2022, 201, 111665. [CrossRef]
51. Zhang, J.; Jing, J.; Lu, P.; Song, S. Concrete crack segmentation based on UAV-enabled edge computing. Neurocomputing 2022, 485, 233–241.
52. Tang, W.; Yang, Q.; Hu, X.; Yan, W. Deep learning-based linear defects detection system for large-scale photovoltaic plants based on an edge-cloud computing infrastructure. Sol. Energy 2022, 231, 527–535. [CrossRef]
53. Wu, Y.; Guo, H.; Chakraborty, C.; Khosravi, M.; Berretti, S.; Wan, S. Edge Computing Driven Low-Light Image Dynamic Enhancement for Object Detection. IEEE Trans. Netw. Sci. Eng. 2022. [CrossRef]
54. Zhu, T.; Kuang, L.; Daniels, J.; Herrero, P.; Li, K.; Georgiou, P. IoMT-Enabled Real-time Blood Glucose Prediction with Deep Learning and Edge Computing. IEEE Internet Things J. 2022, 10, 3706–3719. [CrossRef]
55. Zhang, J.; Qian, S.; Tan, C. Automated bridge surface crack detection and segmentation using computer vision-based deep learning model. Eng. Appl. Artif. Intell. 2022, 115, 105225. [CrossRef]
56. Tzutalin, D. LabelImg. GitHub Repository. June 2015. Available online: <https://pypi.org/project/labelImg/> (accessed on 4 July 2023).

APPENDIX A – SAMPLE CODE

```

9  nc: 80 # number of classes
10
11 # GELAN backbone
12 backbone:
13   - [-1, 1, Conv, [16, 3, 2]] # 0-P1/2
14   - [-1, 1, Conv, [32, 3, 2]] # 1-P2/4
15   - [-1, 1, ELAN1, [32, 32, 16]] # 2
16   - [-1, 1, AConv, [64]] # 3-P3/8
17   - [-1, 1, RepNCSPELAN4, [64, 64, 32, 3]] # 4
18   - [-1, 1, AConv, [96]] # 5-P4/16
19   - [-1, 1, RepNCSPELAN4, [96, 96, 48, 3]] # 6
20   - [-1, 1, AConv, [128]] # 7-P5/32
21   - [-1, 1, RepNCSPELAN4, [128, 128, 64, 3]] # 8
22   - [-1, 1, SPPELAN, [128, 64]] # 9
23
24 head:
25   - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
26   - [[-1, 6], 1, Concat, [1]] # cat backbone P4
27   - [-1, 1, RepNCSPELAN4, [96, 96, 48, 3]] # 12
28
29   - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
30   - [[-1, 4], 1, Concat, [1]] # cat backbone P3
31   - [-1, 1, RepNCSPELAN4, [64, 64, 32, 3]] # 15
32
33   - [-1, 1, AConv, [48]]
34   - [[-1, 12], 1, Concat, [1]] # cat head P4
35   - [-1, 1, RepNCSPELAN4, [96, 96, 48, 3]] # 18 (P4/16-medium)
36
37   - [-1, 1, AConv, [64]]
38   - [[-1, 9], 1, Concat, [1]] # cat head P5
39   - [-1, 1, RepNCSPELAN4, [128, 128, 64, 3]] # 21 (P5/32-large)
40
41   - [[15, 18, 21], 1, Detect, [nc]] # Detect(P3, P4, P5)

```

from	n	params	module	arguments
-1	1	928	ultralytics.nn.modules.conv.Conv	[3, 32, 3, 2]
-1	1	18560	ultralytics.nn.modules.conv.Conv	[32, 64, 3, 2]
-1	1	29056	ultralytics.nn.modules.block.C2f	[64, 64, 1, True]
-1	1	73984	ultralytics.nn.modules.conv.Conv	[64, 128, 3, 2]
-1	2	197632	ultralytics.nn.modules.block.C2f	[128, 128, 2, True]
-1	1	36096	ultralytics.nn.modules.block.SCDwn	[128, 256, 3, 2]
-1	2	788480	ultralytics.nn.modules.block.C2f	[256, 256, 2, True]
-1	1	137728	ultralytics.nn.modules.block.SCDwn	[256, 512, 3, 2]
-1	1	958464	ultralytics.nn.modules.block.C2fCIB	[512, 512, 1, True, True]
-1	1	656896	ultralytics.nn.modules.block.SPPF	[512, 512, 5]
-1	1	990976	ultralytics.nn.modules.block.PSA	[512, 512]
-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
[-1, 6]	1	0	ultralytics.nn.modules.conv.Concat	[1]
-1	1	591360	ultralytics.nn.modules.block.C2f	[768, 256, 1]
-1	1	0	torch.nn.modules.upsampling.Upsample	[None, 2, 'nearest']
[-1, 4]	1	0	ultralytics.nn.modules.conv.Concat	[1]
-1	1	148224	ultralytics.nn.modules.block.C2f	[384, 128, 1]
-1	1	147712	ultralytics.nn.modules.conv.Conv	[128, 128, 3, 2]
[-1, 13]	1	0	ultralytics.nn.modules.conv.Concat	[1]
-1	1	493056	ultralytics.nn.modules.block.C2f	[384, 256, 1]
-1	1	68864	ultralytics.nn.modules.block.SCDwn	[256, 256, 3, 2]
[-1, 10]	1	0	ultralytics.nn.modules.conv.Concat	[1]
-1	1	1089536	ultralytics.nn.modules.block.C2fCIB	[768, 512, 1, True, True]
[16, 19, 22]	1	1639574	ultralytics.nn.modules.head.v10Detect	[1, [128, 256, 512]]

Summary: 234 layers, 8,067,126 parameters, 8,067,110 gradients, 24.8 GFLOPs

```

7 # Parameters
8 nc: 80 # number of classes
9 scales: # model compound scaling constants, i.e. 'model=yolo11n.yaml' will call yolo11.yaml with scale 'n'
10 # [depth, width, max_channels]
11 n: [0.50, 0.25, 1024] # summary: 181 layers, 2624080 parameters, 2624064 gradients, 6.6 GFLOPs
12 s: [0.50, 0.50, 1024] # summary: 181 layers, 9458752 parameters, 9458736 gradients, 21.7 GFLOPs
13 m: [0.50, 1.00, 512] # summary: 231 layers, 20114688 parameters, 20114672 gradients, 68.5 GFLOPs
14 l: [1.00, 1.00, 512] # summary: 357 layers, 25372160 parameters, 25372144 gradients, 87.6 GFLOPs
15 x: [1.00, 1.50, 512] # summary: 357 layers, 56966176 parameters, 56966160 gradients, 196.0 GFLOPs
16
17 # YOLO11n backbone
18 backbone:
19 # [from, repeats, module, args]
20 - [-1, 1, Conv, [64, 3, 2]] # 0-P1/2
21 - [-1, 1, Conv, [128, 3, 2]] # 1-P2/4
22 - [-1, 2, C3k2, [256, False, 0.25]]
23 - [-1, 1, Conv, [256, 3, 2]] # 3-P3/8
24 - [-1, 2, C3k2, [512, False, 0.25]]
25 - [-1, 1, Conv, [512, 3, 2]] # 5-P4/16
26 - [-1, 2, C3k2, [512, True]]
27 - [-1, 1, Conv, [1024, 3, 2]] # 7-P5/32
28 - [-1, 2, C3k2, [1024, True]]
29 - [-1, 1, SPPF, [1024, 5]] # 9
30 - [-1, 2, C2PSA, [1024]] # 10
31
32 # YOLO11n head
33 head:
34 - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
35 - [[-1, 6], 1, Concat, [1]] # cat backbone P4
36 - [-1, 2, C3k2, [512, False]] # 13
37
38 - [-1, 1, nn.Upsample, [None, 2, "nearest"]]
39 - [[-1, 4], 1, Concat, [1]] # cat backbone P3
40 - [-1, 2, C3k2, [256, False]] # 16 (P3/8-small)
41
42 - [-1, 1, Conv, [256, 3, 2]]
43 - [[-1, 13], 1, Concat, [1]] # cat head P4
44 - [-1, 2, C3k2, [512, False]] # 19 (P4/16-medium)
45
46 - [-1, 1, Conv, [512, 3, 2]]
47 - [[-1, 10], 1, Concat, [1]] # cat head P5
48 - [-1, 2, C3k2, [1024, True]] # 22 (P5/32-large)
49
50 - [[16, 19, 22], 1, Detect, [nc]] # Detect(P3, P4, P5)

```

```

AMP: running Automatic Mixed Precision (AMP) checks...
Downloading https://github.com/ultralytics/assets/releases/download/v8.3.0/yolo11n.pt to 'yolo11n.pt'
... 100% 5.35M/5.35M [00:00<00:00, 104MB/s]
AMP: checks passed ✓
train: Scanning /content/datasets/crack-2/train/labels... 3717 images, 0 backgrounds, 0 corrupt: 100%
train: New cache created: /content/datasets/crack-2/train/labels.cache
augmentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7)), ToGray(p=0.01)
val: Scanning /content/datasets/crack-2/test/labels... 112 images, 0 backgrounds, 0 corrupt: 100% 112
val: New cache created: /content/datasets/crack-2/test/labels.cache
Plotting labels to /content/ultralytics/runs/detect/train/labels.jpg...
optimizer: 'optimizer=auto' found, ignoring 'lr=0.01' and 'momentum=0.937' and determining best 'opt
optimizer: AdamW(lr=0.002, momentum=0.9) with parameter groups 99 weight(decay=0.0), 112 weight(decay
TensorBoard: model graph visualization added ✓
Image sizes 640 train, 640 val
Using 2 dataloader workers
Logging results to /content/ultralytics/runs/detect/train
Starting training for 100 epochs...

```

Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
1/100	4.9G	6.96	11.14	7.026	6	640: 100% 233/233 [01:37<00:00]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 4/4 [0.0101 0.00169]
	all	112	148	0.0468	0.0676	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
2/100	5.71G	5.066	5.841	5.407	12	640: 100% 233/233 [01:32<00:00]
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 4/4 [0.0502 0.0127]
	all	112	148	0.0804	0.162	
Epoch	GPU_mem	box_loss	cls_loss	dfl_loss	Instances	Size
3/100	5.71G	4.646	5.215	4.92	34	640: 36% 83/233 [00:32<00:00]